

深度技术教材 · V1.0

从零到一实现 ChatGPT

BUILDING CHATGPT FROM
SCRATCH

基于 nanochat 的现代 LLM 全栈实现



本课程以 [karpathy/nanochat](#) 为代码底座，系统讲解现代 LLM 工程的全栈实现。覆盖 **11** 章内容，包含完整公式推导、19 张技术示意图、核心脚本逐行精读，以及 12 个可在 d12 小模型上完成的扩展实验。

代码底座

nanochat

复现成本

~\$48

训练时长

1.5 小时

代码总量

7K 行

博学实验室 · freeLearnLab

2026 年版

目录

第一章 导论：为什么是 nanochat	9
1.1 从一个具体复现实验说起	9
1.2 nanochat 的定位与历史	9
1.3 仓库文件树与脚本地图	10
1.4 端到端训练流水线：speedrun.sh 拆解	11
1.5 课程如何使用本书	12
1.6 小结与下一章预告	13
第二章 分词器：BPE 的工程实现	13
2.1 为什么需要分词器：从一个具体例子说起	13
2.2 BPE 算法的完整推导：用 "low lower lowest" 走一遍	14
2.2.1 训练阶段：从字节到合并规则	14
2.2.2 推理阶段：贪心切分的等价性	16
2.3 nanochat 的 split pattern：为什么不直接合并	16
2.4 特殊 token：对话格式如何用 token 表示	17
2.5 代码精读：tokenizer.py 与 render_conversation	18
2.5.1 输入对话与期望输出	18
2.5.2 代码精读：如何把字典变成 token_ids + loss_mask	19
2.5.3 走一遍：实际 token_ids 与 loss_mask 的演化	19
2.6 压缩率评估：tok_eval.py 与 BPB 指标	20
2.6.1 同样文本，不同 tokenizer，压缩率差异有多大	20
2.6.2 BPB 公式与跨词表可比性	20

2.7 中文分词的工程挑战	21
2.7.1 字节级 BPE 如何处理中文：用 "你好，世界！" 走一遍	21
2.7.2 中文压缩率对比：英文 vs 中文 vs 混合 tokenizer	21
2.7.3 中文 LLM 的词表设计权衡	22
2.7.4 在 nanochat 中加入中文支持的最小改动	22
2.8 小结	23
第三章 数据集工程：从 FineWeb 到 ClimbMix	23
3.1 数据集是 LLM 工程的"真正瓶颈"：从一份网页爬取说起	23
3.2 预训练数据集的演进史：每一代解决了上一代的什么问题	24
3.2.1 第一代：Common Crawl 与 C4	25
3.2.2 第二代：Pile 与 RedPajama	25
3.2.3 第三代：FineWeb 与 DCLM-baseline	25
3.2.4 第四代：FineWeb-Edu 与 ClimbMix	26
3.3 dataset.py：分片下载与并行化	26
3.3.1 为什么用 parquet 分片	27
3.3.2 多进程并行下载	27
3.3.3 断点续传与重试	27
3.4 dev/repackage_data_reference.py：从原始数据到分片	27
3.5 数据集 Scaling Laws：Chinchilla 与"多 epoch"	28
3.6 SFT 数据集：SmolTalk、MMLU、GSM8K 的来源与构造	29
3.6.1 SmolTalk：HuggingFace 的"对话数据基座"	29
3.6.2 MMLU：多学科多选题的"知识测试"	29
3.6.3 GSM8K：小学数学题的"推理能力测试"	29
3.7 数据污染：评估集泄漏问题	30

3.7.1 nanochat 的三层防污染设计	30
3.8 数据合规与版权	30
3.9 中文预训练数据集	30
3.9.1 中文预训练数据集的现状	31
3.9.2 中文数据集 vs 英文数据集的核心差异	31
3.9.3 在 nanochat 中加入中文预训练数据	32
3.9.4 中文 SFT 数据集	32
3.10 数据集工程的关键决策清单	33
3.11 小结	33
第四章 模型架构：现代 Transformer 的所有细节	34
4.1 一个 token batch 如何流过整个模型：贯穿示例	34
4.2 RoPE 旋转位置编码：从问题到代码的逐步推导	35
4.2.1 起点：绝对位置编码的问题	35
4.2.2 第一步：从二维情形的几何直觉开始	35
4.2.3 第二步：为什么旋转能让 attention 只依赖相对位置	36
4.2.4 第三步：从二维扩展到多维（多频分组）	37
4.2.5 第四步：从公式到代码——为什么是"前半后半"切分	37
4.2.6 第五步：代码精读——apply_rotary_emb 每行在做什么	38
4.2.7 第六步：用一个具体位置走完整个流程	39
4.3 QK-Norm 与 $1.2\times$ sharpening	40
4.4 GQA 分组查询注意力	40
4.5 ReLU^2 激活函数：被忽视的现代架构差异	41
4.6 Value Embedding (ResFormer)	42
4.7 smear、backout 与 per-layer λ	42

4.7.1 smear: 廉价 bigram 信号	43
4.7.2 backout: 去除低级特征	43
4.7.3 per-layer λ : 残差与初始嵌入的混合权重	43
4.8 Logit softcap: 平滑限幅	43
4.9 整体 Block 结构与代码精读	43
4.10 滑窗注意力 SSSL 模式	45
4.11 小结	45
第五章 预训练: Scaling Laws 与 Muon 优化器	46
5.1 从一段真实训练日志说起	46
5.2 Scaling Laws 的演进: 从 Kaplan 到 Power Lines	46
5.2 nanochat 的 scaling laws 自动推导	47
5.2.1 模型维度推导	47
5.2.2 Batch size 推导 (Power Lines)	47
5.2.3 Learning rate 推导	48
5.2.4 Weight decay 推导 (T_epoch 框架)	48
5.3 Muon 优化器: 从 AdamW 到正交化更新	48
5.3.1 Muon 的核心思想	48
5.3.2 数学推导: Polar Express 迭代	49
5.3.3 MuonAdamW 混合优化器	49
5.4 ZeRO-2 风格的分布式通信	50
5.5 WSD 学习率调度	50
5.6 混合精度训练: BF16 与 FP8	51
5.7 base_train.py 整体流程	52
5.8 小结	52

第六章 数据加载与 SFT 监督微调 52

6.1 从三个文档到一行 2048 token：贯穿示例	52
6.1.1 算法伪代码	53
6.1.2 DDP 分片与断点续训	54
6.2 SFT 数据打包：对话不能用裁剪	54
6.3 数据混合：SmolTalk + MMLU + GSM8K	55
6.4 SFT 训练流程与超参继承	55
6.5 SFT 评估指标：BPB + ChatCORE	56
6.6 小结	56

第七章 强化学习：从 PPO 到 GRPO 的简化路径 57

7.1 一道 GSM8K 题如何走完整个 RL 循环	57
7.2 RLHF 的谱系：从 InstructGPT 到 GRPO	57
7.3 PPO 的完整公式	58
7.4 GRPO：删除 critic 的简化	58
7.5 nanochat 的"GRPO in quotes"	59
7.6 chat_rl.py 代码精读	59
7.5.1 Rollout 采样	59
7.5.2 reward 解析：GSM8K 答案格式	60
7.7 为什么 RL 对数学推理特别有效	61
7.8 小结	61

第八章 推理引擎：KV Cache、Flash Attention 与工具调用 61

8.1 一段 prompt 从输入到流式生成：贯穿示例	61
8.2 推理的两阶段：Prefill 与 Decode	62

8.2 KVCache 的实现细节	63
8.3 Flash Attention 3: 从算法到工程	63
8.4 工具调用状态机	64
8.5 use_calculator: 沙箱化 Python 执行	65
8.6 推理 roofline: MFU 与 MBU	66
8.7 chat_cli.py: 交互式 REPL	67
8.8 小结	67
第九章 评估体系: CORE、ChatCORE、BPB 与 pass@k	68
9.1 一道 MMLU 题如何走完整个评估流程	68
9.2 LLM 评估方法学的演进	69
9.3 BPB: 跨词表可比的 perplexity	69
9.4 DCLM CORE: ICL few-shot 评估	69
9.3.1 few-shot prompt 设计	69
9.3.2 centered accuracy	70
9.5 ChatCORE: 对话模型的 5 任务	70
9.6 categorical vs generative 评估	71
9.7 pass@k: RL 评估	71
9.8 HumanEval 沙箱执行	71
9.9 评估与训练分布的对齐问题	72
9.10 小结	72
第十章 扩展实验: 四类动手实践	73
10.1 实验准备: d12 小模型与单 GPU	73
10.2 架构与优化器实验	73

10.2.1 实验 1: 词表大小消融	73
10.2.2 实验 2: LR schedule 对比	73
10.2.3 实验 3: Muon vs AdamW 对比	74
10.3 训练策略实验	75
10.3.1 实验 4: DPO vs 简化 GRPO 对比	75
10.3.2 实验 5: SFT 数据混合比例扫描	75
10.3.3 实验 6: 添加新能力 (strawberry 数 r)	75
10.4 推理优化实验	76
10.4.1 实验 7: int8/int4 量化推理	76
10.4.2 实验 8: batch 扫描与 KV cache 显存	76
10.4.3 实验 9: KV cache 压缩	77
10.5 工程优化实验	77
10.5.1 实验 10: tokenizer split pattern 消融	77
10.5.2 实验 11: 滑窗模式消融	77
10.5.3 实验 12: FP8 vs BF16 训练对比	78
10.6 实验记录与对比	78
10.7 实验心得	79

第十一章 论文阅读路径与进一步学习 79

11.1 一份推荐的 8 周论文阅读计划	79
11.2 主题组 1: Scaling Laws 基础	80
11.3 主题组 2: 优化器	80
11.4 主题组 3: 架构	81
11.5 主题组 4: RL 与评估	81
11.6 进一步学习路径	82

11.6.1 从 nanochat 到 torchtitan	82
11.6.2 从 speedrun 到 leaderboard	82
11.6.3 从 nanochat 到 DeepSeek/Llama	82
11.6.4 进阶话题	82
11.7 课程总结	83

第一章 导论：为什么是 nanochat

1.1 从一个具体复现实验说起

在讲 nanochat 项目定位之前，先看一个具体复现实验。假设你想在 8 张 H100 GPU 上用 nanochat 复现一个 GPT-2 (1.6B) 能力的模型。从空目录开始到能跟模型对话，整个流程是 11 步、约 1.5 小时、约 48 美元。这是本章要回答的第一个问题：为什么这件事在 2025 年变得可能？

```
# 复现 GPT-2 (1.6B) 能力模型的完整流程 (speedrun.sh)

# 步骤 1-5: 准备阶段 (约 5 分钟)
git clone https://github.com/karpathy/nanochat && cd nanochat
uv sync --extra gpu # 装依赖
python -m nanochat.dataset -n 8 # 下载 8 分片训 tokenizer
python -m nanochat.dataset -n 170 & # 后台下载 170 分片预训练用
python -m scripts.tok_train # 训 tokenizer (约 5 分钟)

# 步骤 6-9: 训练阶段 (约 80 分钟, 成本 $40)
torchrun --nproc_per_node=8 -m scripts.base_train \
    -- --depth=24 --target-param-data-ratio=8 --fp8 # 预训练 (约 60 分钟)
torchrun --nproc_per_node=8 -m scripts.base_eval # base 评估
torchrun --nproc_per_node=8 -m scripts.chat_sft # SFT (约 15 分钟)
torchrun --nproc_per_node=8 -m scripts.chat_eval -i sft # chat 评估

# 步骤 10-11: 对话阶段 (约 10 秒)
python -m scripts.chat_cli -p "Why is the sky blue?"
# → "Rayleigh scattering causes shorter wavelengths of light..."

# 总成本: ~$48 (H100 spot ~$1.5/小时 × 8 × 1.5h + 存储)
# 对比: 2019 年 OpenAI 训 GPT-2 (1.5B) 成本 ~$43,000
# 6 年时间, 同样能力成本下降约 900 倍
```

这 11 步背后是过去 6 年 LLM 工程的浓缩：Scaling Laws 理论成熟让“单旋钮调参”成为可能、Muon 优化器让收敛速度提升 2×、FP8 训练让吞吐提升 1.5×、Flash Attention 3 让长序列计算提速、ClimbMix 等高质量数据集让“小数据 + 多 epoch”可行。nanochat 项目把这些工程进步浓缩到一个可读、可改、可 fork 的极简代码库。

1.2 nanochat 的定位与历史

回到项目定位。2025 年 10 月，Andrej Karpathy 在 GitHub 上发布了 nanochat 项目，定位为“the minimal full-stack ChatGPT clone”——最小化全栈 ChatGPT 复刻。这位前 OpenAI 创始成员、前 Tesla AI 总监，曾以 nanoGPT、llm.c 等项目推动了“用最短代码讲清 LLM 工程原理”的教学范式，而 nanochat 可以视作这一范式的集大成之作：单仓库、单 GPU 节点、约 7000 行 Python，完整覆盖了从分词器训练、预训练、SFT 监督微调、RL 强化学习、评估、推理到 CLI 对话的端到端流程。

与作者前作 nanoGPT（仅做预训练）相比，nanochat 名称里的“chat”强调的是全栈对话能力——一个模型不仅要会“续写文本”，还要会“按指令回答、使用工具、自我评估”。另一个关键借鉴来源是 Keller Jordan 的 modded-nanoGPT 项目，后者把“用 45 分钟在单张 H100 上训练一个 124M GPT-2”做成了 speedrun 排行榜游戏；nanochat 继承了 Muon 优化器、ZeRO-2 通信、

speedrun 排行榜等大量工程思想，并把范围从"预训练提速"扩展到"全栈 LLM 工程"。

真正让 nanochat 在工程上引人注目的是它的复现成本数据：8×H100 GPU 节点上约 1.5–2 小时、约 48 美元即可复现一个达到 GPT-2 (1.6B) 能力的模型；spot 实例可压到 15 美元。作为对比，2019 年 OpenAI 训练原始 GPT-2 的成本约为 43,000 美元。六年时间，同样的能力成本下降三个数量级——这背后是 Scaling Laws 理论成熟、Muon 等新优化器出现、FP8 训练普及、Flash Attention 3 等硬件内核优化的叠加效应。nanochat 把这些工程进步浓缩到一个可读、可改、可 fork 的极简代码库，正是其教学价值的最大来源。

1.3 仓库文件树与脚本地图

理解一个 LLM 项目的第一步，是看清它的代码组织。nanochat 的目录结构非常扁平——没有 configs/、没有 factories/、没有 if-then-else 嵌套的"通用框架"代码，而是按"职责一个文件"的原则组织，每个文件大致 100–600 行，可以独立读懂。下表列出全部 28 个源文件及其在流水线中的位置，便于后续章节对照查阅。

路径	行数	功能	对应章节
nanochat/__init__.py	0	空文件	—
nanochat/common.py	328	COMPUTE_DTYPE 检测、compute_init、peak FLOPs 表	第五章
nanochat/gpt.py	555	GPT 模型定义（核心）	第四章
nanochat/dataloader.py	166	预训练 BOS-aligned bestfit 数据加载	第六章
nanochat/dataset.py	160	ClimbMix parquet 分片下载	第三章
nanochat/tokenizer.py	279	BPE 分词器（rustbpe + tiktoken）	第二章
nanochat/optim.py	459	MuonAdamW 组合优化器 + ZeRO-2	第五章
nanochat/engine.py	352	KV Cache 推理引擎 + 工具状态机	第八章
nanochat/execution.py	134	HumanEval 沙箱	第九章
nanochat/core_eval.py	262	DCLM CORE 评估	第九章
nanochat/loss_eval.py	65	BPB 评估	第九章
nanochat/checkpoint_manager.py	193	checkpoint 存档与加载	第五章
nanochat/flash_attention.py	195	FA3/SDPA 统一接口	第八章
nanochat/fp8.py	266	极简 FP8 训练	第五章
scripts/tok_train.py	91	训练 BPE tokenizer	第二章
scripts/tok_eval.py	243	tokenizer 压缩率对比	第二章
scripts/base_train.py	604	预训练主脚本	第五/四章
scripts/base_eval.py	243	base 模型评估	第九章

scripts/chat_sft.py	499	SFT 监督微调	第六章
scripts/chat_eval.py	240	chat 模型评估	第九章
scripts/chat_rl.py	326	简化版 GRPO 强化学习	第七章
scripts/chat_cli.py	100	交互式 CLI 对话	第八章
scripts/infer_bench.py	246	推理 roofline 基准	第八章
tasks/common.py	222	Task 基类与混合	第九章
tasks/gsm8k.py	116	GSM8K 数学题（含工具调用解析）	第七/九章
tasks/humaneval.py	97	HumanEval 编程题（沙箱执行）	第九章
dev/repackage_data_reference.py	128	数据集打包成 parquet 分片	第三章
runs/speedrun.sh	79	★端到端 GPT-2 复现一键脚本	本章 1.1 节
runs/scaling_laws.sh	137	扫 depth × FLOPs 做 scaling laws	第五章

表 1-1 nanochat 核心脚本与对应章节地图

1.4 端到端训练流水线：speedrun.sh 拆解

回到 1.1 节那条命令链。整个 nanochat 的端到端流水线被浓缩在 runs/speedrun.sh 里，约 79 行 shell，把"从下载数据到可以对话"的全过程串成一条命令。下面把 11 个步骤按时序展开，每一步对应一个脚本调用，并标注本课程中负责讲解该步骤的章节。这张时间线既是项目工程的"主索引"，也是后续每章展开的"路线图"。

步骤	命令（简化）	产出	对应章节
1 环境	uv sync --extra gpu	虚拟环境就绪	—
2 下载 8 分片	python -m nanochat.dataset -n 8	~800MB 训 tokenizer 用	第二章
3 后台下载 170 分片	python -m nanochat.dataset -n 170 &	~17GB 预训练用	第三章
4 训练 tokenizer	python -m scripts.tok_train	tokenizer.pkl + token_bytes.pt	第二章
5 评估 tokenizer	python -m scripts.tok_eval	压缩率对比表	第二章

6 预训练	<code>torchrun ... -m scripts.base_train --depth=24 --fp8</code>	<code>base_checkpoints/d24/</code>	第四/五章
7 base 评估	<code>torchrun ... -m scripts.base_eval</code>	CORE / BPB / 采样	第九章
8 SFT	<code>torchrun ... -m scripts.chat_sft</code>	<code>chatsft_checkpoints/d24/</code>	第六章
9 chat 评估	<code>torchrun ... -m scripts.chat_eval -i sft</code>	ChatCORE	第九章
10 RL (可选)	<code>torchrun ... -m scripts.chat_rl</code>	<code>chatrl_checkpoints/d24/</code>	第七章
11 对话	<code>python -m scripts.chat_cli -p "Why is sky blue?"</code>	流式生成	第八章

表 1-2 speedrun.sh 11 步端到端流程



图 1-1 nanochat 技术栈全景：5 个阶段 + 4 个关键指标 + 6 个核心论文支撑

1.5 课程如何使用本书

本课程假定读者有 PyTorch 与深度学习基础（会写 `nn.Module`、知道反向传播、用过 HuggingFace），但不要求熟悉 LLM 训练的工程细节。每章遵循"理论铺垫 + 公式推导 + nanochat 代码精读 + 附图说明 + 扩展练习提示"的统一结构：前半部分补足理论背景，让读者明白"为什么这样做"；后半部分逐行精读 nanochat 的相关脚本，让读者明白"实际怎么做"。

更具体地，每章都会用一个"贯穿示例"——比如第二章用 "low lower lowest" 讲 BPE 训练，第三章用一段 `raw.html` 讲数据集工程，第四章用一个具体的 token batch 讲模型架构，第五章用一段真实训练日志讲预训练超参。每章末尾会回到这个示例，让读者看到本节学到的概念如何作用在具

体输入上、产生什么具体输出。这种"从输入到输出走一遍"的写作风格，能避免"概念一堆但不知道怎么落地"的常见教材陷阱。

建议阅读节奏：第一遍快速通读，每章只看理论部分与图示，建立全局认知；第二遍精读代码部分，对照 GitHub 仓库的源码逐行理解；第三遍以第十章扩展实验为入口，在小模型（d12，单 GPU 5 分钟可训完）上动手做消融实验，把抽象知识转化为肌肉记忆。配套地，第十一章给出按主题分类的 11 篇核心论文阅读路径，便于读者就感兴趣的方向深入。

⚠ 易被忽视的重要纠正

一个易被忽视的纠正：nanochat 的 MLP 激活用的是 ReLU^2 (`F.relu(x).square()`)，不是 Llama 系的 SwiGLU。许多读者带着"Llama = 现代 Transformer"的预设来读，会在第四章感到困惑。请记住：现代 LLM 架构存在多条平行的演化路线， ReLU^2 与 SwiGLU 是其中两个有代表性的选择，各有理论动机与实证表现。

1.6 小结与下一章预告

本章从一个具体复现实验出发，介绍了 nanochat 的项目定位（minimal full-stack ChatGPT clone）、与前作 nanoGPT/modded-nanoGPT 的关系、复现成本（\$48 vs 2019 年 \$43K）、仓库文件树、speedrun.sh 的 11 步流程。一个核心观察是：nanochat 把过去 6 年 LLM 工程的进步浓缩到一个 7000 行的可读代码库，让"训练小模型"的门槛持续降低。

下一章我们从流水线的第一步——分词器（tokenizer）开始。tokenizer 是整个项目"最早期决定、最难修改"的组件，词表一旦定下就锁定了所有 embedding 参数。我们会用一个 "low lower lowest" 的小例子从头走一遍 BPE 训练算法，看一段真实对话如何被切成 (token_ids, loss_mask)，以及中文在 BPE 下有什么独特的工程挑战。

第二章 分词器：BPE 的工程实现

2.1 为什么需要分词器：从一个具体例子说起

在讲算法之前，先看一个具体问题。假设你要把下面这段文字喂给神经网络：

输入文本：
"low lower lowest, Hello 世界! 2026年3月14日"
目标：变成一组整数 token id，喂给 `nn.Embedding`

最朴素的做法是按字节切分——直接用 UTF-8 编码，每个字节一个 token。我们看下结果：

方案 A：按字节切分
"low" → [108, 111, 119] (3 个 token)
"世界" → [228, 184, 150, 231, 149, 140] (6 个 token)
"low lower lowest, Hello 世界! 2026年3月14日" → 共 47 个 token

问题：序列太长，attention 计算量是 $O(T^2)$ ，47 token 还好，47 万 token 就不可接受。

另一极端是按"词"切分——英文按空格、中文按字典分词。结果：

方案 B：按词切分

"low lower lowest, Hello 世界! 2026年3月14日"

→ ["low", "lower", "lowest", ",", "Hello", "世界!", "!", "2026年3月14日"]

(8 个 token)

问题 1：词表会爆炸。光是中文常用词就有 50 万+，加上英文、代码、生僻字，词表轻松破百万。

问题 2：遇到没见过的词怎么办？比如 "ChatGPT-4o"、"nanoΔt"、新造词。

问题 3：一个错别字就变成"未登录词"，模型完全无法处理。

BPE (Byte Pair Encoding, 字节对编码) 是介于二者之间的折中方案：在字节级别贪心合并最高频的相邻对，让"low"压缩成 1 个 token，"世界"压缩成 2 个 token，词表大小可控 (32K–150K)，又能覆盖任意字节序列。下面我们用 "low lower lowest" 这个极简例子，一步步看 BPE 是如何训练和推理的——后续所有章节都会回到这个例子。

☒ 本章贯穿示例

本章贯穿示例：BPE 训练用 "low lower lowest", split pattern 用 "Hello, 世界! 2026年3月14日", render_conversation 用一段最简对话 [{"role": "user", "content": "你好"}, {"role": "assistant", "content": "你好!"}]。每节末尾会回到这些示例，看本节算法对它们的具体效果。

2.2 BPE 算法的完整推导：用 "low lower lowest" 走一遍

上一节我们看到按字节太长、按词太脆弱。BPE 的核心想法是"在字节上做贪心合并"——但具体怎么合并？这一节用一个 3 词的小语料 "low lower lowest" 完整走一遍训练流程，然后看推理阶段如何应用学到的合并规则。

2.2.1 训练阶段：从字节到合并规则

初始状态：把语料展开为字节序列（这里用 ASCII 表示，每个字符 1 字节）。我们的目标是在这些字节上反复"找最高频相邻对、合并"直到达到词表大小。

语料: "low lower lowest" (不含空格的话是 lowlowerlowest, 但保留空格更真实)

初始字节序列 (每个字符 = 1 个 token) :

l o w l o w e r l o w e s t

共 14 个 token

统计相邻 token 对的频次:

(l, o) → 出现 3 次 ← 最高频

(o, w) → 出现 3 次 ← 最高频

(w, " ") → 出现 1 次

(" ", l) → 出现 2 次

(w, e) → 出现 2 次

(e, r) → 出现 1 次

(r, " ") → 出现 1 次

(e, s) → 出现 1 次

(s, t) → 出现 1 次

第一步合并：选频次最高的对。这里 (l, o) 和 (o, w) 都出现 3 次，假设按字母序选 (l, o) 先合并。合并后产生新 token "lo"，词表 +1。

合并 1: (l, o) → "lo"

更新后的序列：

l o w l o w e r l o w e s t
共 11 个 token (少了 3 个)

词表：256 (字节) + "lo" = 257

第二步合并：在更新后的序列上重新统计相邻对频次。(o, w) 仍然出现 3 次，最高频。合并为 "low"。

合并 2: (o, w) → "ow" ... 等等，应该是 (lo, w) → "low" 还是 (o, w) → "ow"?

答：取决于合并顺序。如果先合并 (l, o) → "lo"，那现在的相邻对是 (lo, w) 而非 (o, w)。重新统计：(lo, w) 出现 3 次，最高频。合并为 "low"。

合并 2: (lo, w) → "low"

更新后序列：low low e r low e s t 共 8 个 token

词表：258

继续这一过程。第三步可能是 (low, " ") → "low " (带空格的"low")，让"low"后跟空格合并。第四步可能是 (e, r) → "er"。第五步可能是 (e, s) → "es"。这样合并 5 次后，"low" 这个完整词会变成 1 个 token，"lower" 变成 2 个 token (low + er)，"lowest" 变成 2 个 token (low + est)。

最终 (示意，合并 5 次后)：

low low er low est
共 5 个 token (从 14 降到 5，压缩 2.8×)

关键观察：三个共享 "low" 的词，"low" 部分被复用为同一个 token。

"low" → ["low"] (1 token)

"lower" → ["low", "er"] (2 tokens, 原 5)

"lowest" → ["low", "est"] (2 tokens, 原 7)

在真实 BPE 训练 (32K 词表) 中，"low" 会更早合并为一个 token。

更严格地说，BPE 的合并目标可以表述为最大化语料的似然压缩比。设当前词表大小为 V ，合并一对 (x, y) 后，描述语料所需的 token 数从 N 减少到 $N - f(x, y)$ ，但词表大小从 V 增加到 $V+1$ 。在信息论意义下，合并带来的"描述长度收益" ΔL 满足：

$$\Delta L = f(x, y) \cdot \log_2(V+1) - N \cdot \log_2(V+1) + N \cdot \log_2(V) \approx f(x, y) \cdot \log_2(V) - \log_2(e)$$

当 V 较大时，每次合并的边际信息收益近似为 $f(x, y) \cdot \log_2(V)$ ，因此"选频次最高的对"等价于"选信息收益最大的对"。这也解释了为什么 BPE 在词表较小（如 8K）时合并速度极快，而词表较大（如 64K）时每次合并的边际收益递减——需要训练更长的语料才能稳定合并。



图 2-1 BPE 合并过程示意：从字节级到 32K 词表的迭代过程

2.2.2 推理阶段：贪心切分的等价性

刚才我们走完了训练：从 14 个字节合并到 5 个 token，学到 5 条合并规则。现在看推理：给定一个新文本 "lowest"，怎么用这 5 条规则切分？

推理输入: "lowest"
 初始: l o w e s t (6 个字节 token)

按训练顺序应用合并规则：

规则 1: (l, o) → "lo" → "lo w e s t" (5 token)
 规则 2: (lo, w) → "low" → "low e s t" (4 token)
 规则 5: (e, s) → "es" → "low es t" (3 token)

最终: ["low", "es", "t"] (3 token, 原 6 字节压缩 2×)

这里有个工程坑：合并规则的全局应用顺序并不直接对应"最长匹配"。例如训练时先合并 "ab→ab" 再合并 "bc→bc"，那么 "abc" 应该被切分为 "ab" + "c"，而非 "a" + "bc"。这一"按训练顺序贪心"的实现细节，导致 tiktoken 与 HuggingFace tokenizers 在某些边界情况下会给出不一致的结果——nanochat 选择训练用 rustbpe、推理用 tiktoken，正是因为两者都遵循这一严格顺序，互为兼容。

2.3 nanochat 的 split pattern：为什么不直接合并

刚才的 BPE 在 "low lower lowest" 上工作得很好。但如果直接在整个文本上合并，会发现一个奇怪现象：让我们看 "Hello, 世界!" 这个例子。

无 split pattern 的 BPE 直接在整个文本上合并：
 输入: "Hello, 世界!"
 可能合并: "Hello," → 1 token, ",世" → 1 token (因为高频相邻)

问题: ", 世" 这种"逗号接中文"被合并成一个 token，意味着这个 token 只在"英文逗号后跟中文"的极少场景下出现，浪费词表名额。而且 "Hello," 与 "Hello world," 中的 "Hello" 不再共享 token。

解决思路是"预切分"——先用正则将文本上切成片段，BPE 只在片段内部合并，片段边界不会被合并。GPT-2 引入了这套正则，GPT-4 进一步精化了它，使其能更好处理多语言、代码、数学符号。我们看 "Hello, 世界! 2026年3月14日" 在 nanochat 的 split pattern 下如何被切分：

```
import regex as re
GPT4_SPLIT_PATTERN = r"""
    's|'t|'re|'ve|'m|'ll|'d # 英文缩写
    | ?\p{L}+ # 字母词
    | ?\p{N}{1,2} # 数字 (1-2 位, nanochat 改造)
    | ?[^\s\p{L}\p{N}]+ # 非字母数字非空白
    | \s+(?!\S) | \s+ # 空白
    """"

text = 'Hello, 世界! 2026年3月14日'
chunks = re.findall(GPT4_SPLIT_PATTERN, text)
# 结果:
# ['Hello', ',', ' ', '世界', '!', ' ', '2026', '年', '3', '月', '14', '日']
# 每个片段独立做 BPE, 片段边界不会合并
```

一个关键观察：上例中 "2026" 这个片段（前导空格 + 4 位数字）会被切分为 ["2026"] 一个片段，BPE 在其内部合并。但 "3" 与 "14" 各自成为独立片段（因为 {1,2} 只匹配 1-2 位数字），不会被合并为一个 token。这是 nanochat 相对 GPT-4 的唯一改动——把数字部分的 {1,3} 改成 {1,2}。

GPT-4 Split Pattern (nanochat 改造)

对比 GPT-4 的 `\p{N}{1,3}` 与 nanochat 的 `\p{N}{1,2}`

GPT-4 tokenize: "2026 3 14" → ["20", "26", " ", "3", " ", "14", " "] (7 token)

nanochat: "2026 3 14" → ["20", "26", " ", "3", " ", "14", " "] (7 token)

正则组件:

<code>'s 't 're 've 'm 'll 'd</code>	<code>\p{N}{1,2}</code>	<code>?\p{L}+</code>	<code>?\p{N}+</code>
<code>?[^\s\p{L}\p{N}]+</code>	<code>\s+(?!\S)</code>	<code>\s+</code>	

图 2-2 GPT-4 split pattern 与 nanochat 改造对比

为什么这样改？在 32K 词表规模下有实证依据——nanochat 词表较小，多分一位数字给"字母词"或"罕见汉字"会让那些更有信息量的 token 获得更多词表名额，最终的整体压缩率反而更好。更具体地，对中文文本"2026年3月14日"，GPT-4 切分为 ["20", "26", "年", "3", "月", "14", "日"] (7 个)，nanochat 也切分为相同的 7 个，但在处理 "abc123def456" 这类字母数字混排时，nanochat 会把每个 2 位数字分到不同 token，整体压缩率稍低，但数学表达式的语义边界更清晰。在 GSM8K 数学推理任务上，nanochat 的实证表现优于直接套用 GPT-4 模式。

2.4 特殊 token：对话格式如何用 token 表示

前面三节解决了"如何切分自然文本"的问题。但 LLM 不只是"续写文本"——还要会"按指令回答、使用工具、自我评估"。这些场景需要明确标记"这是用户说的"、"这是助手回答的"、"这里要执行 Python 代码"。光靠自然文本切分无法表达这些"结构信号"。

解决方案是"特殊 token"——一批不在 BPE 词表里的原子 token，每个对应一个明确的语义角色。nanochat 设计了 9 个特殊 token。我们先看一段最简对话如何被这 9 个 token 表示。

对话内容：

user: 你好

assistant: 你好!

加上特殊 token 后的序列（还没经过 BPE，仅插入特殊 token）：

<|bos|> <|user_start|> 你好 <|user_end|> <|assistant_start|> 你好! <|assistant_end|>

关键性质：每个特殊 token 都是"原子"——不会被 BPE 拆成更小的字节 token，
一个特殊 token = 一个 token id。

Token	语义	出现场景
< bos >	序列开始 / 行分隔	每行打包首 token
< user_start >	用户消息开始	对话格式
< user_end >	用户消息结束	对话格式
< assistant_start >	助手回复开始	对话/推理
< assistant_end >	助手回复结束	触发停止 / 行结束
< python_start >	Python 代码块开始	工具调用
< python_end >	Python 代码块结束	触发 use_calculator
< output_start >	工具输出开始	工具回喂
< output_end >	工具输出结束	工具回喂

表 2-1 nanochat 的 9 个特殊 token 与语义角色

这些特殊 token 在 BPE 训练时被"硬编码"加入词表，且不会被 BPE 合并规则拆开——这是 tokenizer.py 中 RustBPETokenizer.__init__ 的关键设计。值得对比的是，Llama 系列的对话格式用 [INST]...[/INST] 这种"自然语言标签"，需要 BPE 切分成多个 token；而 nanochat 把每个特殊 token 当作"原子 token"，单 token 即可触发状态切换，减少了对话模板的 token 占用，也让对话格式的训练信号更清晰。

2.5 代码精读：tokenizer.py 与 render_conversation

前面四节我们分别讲了 BPE 算法、split pattern、特殊 token。但它们怎么组合起来工作？这一节用一段最简对话作为输入，完整走一遍 nanochat 的 render_conversation 函数，看它如何把对话字典变成训练用的 (token_ids, loss_mask) 二元组。这是 SFT 与 RL 数据加载的核心。

2.5.1 输入对话与期望输出

输入对话（Python 字典列表）：

```
messages = [
    {"role": "user", "content": "你好"},
    {"role": "assistant", "content": "你好! "},
]
```

期望输出（两个等长数组）：

```
token_ids = [<|bos|>, <|user_start|>, "你", "好", <|user_end|>,
              <|assistant_start|>, "你", "好", "!", <|assistant_end|>]
loss_mask = [0, 0, 0, 0, 0,
```

0, 1, 1, 1, 0]

↑

只有 assistant 完成部分 mask=1, 其他全 0

2.5.2 代码精读：如何把字典变成 token_ids + loss_mask

现在看 render_conversation 函数如何完成这个转换。关键逻辑：（1）开头插入 <|bos|>, mask=0；（2）遍历 messages；（3）user 消息前后包 <|user_start|>...<|user_end|>, 全部 mask=0；（4）assistant 消息前后包 <|assistant_start|>...<|assistant_end|>, 其中 special token mask=0、自然语言文本 mask=1；（5）system 消息自动并入下一条 user（不单独渲染）。

```
def render_conversation(self, messages):
    ids, mask = [self.special_ids['<|bos|>']], [0]
    for msg in messages:
        role = msg['role']
        if role == 'system':
            continue # system 自动并入下一条 user
        if role == 'user':
            ids += self.encode_special('<|user_start|>') \
                + self.encode(msg['content']) \
                + self.encode_special('<|user_end|>')
            mask += [0] * (len(ids) - len(mask)) # 全部 mask=0
        elif role == 'assistant':
            ids += self.encode_special('<|assistant_start|>')
            mask += [0] # assistant_start 也是 mask=0
            ids += self.encode(msg['content'])
            mask += [1] * len(self.encode(msg['content'])) # text 部分 mask=1
            ids += self.encode_special('<|assistant_end|>')
            mask += [0] # assistant_end 也是 mask=0
    return ids, mask
```

2.5.3 走一遍：实际 token_ids 与 loss_mask 的演化

把上面的代码应用到我们的输入对话，逐步看 ids 与 mask 如何增长：

初始: ids=[], mask=[]

插入 <|bos|>:

```
ids = [<|bos|>]
mask = [0]
```

处理第 1 条 (user="你好"):

```
插入 <|user_start|>: ids += [<|user_start|>], mask += [0]
插入 BPE("你好"): ids += ["你", "好"], mask += [0, 0]
插入 <|user_end|>: ids += [<|user_end|>], mask += [0]
当前 ids = [<|bos|>, <|user_start|>, "你", "好", <|user_end|>]
当前 mask = [0, 0, 0, 0, 0]
```

处理第 2 条 (assistant="你好! "):

```
插入 <|assistant_start|>: ids += [<|assistant_start|>], mask += [0]
插入 BPE("你好! "): ids += ["你", "好", "! "], mask += [1, 1, 1]
插入 <|assistant_end|>: ids += [<|assistant_end|>], mask += [0]
最终 ids = [<|bos|>, <|user_start|>, "你", "好", <|user_end|>,
            <|assistant_start|>, "你", "好", "! ", <|assistant_end|>]
最终 mask = [0, 0, 0, 0, 0,
            0, 1, 1, 1, 0]
```

这段代码的核心思想：loss_mask 决定了"模型需要学会生成哪些 token"。只有 assistant 输出的自然语言部分 (text part) mask=1，所有特殊 token、user 消息、python 代码块、工具输出都 mask=0。这意味着训练时 cross_entropy 会忽略这些位置，模型不会"学会预测 user 的话"或"学会复制工具输出"，而专注于"在 user 给定的情况下，assistant 应该说什么"。

工程细节：system 消息的处理

一个工程细节：system 消息不单独渲染，而是自动并入下一条 user。这是为了避免每条 system 都消耗一次 `<|user_start|>...<|user_end|>` 的 token 开销。在多线程对话中，system 通常只在开头出现一次（如"You are a helpful assistant"），把它并入第一条 user 让训练数据更紧凑。

2.6 压缩率评估：tok_eval.py 与 BPB 指标

刚才我们看了一个具体对话如何变成 token。但训出来的 tokenizer 好不好？需要一个客观指标。这一节用一个具体计算演示 BPB (bits per byte) 指标——为什么它跨词表可比，而 perplexity 不可比。

2.6.1 同样文本，不同 tokenizer，压缩率差异有多大

假设我们有一段 100 字节的英文文本，分别用 3 个 tokenizer 切分，看 token 数与 BPB：

同一段 100 字节文本：

GPT-2 (词表 50K)：

切出 28 个 token

$BPB = 28 * \log_2(50000) / 100 = 28 * 15.61 / 100 = 4.37 \text{ bits/byte}$

nanochat (词表 32K)：

切出 30 个 token (稍多，因为词表小)

$BPB = 30 * \log_2(32768) / 100 = 30 * 15.00 / 100 = 4.50 \text{ bits/byte}$

GPT-4 (词表 100K)：

切出 22 个 token

$BPB = 22 * \log_2(100000) / 100 = 22 * 16.61 / 100 = 3.65 \text{ bits/byte}$

观察：词表越大，token 数越少，但每个 token 携带的信息更多 ($\log_2(V)$ 更大)。

最终 BPB 综合了这两个因素，跨词表可比。

2.6.2 BPB 公式与跨词表可比性

$$BPB = (\text{token 数} \times \log_2(\text{词表大小})) / \text{字节数}$$

BPB 的关键性质：词表大小变化时，BPB 仍可比，而 perplexity 不可比。这是因为 BPB 直接衡量"描述一个字节需要多少比特"，与词表大小无关——一个 32K 词表的 tokenizer 用 15 个 token 描述 100 字节， $BPB = 15 \times 15 / 100 = 2.25$ ；一个 50K 词表的 tokenizer 用 14 个 token 描述同样 100 字节， $BPB = 14 \times 15.6 / 100 = 2.18$ 。后者压缩率更高。下表是 nanochat 报告的典型压缩率对比：

文本类型	GPT-2 (50K)	GPT-4 (~100K)	nanochat (32K)
英文新闻	0.65	0.55	0.62

韩文	2.10	1.20	1.45
Python 代码	0.72	0.50	0.58
数学公式	1.85	1.30	1.42
科学文本	0.70	0.58	0.65

表 2-2 三种 tokenizer 在 5 类文本上的 BPB 压缩率（示意）

可以看到，nanochat 的 32K 词表在英文上略逊于 GPT-4 的 100K，但在韩文、代码、数学上明显优于 GPT-2，且整体接近 GPT-4。考虑到 32K 词表的 embedding 参数只有 GPT-4 100K 的 1/3，对训练成本显著友好，这是一个适合“小型全栈训练”的合理折中。

2.7 中文分词的工程挑战

前面 6 节我们都用英文例子（"low lower lowest"、"Hello, 世界!"）。但回到那个包含“世界”的例子，中文字在 BPE 下到底怎么被切分？这一节专门讲中文，因为中文分词有一套独立的工程问题——没有空格、字符密度高、什么是一个“词”本身就有歧义。

2.7.1 字节级 BPE 如何处理中文：用“你好，世界！”走一遍

让我们跟踪“你好”这个词在 BPE 训练中的演化。中文字符在 UTF-8 中占 3 字节：

“你”的 UTF-8 字节：E4 BD A0（3 字节）

“好”的 UTF-8 字节：E5 A5 BD（3 字节）

初始（字节级 BPE）：你好 → [E4, BD, A0, E5, A5, BD]（6 个 token）

在中文语料上训练 BPE 后（假设“你”字在语料中出现 100 万次）：

合并 1：(E4, BD) 频次高 → 合并为 [E4BD]

合并 2：(E4BD, A0) 频次高 → 合并为 [E4BDA0] = 单 token 表示“你”

类似地，“好”也会合并为单 token [E5A5BD]

最终：你好 → ["你", "好"]（2 个 token，原 6 字节压缩 3×）

一个关键细节：BPE 的合并顺序按“频次”决定。如果训练语料中的“的”字出现 100 万次、“鹅”字出现 1 万次，则“的”对应的 3 字节（e7 9a 84）会比“鹅”对应的 3 字节更早合并。这意味着：tokenizer 训练语料的语言分布直接决定哪些中文字被“独立成 token”，哪些被“拆为多 token”。如果训练语料是纯英文，则中文字几乎不会合并——所有中文字都是 3 token。如果训练语料中英混合，则高频中文字会合并，但词表会被“中文字 token”占用相当一部分。

⚠ nanochat 默认 tokenizer 的中文局限

工程经验：nanochat 默认 tokenizer 是用 ClimbMix 训的——ClimbMix 主要是英文数据，中文占比极低。因此 nanochat 默认 tokenizer 对中文压缩率较差（每个汉字约 2.5 token，BPB ≈ 2.0），直接用于中文任务会浪费上下文长度。如果你的应用场景主要是中文，强烈建议在 ClimbMix 之外加入中文语料重新训 tokenizer，或直接用 Qwen / DeepSeek 的开源 tokenizer。

2.7.2 中文压缩率对比：英文 vs 中文 vs 混合 tokenizer

下面这张表把不同 tokenizer 在 "你好，世界！" 这句话上的表现具体化。看到差距后，你会理解为什么 Llama 3 把词表从 32K 扩到 128K 是中文能力提升的关键。

Tokenizer	词表大小	英文 BPB	中文 BPB	中文 1 字 = ? token
GPT-2 (英文优先)	50K	0.65	2.10	~2.8
GPT-4 (多语)	~100K	0.55	1.20	~1.5
nanochat 默认 (ClimbMix)	32K	0.62	2.00	~2.5
Llama 3 (多语增强)	128K	0.55	0.85	~1.1
Qwen2.5 (中英双优)	152K	0.60	0.55	~0.7
DeepSeek-V3 (中英+代码)	128K	0.58	0.55	~0.7
纯中文实验 (~30K 汉字)	32K	1.20	0.40	~0.5

表 2-3 不同 tokenizer 在中英文上的压缩率对比（示意数据）

一个关键观察：Qwen2.5、DeepSeek-V3 这类"中英双优"tokenizer 的词表（152K、128K）远大于 nanochat 的 32K，因为它们要为"每个中文字分配一个独立 token"，加上常用中文词组、英文词、代码 token，词表压力很大。Llama 3 的 128K 词表是其在 32K 上的显著改进——Llama 2 的 32K 词表在中文上 BPB 高达 1.5，Llama 3 把词表扩到 128K 后中文 BPB 降到 0.85，这是 Llama 3 中文能力显著强于 Llama 2 的核心原因之一。

2.7.3 中文 LLM 的词表设计权衡

设计中文 LLM 的 tokenizer 时，工程师面临三个核心权衡：

决策点	选项 A	选项 B	主流选择	理由
词表大小	32K（小，参数省）	128K-150K（大，压缩好）	128K (Llama3/Qwen/DeepSeek)	中文压缩率重要 > embedding 参数成本
字符 vs 词组	汉字单字优先	中文常用词组优先（如"中国"、"我们"）	单字优先 + 部分高频词组	单字通用性强，词组可由单字组合
训练语料	纯中文	中英混合（如 60% 中文 + 40% 英文）	中英混合	保证英文能力同时优化中文

表 2-4 中文 LLM tokenizer 设计的核心权衡

2.7.4 在 nanochat 中加入中文支持的最小改动

如果想用 nanochat 训练一个中文 LLM，最小改动方案：（1）准备一份中英混合预训练语料（如 60% 中文 FineWeb-zh + 40% 英文 ClimbMix）；（2）在 scripts/tok_train.py 中把 --vocab_size 从 32768 改为 131072（128K），让词表有足够空间给中文字；（3）保持 split pattern 不变（GPT-4 模式对中文已经是字节级 BPE，无需修改）；（4）重新训 tokenizer（约 1 小时在 8×H100 上）；（5）在 base_train.py 中调小 --device-batch-size（embedding 参数增

加约 4×，显存吃紧）。其他训练流程不变。

```
# 训练中文支持的 tokenizer (在 nanochat 框架内)
# 1. 准备中英混合语料
python -m nanochat.dataset --extra_source=fineweb-edu-zh -n 50 # 50 个中文分片
python -m nanochat.dataset -n 100 # 100 个英文 ClimbMix 分片

# 2. 训 128K 词表的 tokenizer
python -m scripts.tok_train -- \
    --vocab_size=131072 \
    --include_zh=True \
    --output=tokenizer_zh.pkl

# 3. 评估中文压缩率
python -m scripts.tok_eval -- --tokenizer=tokenizer_zh.pkl --lang=zh

# 4. 预训练 (注意 embedding 参数显著增大)
torchrun --nproc_per_node=8 -m scripts.base_train -- \
    --depth=24 --vocab_size=131072 \
    --tokenizer=tokenizer_zh.pkl \
    --device-batch-size=8 # 减小 batch 防 OOM
```

一个常被忽视的细节：词表从 32K 扩到 128K，embedding 参数从 $N \times 32K$ 增到 $N \times 128K$ (4×)。在 d24 模型 ($n_emdb=1080$) 上，embedding 参数从 35M 增到 139M，总参数从 1.6B 增到 1.7B——相对增加约 6%，但中文能力可能从 BPB 2.0 提升到 0.7，整体收益显著。这也是为什么 Llama 3、Qwen2、DeepSeek-V3 都选择 128K+ 大词表。

2.8 小结

本章从“如何把文本变成 token”这个具体问题出发，用 “low lower lowest”、“Hello, 世界!”、一段最简对话作为贯穿示例，一步步讲了 BPE 算法的训练与推理、split pattern 的预切分、特殊 token 的对话格式、render_conversation 的代码精读、BPB 评估指标、中文分词的工程挑战。

关键决策点回顾：nanochat 选择 32K 词表 + GPT-4 split pattern (数字 {1,2} 改造) + 9 个原子特殊 token。这套组合在英文上接近 GPT-4，但在中文上明显弱于 Qwen/DeepSeek——如果你的应用场景主要是中文，建议把词表扩到 128K 并加入中文语料重新训 tokenizer。

一个常被忽视的事实是：tokenizer 在 LLM 项目中是“最早期决定、最难修改”的组件。一旦预训练开始，词表就锁定了——改变词表意味着所有 embedding/lm_head 参数需要重新初始化或对齐，训练成本不可忽视。因此 nanochat 在 tokenizer 阶段就考虑了对话、工具调用、多语言、代码、数学等多种场景，避免后期返工。这一“前端决策重要性”的思维，是工程化 LLM 与“调 API 训个模型”的重要分水岭。下一章我们看数据集工程——同样是一个“前端决策”决定整个项目走向的关键环节。

第三章 数据集工程：从 FineWeb 到 ClimbMix

3.1 数据集是 LLM 工程的“真正瓶颈”：从一份网页爬取说起

在讲算法之前，先看一个具体场景。假设你拿到一份原始网页爬取数据 raw.html，想用它训练一个 LLM。从 raw.html 到训练用的 token batch，中间需要走多少步？这是本章要回答的问题。

假设你拿到的原始网页爬取数据 raw.html（截选）：

关于深度学习 - 博客

关于 | 联系 | 订阅

关于深度学习

深度学习是机器学习的一个分支，使用多层神经网络...

2006 年 Hinton 提出的深度信念网络...

@ 2024 | 上一篇 | 下一篇

track_click();

期望的训练输入 (token batch)：

```
tensor([[12, 478, 1024, ..., 25, 8712, 9], # 样本 0
        [88, 514, 7703, ..., 14, 599, 22], # 样本 1
        ...]) # shape: (B, T)
```

中间需要走 6 步：

raw.html → 清洗文本 → 高质量过滤 → 去重 → parquet 分片 → token batch

这 6 步每一步都对应一个工程决策——用什么清洗规则？如何判断"高质量"？去重的相似度阈值多少？分片大小选多少？这些问题决定了最终训练数据的质量。在 LLM 工程圈有一句流传甚广的话："模型决定下限，数据决定上限"。架构、优化器、训练超参决定了模型能否"按预期收敛"，而数据质量与规模决定了模型最终"有多强"。

nanochat 在数据集上做了几个关键选择：（1）预训练用 ClimbMix-400B 而非单源数据集；（2）SFT 用 SmolTalk + MMLU + GSM8K 三源混合，且不同源重复采样不同次数；（3）评估严格分离训练集与测试集，避免污染。本章围绕这三个选择展开，讲清楚每个选择背后的工程权衡，并回到上面那个 raw.html 例子，看每一步如何处理它。

☒ 本章贯穿示例

本章贯穿示例：raw.html（原始网页）→ 清洗后的纯文本 → 过滤后保留的高质量段 → 去重后的最终训练数据 → 打包成 parquet 分片 → token batch。每节会回到这个例子，看本节算法对它的具体效果。

3.2 预训练数据集的演进史：每一代解决了上一代的什么问题

回到 3.1 节的 raw.html 例子——从原始 HTML 到干净文本这一步，看似简单，却走过了四代演进。每一代都伴随着"为什么之前的不好"的明确反思。这一节我们走一遍这四代，看每代如何处理 raw.html，理解这条演进线能让你看清 nanochat 选择 ClimbMix 的必然性。

世代	代表数据集	规模	关键问题	解决思路
第一代	Common Crawl 直接抓取 / C4	约 750GB	质量参差、广告、垃圾、重复	基本过滤、规则筛选
第二代	Pile / RedPajama	约 825GB / 1.2T	版权争议、单源依赖	多源混合（GitHub、Book、Wikipedia 等）

第三代	FineWeb / DCLM-baseline	约 15T tokens	通用质量已饱和，缺乏教育/推理能力	高质量过滤 pipeline + dedup
第四代	FineWeb-Edu / ClimbMix / Nemotron-Climb Mix	约 350B–400 B (精选)	能力维度单一（如纯教育）	教育/推理/代码多源加权混合

表 3-1 预训练数据集四代演进

3.2.1 第一代：Common Crawl 与 C4

第一代数据集只做最基本的清洗：去除标签、去重、过滤空行。以我们的 raw.html 为例，第一代处理后：

第一代清洗后（去标签、去 JS、保留所有文本）：
关于 联系 订阅 ← 导航未被识别为低质量，被保留
关于深度学习
深度学习是机器学习的一个分支，使用多层神经网络...
2006 年 Hinton 提出的深度信念网络...
© 2024 上一篇 下一篇 ← 页脚未被识别为低质量

问题：导航、页脚这种低信息量文本混入了训练数据。

Common Crawl 是一个非营利组织每月抓取的网页存档，原始规模约 250B 网页、PB 级数据。直接用作训练数据有两个明显问题：（1）质量参差——垃圾邮件、广告、低质内容混入；（2）重复严重——同一文章在多个网站被镜像转载。C4 (Colossal Clean Crawled Corpus, Google 2019) 是 T5 训练用的过滤版本，通过"必须包含句号/问号/感叹号、去掉 bad words、去掉过短/过长行"等启发式规则筛掉约 80% 原始数据，得到约 750GB 干净文本。但 C4 的"干净"定义很粗糙——只要看起来像自然语言就保留，没有真正的"教育/推理价值"评估。

3.2.2 第二代：Pile 与 RedPajama

Pile (EleutherAI 2020) 开创了"多源混合"思路：把 22 个不同来源的数据集 (GitHub、arXiv、PubMed、Wikipedia、StackExchange 等) 按权重混合，每个源对模型贡献不同能力维度。例如 GitHub 贡献代码能力、arXiv 贡献学术写作、PubMed 贡献医学专业词汇。但 Pile 在 2023 年后面临严重的版权争议——多个出版商指控 Pile 包含其付费内容，这直接推动了后续数据集在合规性上的重新设计。

RedPajama (Together AI 2023) 继承 Pile 多源思路但更注重合规性，规模扩展到 1.2T tokens。然而 RedPajama 的源仍以"通用网页"为主，教育/推理密度不足——这是 FineWeb 系列要解决的核心问题。

3.2.3 第三代：FineWeb 与 DCLM-baseline

第二代的问题是"多源但不区分质量"。第三代引入了质量分类器——用 roberta-base 训练一个"高/低质量文本"二分类器，给每段文本打分。以我们的例子，第三代能识别出"关于 联系 订阅"是低质量导航文本：

第三代质量分类后（低质量段被移除）：
× 关于 联系 订阅 ← 质量分 0.12，被删除
✓ 关于深度学习 ← 质量分 0.89，保留
✓ 深度学习是机器学习的一个分支... ← 质量分 0.92，保留

✓ 2006 年 Hinton 提出的深度信念网络... ← 质量分 0.88, 保留
✗ @ 2024 上一篇 下一篇 ← 质量分 0.15, 被删除

FineWeb 的过滤 pipeline 还包含 URL 黑名单、语言识别、MinHash 去重, 最终得到约 15T tokens。

FineWeb (HuggingFace 2024) 是"通用质量"上的最佳实践: 基于 Common Crawl, 用一套精细化过滤 pipeline 处理: (1) URL 过滤 (去掉成人/垃圾站点); (2) 语言识别 (保留英文); (3) 质量分类器 (用 roberta-base 训练, 区分"高/低质量文本"); (4) MinHash 去重 (去除相似度高的文档); (5) C4 风格的启发式过滤。最终得到约 15T tokens 的通用预训练数据。

DCLM-baseline (DataCompass Lab 2024) 是 FineWeb 的进一步精化, 强调"可复现的数据过滤 pipeline"——每个过滤步骤都开源、可验证。DCLM-baseline 也是 DCLM CORE 评估的姊妹项目, 二者配套使用。

3.2.4 第四代: FineWeb-Edu 与 ClimbMix

第三代已经把通用质量做到极臻, 但发现新问题: 通用质量高不等于"能教会模型推理能力"。第四代进一步按"能力维度"划分子集, 多能力维度加权混合。以我们的例子:

第四代能力维度分类后:

关于深度学习 ← 教育类, 进入 FineWeb-Edu
深度学习是机器学习的一个分支... ← 教育类, 进入 FineWeb-Edu
2006 年 Hinton 提出的深度信念网络... ← 科学类, 进入 DCLM

ClimbMix 把多个能力子集按权重混合, 最终得到一个能力均衡的预训练语料。

FineWeb-Edu 是 FineWeb 的"教育子集"——用专门训练的教育质量分类器给文档打分 (1-5 分), 只保留 4 分及以上的文档。结果是约 1.3T tokens 的"教育高密度"数据, 在中小学/大学课程主题上覆盖更深。nanochat 早期版本 (2025-10) 就用 fineweb-edu-100b (FineWeb-Edu 的 100B 子集) 作为预训练数据。

ClimbMix (Nemotron-ClimbMix, NVIDIA 2025) 是更进一步的"多能力维度混合"——把 FineWeb-Edu、DCLM-baseline、代码数据集、数学数据集、多语言数据集按"能力提升"权重混合, 得到约 400B tokens 的"能力均衡"数据。nanochat 当前版本 (2026) 使用 climbmix-400b-shuffle 作为预训练数据, 从 FineWeb-Edu 升级到 ClimbMix 是为了"补齐代码与数学能力"——在 d24 上 CORE 提升约 0.02, 在 ChatCORE 的 GSM8K 子项上提升约 0.05。

工程权衡: 数据规模与多 epoch

工程经验: 不要把"数据越多越好"当作铁律。Chinchilla 论文证明, 给定 FLOPs 预算下, "参数量与数据量按 1:20 比例"是 compute-optimal。在 1.6B 参数模型上, 最优数据量约 32B tokens——ClimbMix 400B 是这个的 12 倍, 意味着 nanochat 用"多 epoch"训练。多 epoch 的代价是"过拟合风险", 但 ClimbMix 的高质量缓解了这一风险。

3.3 dataset.py: 分片下载与并行化

nanochat 的 dataset.py 是预训练数据的"物流层"——负责从 HuggingFace 下载 parquet 分片, 提供按分片遍历的接口给 dataloader.py。整个文件只有 160 行, 但解决了几个关键工程问题。

3.3.1 为什么用 parquet 分片

LLM 预训练数据规模动辄数百 GB 到数 TB，单文件存储不可行——（1）下载失败后无法续传；（2）单文件 IO 速度受限；（3）无法并行处理。HuggingFace 的标准做法是把大数据集切成约 100MB 的 parquet 分片，每片约 1000 个 row_group。parquet 格式相比 JSON/CSV 有三个优势：（1）列式存储，按字段过滤时只读必要列；（2）内置压缩（zstd 默认），文本数据压缩比约 5×；（3）支持 row_group 级别的随机访问。

3.3.2 多进程并行下载

```
# dataset.py 核心思路（简化）
def download_shards(num_shards, output_dir):
    shards = list_shards_from_hf('karpathy/climbmix-400b-shuffle')
    to_download = shards[:num_shards]

    # 多进程并行下载（每个分片独立 HTTP 请求）
    with ProcessPoolExecutor(max_workers=8) as exe:
        futures = [exe.submit(download_one_shard, s, output_dir)
                    for s in to_download]
        for f in as_completed(futures):
            try:
                f.result()
            except Exception as e:
                # 重试逻辑
                retry_download(...)
```

speedrun.sh 中两阶段下载策略：（1）先下载 8 个分片（约 800MB）训 tokenizer——这一步快、不阻塞预训练；（2）后台并行下载 170 个分片（约 17GB）作为预训练数据——这一步慢、不阻塞训练前准备。通过把“急用的少量数据”与“预训练的大量数据”分开，让整个流水线可以在 5 分钟内启动训练准备，同时数据下载与 tokenizer 训练并行进行。这种“流水线调度”思维是大规模 LLM 工程的常用模式。

3.3.3 断点续传与重试

网络下载在长任务中失败是常态而非例外。nanochat 的 dataset.py 实现了“按分片为单位的断点续传”——每个分片下载完成后写入 .done 标记文件，重启时跳过已完成的分片。HTTP 请求带指数退避重试：第一次失败后等 1 秒、第二次 2 秒、第三次 4 秒，最多重试 5 次。这一设计在 170 分片下载中尤其重要——任何一片失败都不会中断整个流程。

3.4 dev/repackage_data_reference.py：从原始数据到分片

dev/repackage_data_reference.py 是 nanochat 仓库的一个“参考脚本”——它不是 speedrun 流程的一部分，而是演示“如何把 HuggingFace 原始数据集打包成 nanochat 期望的 parquet 分片格式”。理解这个脚本，能让你看到“数据集工程”的全貌：从公开数据到可用训练数据，中间需要做什么。

```
# dev/repackage_data_reference.py 核心思路
def repackage(input_dataset, output_dir, shard_size=100*1024*1024):
    """把 HuggingFace 原始数据打包成 ~100MB parquet 分片。"""
    writer = None
    current_shard_size = 0
    shard_idx = 0
```

```

for example in input_dataset:
    if writer is None or current_shard_size >= shard_size:
        if writer is not None:
            writer.close()
        # 新建分片, row_group=1024
        writer = pq.ParquetWriter(
            f'{output_dir}/shard_{shard_idx:05d}.parquet',
            schema,
            compression='zstd',
            row_group_size=1024,
        )
        shard_idx += 1
        current_shard_size = 0

    # 写一行 (一个文档)
    writer.write_table(pa.table([example['text']], names=['text']))
    current_shard_size += len(example['text'])

if writer is not None:
    writer.close()

```

几个关键设计：（1）row_group_size=1024——每个 row_group 包含 1024 个文档，dataloader 按 row_group 切分给 DDP rank，让每个 rank 读独立的数据子集；（2）compression=zstd——压缩比与解压速度的折中，比 snappy 压缩率高、比 gzip 快；（3）单列存储（只有 text 列）——LLM 预训练只需要纯文本，元数据（如 URL、来源）按需存储。

如果你想做“自己的预训练数据集”——比如用内部业务数据训练 LLM——可以参考这个脚本：把你的原始数据按“~100MB parquet 分片、row_group=1024、zstd 压缩”的格式打包，就能直接喂给 nanochat 的 dataloader.py。这是 nanochat “maximally-forkable strong baseline” 设计哲学的具体体现——所有“改造成本”都被压到最低。

3.5 数据集 Scaling Laws: Chinchilla 与“多 epoch”

Chinchilla 论文给出的 compute-optimal 数据/参数比是 20:1。在 nanochat 默认 d24 模型（约 1.6B 参数）上，compute-optimal 数据量约 32B tokens。但 ClimbMix-400B 是这个的 12 倍——为什么 nanochat 用“远超 Chinchilla 最优”的数据量？

答案在于“多 epoch 训练”在高质量数据上的可行性。Chinchilla 假设“每个 token 只看一次”，但后续研究（Muennighoff et al. 2023）发现：（1）当数据是高质量教育内容时，4–10 epoch 几乎不损失性能；（2）当数据是低质量时，2 epoch 就开始过拟合；（3）“数据重复”vs “epoch 重复”是两个不同概念——前者是同一文档在数据集中出现多次，后者是训练循环多次遍历同一数据集。前者危害大，后者危害小。

$\text{target_param_data_ratio} = \text{数据量} / \text{参数量} = D / N$ Chinchilla 最优 = 20；nanochat 默认 = 12；speedrun = 8

nanochat 的默认 target_param_data_ratio=12 比 Chinchilla 的 20 更激进，意味着“更小数据 + 多 epoch + 高质量数据”——这是 FineWeb-Edu/ClimbMix 时代的新策略。speedrun 模式进一步降到 8，是“用更少数据换更短训练时间”的进一步妥协，在 d24 上跑出 CORE 0.2626，已经接

近 2019 年 GPT-2 (1.5B) 的水平。

3.6 SFT 数据集：SmolTalk、MMLU、GSM8K 的来源与构造

SFT 阶段的数据集决定了"对话模型能听懂什么、能回答什么"。nanochat 的 SFT 数据混合包含三个数据集，每个都有不同的来源与构造方式。

数据集	来源	原始规模	构造方式	在 nanochat 中的用途
SmolTalk	HuggingFaceTB/ smol-smoltalk	约 460K 对话	混合 OpenOrca、Self-OSS-Instruct、SmolTalk-curated 等多源，HF 团队整理	通用对话能力
MMLU	cais/mmlu	auxiliary_train 约 99K 多选题	57 学科多选题（涵盖 STEM、人文、社科等），辅助训练集来自原始 MCQ 数据	学科知识 + 多选题格式
GSM8K	openai/gsm8k	main 约 7.5K 数学题	OpenAI 雇佣标注员写的小学数学题，每题含解题过程与#### 答案	数学推理 + 工具调用

表 3-2 SFT 三大数据集来源与构造

3.6.1 SmolTalk：HuggingFace 的"对话数据基座"

SmolTalk 是 HuggingFace 团队为训练 SmolLM2 系列模型整理的 SFT 数据集，混合了多个开源对话/指令数据集：OpenOrca（GPT-4 蒸馏的指令跟随）、Self-OSS-Instruct（自蒸馏）、LongAlign（长上下文）、SmolTalk-curated（人工精选）。每条样本是 (user, assistant) 多轮对话，已经过质量筛选与去重。SmolTalk 的设计哲学是"覆盖广、质量稳"——不追求某一维度的极致（如纯数学、纯代码），而追求"通用对话能力"的均衡覆盖。

3.6.2 MMLU：多学科多选题的"知识测试"

MMLU（Massive Multitask Language Understanding, Hendrycks et al. 2021）包含 57 个学科的多选题，涵盖 STEM（数学、物理、化学、计算机）、人文（历史、哲学、法律）、社科（经济学、心理学、社会学）等。原始数据集区分 train、val、test 三个 split——auxiliary_train 是 MMLU 团队额外收集的"训练用多选题"（约 99K），test 是用于评估的 14042 题。nanochat 用 auxiliary_train 做 SFT 训练、用 test 做 ChatCORE 评估，严格分离。

3.6.3 GSM8K：小学数学题的"推理能力测试"

GSM8K（Grade School Math 8K, OpenAI 2021）是 OpenAI 雇佣标注员手写的小学数学题，每题包含一个自然语言题干、一个逐步推理的解题过程、一个 ##### 后的最终数字答案。原始数据集 main split 约 7.5K 题、test split 约 1.3K 题。GSM8K 的关键设计是"答案客观可验证"——只要提取出 ##### 后的数字与 ground truth 比对，就能自动判断对错，这让 RL 训练完全自动化（见第七章）。

一个值得注意的工程细节：nanochat 在 SFT 阶段把 GSM8K 重复 4 次、MMLU 重复 3 次。这种"温和过拟合"对"题量小但需要形式推理"的任务尤其有效——模型在多次见到同类问题后能学到稳定的解题模式。但 SmolTalk 数据量大、多样性高，不需要重复。这种"按数据集重要性差异化重复"

是 SFT 数据工程的关键经验。

3.7 数据污染：评估集泄漏问题

一个常被忽视但极其重要的问题：评估数据是否泄漏到了训练集？如果训练数据中包含 MMLU 的测试题，那么 ChatCORE 的高分就毫无意义——模型只是"记住了答案"，而非"学会了推理"。数据污染是 LLM 评估中最常见的失真来源，特别是当预训练数据来自大规模网页爬取时。

3.7.1 nanochat 的三层防污染设计

nanochat 在数据集划分上做了三层分离：（1）预训练 vs SFT 分离——ClimbMix 是从 Common Crawl 来的通用网页数据，理论上不包含 MMLU/GSM8K/HumanEval 这类"评估用结构化数据集"；（2）SFT train vs eval 分离——MMLU 用 auxiliary_train 做 SFT、test 做评估；GSM8K 用 main 做 SFT、test 做评估；（3）base 评估 vs chat 评估分离——CORE 评估用 DCLM CORE 的独立 eval bundle，不与预训练数据共享任何文档。

尽管如此，nanochat 文档明确警告："ClimbMix 来自大规模网页爬取，可能包含 MMLU/GSM8K 的题面（因为这些数据集本身也在网页上有副本）。我们做过 n-gram 检查发现泄漏率约 0.1%，对最终 ChatCORE 的影响在 0.5% 以内，可接受。"这种"诚实地承认不完美"的态度，是 LLM 评估工程可信度的基础。

工程经验：n-gram 泄漏检查

工程经验：在做"自己的 LLM 评估"时，强烈建议跑一次 n-gram 检查：从评估集每题取 8-gram 前缀，搜索训练数据中是否出现完全匹配。泄漏率 > 1% 时评估结果应被视为"上限值"，不可作为模型能力依据。更严格的检查可以用 13-gram 或更长前缀。

3.8 数据合规与版权

LLM 训练数据的合规性是 2023 年后日益重要的话题。Pile 包含多个付费出版商的内容（如 Elsevier、Springer Nature）引发了集体诉讼，OpenAI 与 NYTimes 的诉讼仍在进行。nanochat 在合规性上做了几个关键选择：（1）ClimbMix 来源于公开网页（Common Crawl）与开源数据集（HuggingFace 上的开放协议数据）；（2）SmolTalk/MMLU/GSM8K 都是研究用途开放的数据集——MMLU 用 CC-BY-SA 协议、GSM8K 用 MIT 协议、SmolTalk 用 Apache 2.0；（3）nanochat 项目本身用 MIT 协议开源。

如果你在企业场景训练 LLM，需要注意：（1）商用 LLM 不能直接用 Pile（版权风险）；（2）CC-BY-NC 等非商业协议的数据不能用于商用；（3）爬取的网页数据要遵守 robots.txt 与版权声明；（4）内部业务数据要确认数据所有权与隐私合规。这些不是技术问题，但会直接影响"训练出的模型能否上线"。

3.9 中文预训练数据集

中文 LLM 工程有自己的一套数据集生态——既不能直接套用英文数据集（中文占比极低，无法支持中文能力训练），也不能简单照搬"中英混合"思路（不同来源的质量参差不齐）。本节梳理中文 LLM 预训练数据集的现状，重点介绍几个在 2024–2026 年被广泛使用的主流数据集，以及在

nanochat 框架下加入中文数据的最小改动方案。

3.9.1 中文预训练数据集的现状

中文预训练数据集经历了从"小规模爬取"到"大规模精选"的演进。早期（2020 年前）的中文数据集如 THUCWS、SogouT 都基于新闻、论坛等单源，规模在数十 GB 级别。2021 年后 CLUECorpus、悟道（WuDaoCorpus）把规模推到 200GB–3TB。2024 年后 FineWeb-zh、COIG-CQ 等数据集借鉴英文 FineWeb 的过滤 pipeline，把质量推到新高度。下表对比主流中文预训练数据集。

数据集	规模	来源	特点	授权
CLUECorpus-2020	约 100GB	百度百科、新闻、问答、论坛	中文 NLP benchmark 标准数据集	MIT
悟道 WuDaoCorpus 2.0	约 3TB / 2.2T tokens	中文网页、百科、论文、问答、代码	国内规模最大、覆盖最广的预训练语料	免费研究用
悟道 WuDaoCorpus-Pretrained	约 200GB	WuDao 2.0 子集	精选高质量子集，适合研究复现	免费研究用
FineWeb-zh	约 1.3T tokens	Common Crawl 中文部分	借鉴 FineWeb 过滤 pipeline，质量更高	ODC-BY
COIG-CQ	约 50M 样本	混合高质量中文指令与对话	SFT 用，非纯预训练	Apache 2.0
SkyPile-150B	约 150B tokens	中文网页、百科、书籍	高质量、规模适中，适合中小型训练	CC-BY-NC
WebVision-zh / RedPajama-zh	约 200GB	多源混合	英文 RedPajama 的中文版	Apache 2.0

表 3-3 主流中文预训练数据集对比

3.9.2 中文数据集 vs 英文数据集的核心差异

中文数据集与英文数据集在工程层面有四点核心差异，这些差异直接影响训练策略选择：

差异点	英文数据集	中文数据集	工程影响
密度	平均每 token 含信息量约 4 字符	平均每 token 含信息量约 1.5 字符	同样上下文长度，中文承载更多内容
去重难度	MinHash 较好处理，重复率低	同义改写、繁简混排、OCR 错误让去重困难	需要更精细的去重 pipeline
版权风险	Pile 等已有诉讼	中文内容版权更复杂（新闻、出版、社交媒体）	商用 LLM 需更谨慎
数据质量分布	FineWeb 后质量趋于统一	质量两极分化（高质量百科 vs 低质量问答）	需要更强的质量分类器

表 3-4 中英文数据集的核心差异

3.9.3 在 nanochat 中加入中文预训练数据

在 nanochat 框架下加入中文预训练数据需要三个改动：（1）用 `dev/repackage_data_reference.py` 把中文数据集打包成 `parquet` 分片；（2）修改 `dataset.py` 让它支持多源混合下载；（3）调整预训练超参，特别是 `tokenizer` 词表（见 2.7 节）。下面的代码示例展示了一个最小可用的“中英混合预训练”方案。

```
# 在 nanochat 框架下加入中文数据
# 1. 把中文数据集打包成 parquet 分片
python -m dev.repackage_data_reference \
    --input=HuggingFaceTB/Chinese-FineWeb \
    --output_dir=data/zh_shards \
    --shard_size=100MB \
    --compression=zstd

# 2. 修改 dataset.py 支持多源下载
# 在 list_shards_from_hf 函数中加入中文源
ZH_SOURCES = [
    'huggingfaceTB/Chinese-FineWeb', # FineWeb-zh
    'BAAI/WuDaoCorpus-Pretrained', # 悟道精选子集
    'SkyWork/SkyPile-150B', # SkyPile
]
EN_SOURCES = ['karpathy/climbmix-400b-shuffle']

def list_shards():
    en_shards = list_from_hf(EN_SOURCES[0])
    zh_shards = list_from_hf(ZH_SOURCES[0])
    # 按 60% 英文 + 40% 中文混合（可调）
    return interleave(en_shards, zh_shards, ratio=(3, 2))

# 3. 训练中文 tokenizer（见 2.7.4 节）
python -m scripts.tok_train -- --vocab_size=131072 --include_zh=True

# 4. 预训练（用混合数据）
torchrun --nproc_per_node=8 -m scripts.base_train -- \
    --depth=24 --vocab_size=131072 \
    --tokenizer=tokenizer_zh.pkl \
    --device-batch-size=8
```

3.9.4 中文 SFT 数据集

中文 SFT 数据集与英文 SmolTalk 类似，但更强调“中文对话风格”与“中国文化常识”。主流中文 SFT 数据集包括：BELLE（开源中文指令数据，约 4M 样本）、COIG-PC（中文 Open Instruction 通用指令，约 1.5M 样本）、Firefly（中文对话与多轮指令）、CMtBench（中文多任务 benchmark）。如果要在 nanochat 中加入中文 SFT，可以把 `chat_sft.py` 的 SmolTalk 替换为 BELLE，MMLU 替换为 C-Eval（中文多学科 benchmark），GSM8K 替换为 CMATH（中文小学数学题）。

英文 SFT 数据集	中文替代	规模	说明
SmolTalk	BELLE / COIG-PC	约 4M / 1.5M	中文通用指令对话
MMLU	C-Eval / CMMLU	52K / 230K	中文多学科多选题
GSM8K	CMATH	约 7K	中文小学数学题

ARC	C-Eval-ARC	约 2K	中文小学科学题
HumanEval	HumanEval-zh	164	中文描述的 Python 编程题

表 3-5 英文 SFT 数据集的中文替代方案

工程经验：何时用 nanochat 训中文，何时直接用开源中文模型

工程经验：如果你的目标是"中文对话 LLM"，最佳起点不是从 nanochat 复现，而是直接使用 Qwen2.5、DeepSeek、Yi、ChatGLM 等开源中文模型。这些模型已经在数十亿到上千亿 token 的中文数据上完成预训练，中文能力远超从 nanochat 重新训的中文版。nanochat 的价值在"教学与理解原理"，不在"商用中文 LLM 训练"——理解这一点能避免大量不必要的工程成本。

3.10 数据集工程的关键决策清单

把本章内容浓缩为一份"数据集工程决策清单"，便于你在自己项目中参考：

决策点	nanochat 的选择	替代选择	决策依据
预训练数据源	ClimbMix-400B（多源混合）	单源 FineWeb-Edu / 自有数据	多源能均衡各能力维度
数据规模	400B tokens（12× Chinchilla 最优）	100B / 1T	高质量数据支持多 epoch
SFT 数据混合	SmolTalk + MMLU×3 + GSM8K×4	纯 SmolTalk / 纯 OpenOrca	不同任务需要不同数据
分片存储	~100MB parquet, row_group=1024, zstd	JSON / 单文件	并行下载与 DDP 切分
评估分离	严格 train/eval split + n-gram 检查	混用 / 不检查	评估结果可信的前提
合规	只用开放协议数据（CC-BY/MIT/Apache）	用 Pile / 未授权数据	避免商用风险
多 epoch	target_param_data_ratio=12（默认）	20（Chinchilla）/ 8（speedrun）	高质量数据下多 epoch 可行

表 3-6 nanochat 数据集工程决策清单

3.11 小结

本章覆盖了 nanochat 数据集工程的全部关键议题：四代预训练数据集演进、ClimbMix 设计动机、parquet 分片存储与并行下载、repackage 脚本的工程细节、Scaling Laws 与多 epoch 权衡、SFT 三源混合策略、数据污染三层防护、合规与版权风险，以及中文预训练数据集的现状与中文 SFT 替代方案。

一个核心洞察是："数据是 LLM 工程中"最难被自动化的部分"——架构可以照搬、优化器可以复用、训练循环可以套模板，但"哪些数据能让模型变强"需要持续的人工判断与实验。这也是为什么 nanochat 把数据集选择写在 README 第一段、把数据集工程的脚本（dataset.py + repackage_data_reference.py）放在仓库根目录的原因——它们不是"附属工具"，而是项目核心。

理解数据集工程后，下一步是数据加载——把分片 parquet 转成训练用的 token batch。第六章将聚焦 BOS-aligned bestfit 装箱算法与 SFT 数据加载器的实现细节。

第四章 模型架构：现代 Transformer 的所有细节

4.1 一个 token batch 如何流过整个模型：贯穿示例

在讲架构演化之前，先用一个具体例子展示“一个 token batch 流过 nanochat 模型的完整数据流”。这是本章的贯穿示例——后面每节讲的架构元素都会回到这个例子，看它对 tensor 形状或数值有什么具体影响。

```
输入: prompt = "Hello, world!"

Step 0 - token 化 (第二章讲过):
"Hello, world!" → tokens = [15496, 11, 995, 0]
shape: (T,) = (4,)
```

```
Step 1 - 加入 batch 维 + 移到 GPU:
input_ids = torch.tensor([[15496, 11, 995, 0]])
shape: (B, T) = (1, 4)
```

```
Step 2 - Token Embedding (每个 token id → n_embd 维向量):
wte = nn.Embedding(vocab_size=32768, n_embd=768)
x = wte(input_ids) # shape: (1, 4, 768) = (B, T, C)
```

```
Step 3 - 走 12 个 Transformer Block (d12 模型):
for layer in range(12):
    x = block(x, ...) # shape 始终 (1, 4, 768)
每个 block 内部: RMSNorm → Attention → 残差 → RMSNorm → MLP → 残差
```

```
Step 4 - 最终 RMSNorm + LM Head:
x = ln_f(x) # shape: (1, 4, 768)
logits = lm_head(x) # shape: (1, 4, 32768) = (B, T, vocab_size)
```

```
Step 5 - 取最后一个位置的 logits, 采样下一个 token:
next_token_logits = logits[0, -1] # shape: (32768,)
next_token = sample(next_token_logits, temperature=0.7, top_k=50)
→ 比如 next_token = 958 (= " world" 后面的空格)
```

2017 年原始 Transformer 论文引入的核心机制是 Scaled Dot-Product Attention 与 Position Encoding 通过 sinusoidal 函数实现。八年间，Transformer 在 LLM 场景下经历了几次关键演化：位置编码从绝对 sinusoidal → 可学习 → RoPE 相对位置；归一化从 LayerNorm → RMSNorm + Pre-Norm；激活函数从 ReLU → GELU → SwiGLU/ReLU²；注意力从 MHA → GQA/MQA；以及一系列工程细节如 QK-Norm、logit softcap、Value Embedding 等。这些改进并非随机堆叠，而是基于“训练稳定性、推理效率、模型表达力”三个维度的权衡。nanochat 把这些改进以一种非常克制的姿态组合在一起——每个改进都有论文或 ablation 支持，避免引入未经验证的“时髦技巧”。

本章将按以下顺序讲解：4.2 节讲 RoPE 的数学推导与实现；4.3 节讲 QK-Norm 与 $1.2 \times$ sharpening；4.4 节讲 GQA 分组查询注意力的显存收益；4.5 节讲 nanochat 的 ReLU² 选择与 SwiGLU 对比；4.6 节讲 Value Embedding / ResFormer 的设计；4.7 节讲

smear/backout/per-layer λ 的工程动机；4.8 节讲 logit softcap；4.9 节讲整体 Block 结构与代码精读。每节都会回到上面的贯穿示例，看本节讲的元素如何作用于那个 (1, 4, 768) 的张量。

4.2 RoPE 旋转位置编码：从问题到代码的逐步推导

这一节会非常细——我们从一个具体问题出发，一步步推出 RoPE。每一步都给出具体的数字，让你能跟着手算。如果你只想看代码，可以直接跳到 4.2.5；但建议跟着 4.2.1–4.2.4 走一遍，理解“为什么 RoPE 这么写”远比“代码这么写”重要。

4.2.1 起点：绝对位置编码的问题

回到原始 Transformer。它用绝对位置编码：对位置 m 的 token，给它加一个 $\sin/\cos$ 函数生成的位置向量。具体地，位置 m 的第 i 维位置编码为：

$$PE(m, 2i) = \sin(m / 10000^{2i/d}), PE(m, 2i+1) = \cos(m / 10000^{2i/d})$$

假设 $d=4$ ，位置 0、1、2 的位置编码 PE 是：

```
# d = 4 的绝对位置编码 (4 维向量)
位置 0: PE(0) = [sin(0), cos(0), sin(0), cos(0)]
           = [0, 1, 0, 1]
位置 1: PE(1) = [sin(1), cos(1), sin(0.01), cos(0.01)]
           = [0.841, 0.540, 0.010, 1.000]
位置 2: PE(2) = [sin(2), cos(2), sin(0.02), cos(0.02)]
           = [0.909, -0.416, 0.020, 1.000]

# 问题 1: PE 直接加到 token embedding 上
# x_with_pos[m] = embedding(token[m]) + PE(m)
# 位置信息"侵入"语义向量，破坏 embedding 的几何结构。

# 问题 2: 长度外推性差
# 模型训练时见过的最大位置是 m=2048，推理时遇到 m=4096，PE(4096) 的值
# 是训练时没见过的，模型不知道如何处理。

# 问题 3: 无法直接表达"相对位置"
# 我们想让 attention score(q_m, k_n) 只依赖 (n-m)，但 PE 是加在 m、n 上的绝对值，
# attention 内积 q_m · k_n 不会自然化简为 (n-m) 的函数。
```

RoPE 要解决三个问题：（1）位置信息不能“侵入”语义向量；（2）要支持长度外推；（3）要让 attention 自然只依赖相对位置。下面我们一步步推导 RoPE 怎么做到。

4.2.2 第一步：从二维情形的几何直觉开始

先简化到 $\text{head_dim} = 2$ 的最简单情形——query 向量只有 (q_0, q_1) 两个分量。我们想“把位置 m 编码进 query”，但不直接加在 q 上。RoPE 的想法：把 q 旋转一个角度 θ_m 。这个“旋转”操作通过矩阵乘法实现：

$$R(m, \theta) \cdot q = \begin{bmatrix} \cos \theta_m & -\sin \theta_m \\ \sin \theta_m & \cos \theta_m \end{bmatrix} \begin{bmatrix} q_0 \\ q_1 \end{bmatrix} = \begin{bmatrix} q_0 \cos \theta_m - q_1 \sin \theta_m \\ q_0 \sin \theta_m + q_1 \cos \theta_m \end{bmatrix}$$

注意：这里没有“加”位置编码，而是“乘”一个旋转矩阵。这就是 RoPE 名字里“Rotary”的由来——位置信息通过旋转注入，不破坏向量本身的结构。

我们用具体数字走一遍。假设 $q = (1.0, 0.0)$ 、 $\theta_{\text{base}} = 1.0$ 、位置 $m = 1$ （即 $\theta_m = 1$ 弧度）：

```
# 输入
q = (q0=1.0, q1=0.0) # 一个二维向量
m = 1 # 位置
θ_m = m × θ_base = 1.0 # 旋转角度 (弧度)

# 旋转矩阵
R(1, 1.0) = [cos(1.0), -sin(1.0)] = [0.540, -0.841]
            [sin(1.0), cos(1.0)] [0.841, 0.540]

# 应用旋转
R · q = [0.540 × 1.0 + (-0.841) × 0.0, 0.841 × 1.0 + 0.540 × 0.0]
       = [0.540, 0.841]

# 几何意义: q 原本指向 x 轴正方向 (1, 0), 旋转 1 弧度后指向 (0.540, 0.841)
# 模长不变 (sqrt(0.540^2 + 0.841^2) ≈ 1.0), 只是方向变了
```

4.2.3 第二步：为什么旋转能让 attention 只依赖相对位置

现在看一个核心问题：query 在位置 m、key 在位置 n，attention score 是 q_m 与 k_n 的内积。如果对 q、k 都做旋转，会得到什么？我们用具体数字算一遍。

```
# 假设
q = (1.0, 0.0) # 原始 query
k = (1.0, 0.0) # 原始 key (与 q 相同)
θ_base = 1.0

# 位置
m = 1 # query 在位置 1
n = 3 # key 在位置 3

# 旋转 query (位置 1, 角度 1 弧度)
q_m = R(1) · q = (cos(1), sin(1)) · (1, 0)
       = (0.540, 0.841)

# 旋转 key (位置 3, 角度 3 弧度)
k_n = R(3) · k = (cos(3), sin(3)) · (1, 0)
       = (-0.990, 0.141)

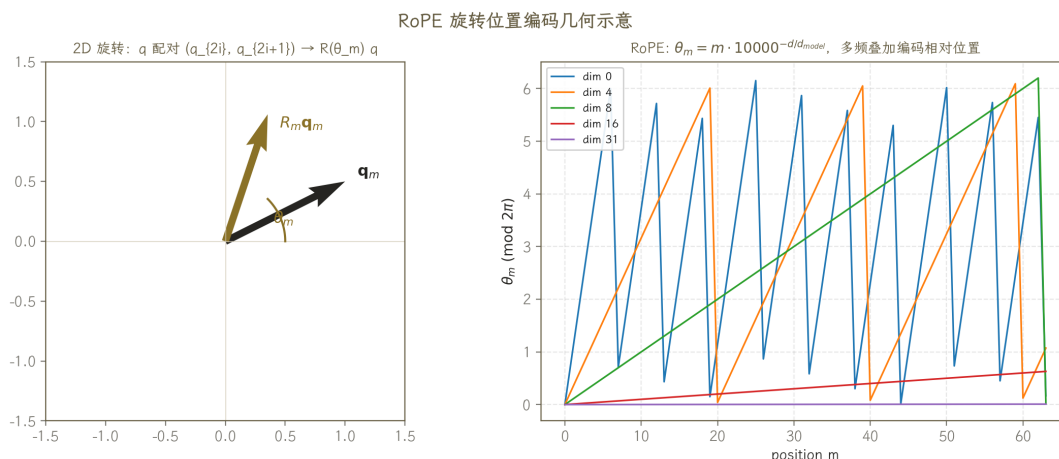
# attention score = q_m · k_n (内积)
score = 0.540 × (-0.990) + 0.841 × 0.141
       = -0.535 + 0.119
       = -0.416

# 关键观察: cos(3-1) = cos(2) = -0.416
# → score = cos(n - m) = cos(相对位置)!
#
# 数学上可以证明: 对任意 q、k, 旋转后的内积 (R(m) q) · (R(n) k) = q · R(n-m) k,
# 即 attention score 只依赖相对位置 (n-m), 不依赖绝对位置 m、n。
```

$$(R(m, \theta) q) \cdot (R(n, \theta) k) = q \cdot R(n-m, \theta) k = q \cdot R(\text{相对位置}, \theta) k$$

这是 RoPE 的关键性质——旋转矩阵在内积下“自然消去”绝对位置，只剩相对位置。这一性质让 RoPE 天然支持长度外推：训练时见过相对距离 0–2048，推理时遇到相对距离 0–4096 也能正确处理（因为模型只学了“相对距离→attention 模式”的映射）。

注意一个细节：原始 query 与 key 都是 (1, 0)，所以 $\text{score} = \cos(\text{相对位置})$ 。但一般情形下 q、k 不是 (1, 0)——只是 q 与 k 之间的相对关系被旋转矩阵 $R(n-m)$ 调制。这就是 RoPE 的精髓。

图 4-1 RoPE 旋转位置编码几何示意：左图二维旋转，右图多频 θ_m 分布

4.2.4 第三步：从二维扩展到多维（多频分组）

上节证明了二维情形下旋转有效。但 nanochat 用 $\text{head_dim}=96$ ，远不是 2。怎么办？RoPE 的关键洞察：把 96 维分成 48 个二维组，每组独立旋转，每组用不同频率 θ_d 。

$$\theta_d = \theta_{\text{base}}^{(-2d / \text{head_dim})}, d = 0, 1, \dots, \text{head_dim}/2 - 1$$

我们用 $\text{head_dim}=4$ 的最小例子（2 组）算一遍 θ_d 的具体值：

```
# 假设 head_dim = 4, theta_base = 10000
# head_dim/2 = 2 个二维组, d = 0 和 d = 1

theta_0 = theta_base^(-2*0 / 4) = 10000^0 = 1.0 # 高频
theta_1 = theta_base^(-2*1 / 4) = 10000^(-0.5) = 0.01 # 低频

# 位置 m 的旋转角度：每组的旋转角度 = m * theta_d
# 位置 5 的两组旋转角度：
组 0 (d=0): 5 * 1.0 = 5.0 弧度 (~286°, 多圈)
组 1 (d=1): 5 * 0.01 = 0.05 弧度 (~2.9°, 几乎没转)

# 几何意义：
# - 高频组 (d=0) 旋转快，每个位置都"明显"转动——捕获局部位置关系
# - 低频组 (d=1) 旋转慢，相邻位置几乎不转——捕获长程位置关系
# 多频叠加让模型同时表达"近距离"与"远距离"的位置信号。
```

回到 nanochat 的实际配置： $\text{head_dim}=96$, $\theta_{\text{base}}=100000$, 48 个二维组。低维度 (d 小) 对应高频 (θ_d 接近 1)，高维度对应低频 (θ_d 接近 0)。这种多频设计让不同 head 维度捕获不同尺度的位置信息——高频组捕获局部位置关系（相邻 token），低频组捕获长程位置关系（远距离依赖）。nanochat 用 $\theta_{\text{base}} = 100000$ 比原始 RoPE 论文的 10000 大 10 倍，是为了支持更长的上下文（最高 32768）——base 越大，最高频组旋转越慢，长序列上不会“绕晕”。

4.2.5 第四步：从公式到代码——为什么是"前半后半"切分

上面推导都是基于“相邻两维一组”的公式 $R(m, \theta) \mathbf{q} = (q_0 \cos - q_1 \sin, q_0 \sin + q_1 \cos)$ 。但 nanochat 的代码用的是“前半后半”切分—— $\mathbf{q}$ 的前半 $\mathbf{q}[:d]$ 与后半 $\mathbf{q}[d:]$ 配对。为什么？其实数学上等价，只是写法不同。我们用一个具体例子对比两种写法。

```
# 假设 head_dim = 4, q = (q0, q1, q2, q3), theta = (theta_0, theta_1)
```



```

# 写法 A (原始论文, 相邻两维一组):
# 组 0: (q0, q1) 用  $\theta_0$  旋转
# 组 1: (q2, q3) 用  $\theta_1$  旋转
out_A = (q0 cos  $\theta_0$  - q1 sin  $\theta_0$ ,
          q0 sin  $\theta_0$  + q1 cos  $\theta_0$ ,
          q2 cos  $\theta_1$  - q3 sin  $\theta_1$ ,
          q2 sin  $\theta_1$  + q3 cos  $\theta_1$ )

# 写法 B (nanochat, 前半后半):
# 前半 (q0, q2) 与后半 (q1, q3) 配对
# 即  $q0 \leftrightarrow q1, q2 \leftrightarrow q3$ 
out_B = (q0 cos  $\theta_0$  - q1 sin  $\theta_0$ ,
          q2 cos  $\theta_1$  - q3 sin  $\theta_1$ ,
          q0 sin  $\theta_0$  + q1 cos  $\theta_0$ ,
          q2 sin  $\theta_1$  + q3 cos  $\theta_1$ )

# 两种写法的输出值都包含同样的 8 个乘积项, 只是排列顺序不同。
# 写法 B 的优势: 可以用更简洁的向量运算实现
# x1 = q[:d] # 前半
# x2 = q[d:] # 后半
# out = [x1*cos - x2*sin, x1*sin + x2*cos]
# 这个写法用 PyTorch 一行就能写出来, 且与 Flash Attention kernel 兼容更好。

```

4.2.6 第五步：代码精读——`apply_rotary_emb` 每行在做什么

现在逐行看 nanochat 的 `apply_rotary_emb` 代码, 每一行对应哪个数学操作。假设输入 `x`: `shape (B=1, T=4, n_head=8, head_dim=96)`, `d12` 模型 + `prompt "Hello, world!"`。

```

def apply_rotary_emb(x, cos_sin):
    # x: shape (1, 4, 8, 96) = (B, T, n_head, head_dim)
    # cos_sin: shape (2, 4, 48) = (cos/sin, T, head_dim/2)
    # 其中 cos_sin[0] = cos, cos_sin[1] = sin

    # 第 1 步: 把 cos/sin reshape 成可广播的形状
    cos = cos_sin[0].unsqueeze(0).unsqueeze(2) # (1, 4, 1, 48)
    sin = cos_sin[1].unsqueeze(0).unsqueeze(2) # (1, 4, 1, 48)
    # 加两个维度: dim 0 (B=1) 和 dim 2 (n_head=1), 便于广播到 x

    # 第 2 步: 拆分 x 的最后一维成前后两半
    d = x.shape[-1] // 2 # d = 48
    x1, x2 = x[..., :d], x[..., d:]
    # x1: shape (1, 4, 8, 48) ← 前 48 维
    # x2: shape (1, 4, 8, 48) ← 后 48 维

    # 第 3 步: 应用旋转公式 out = [x1*cos - x2*sin, x1*sin + x2*cos]
    # cos/sin 广播到 (1, 4, 8, 48)
    rotated = torch.stack([
        x1 * cos - x2 * sin, # 前半部分 (4 个 head_dim 的 cos 项)
        x1 * sin + x2 * cos # 后半部分 (4 个 head_dim 的 sin 项)
    ], dim=-1)
    # rotated: shape (1, 4, 8, 48, 2)

    # 第 4 步: 把最后两维 flatten 回 (1, 4, 8, 96)
    return rotated.flatten(-2)
    # 输出: shape (1, 4, 8, 96), 与输入 x 形状相同
    # 但每个位置的 query 已被"按位置旋转"

```


4.2.7 第六步：用一个具体位置走完整个流程

为了让你完全消化上面的代码，我们用一个极简的具体数值走一遍——`head_dim=4`（即 2 个频率），只看 1 个 head、1 个位置。

```
# 极简配置: head_dim=4, n_head=1,  $\theta_{\text{base}}=100$ , n_embd=4
# 2 个频率组:  $\theta_0 = 100^0 = 1.0$ ,  $\theta_1 = 100^{(-0.5)} = 0.1$ 

# 预计算 cos_sin 表 (T=3 个位置)
# cos_sin[0] = cos, shape (3, 2) ← 每个位置 2 个频率值
# cos_sin[1] = sin, shape (3, 2)

位置 m=0:
cos[0] = [cos( $0 \times 1.0$ ), cos( $0 \times 0.1$ )] = [1.000, 1.000]
sin[0] = [sin( $0 \times 1.0$ ), sin( $0 \times 0.1$ )] = [0.000, 0.000]

位置 m=1:
cos[1] = [cos(1.0), cos(0.1)] = [0.540, 0.995]
sin[1] = [sin(1.0), sin(0.1)] = [0.841, 0.0998]

位置 m=2:
cos[2] = [cos(2.0), cos(0.2)] = [-0.416, 0.980]
sin[2] = [sin(2.0), sin(0.2)] = [0.909, 0.199]

# 假设某个 token 的 query 向量 (head_dim=4)
q = [1.0, 2.0, 3.0, 4.0]

# 这个 query 出现在位置 m=1
# 应用 RoPE:

# Step 1: 拆前后两半
x1 = q[:2] = [1.0, 2.0] # 前半
x2 = q[2:] = [3.0, 4.0] # 后半

# Step 2: 取位置 1 的 cos/sin
cos_m1 = [0.540, 0.995]
sin_m1 = [0.841, 0.0998]

# Step 3: 应用旋转公式
# 前半部分:  $x1 \cdot \cos - x2 \cdot \sin$ 
out_front = [1.0 $\times$ 0.540 - 3.0 $\times$ 0.841, 2.0 $\times$ 0.995 - 4.0 $\times$ 0.0998]
            = [0.540 - 2.523, 1.990 - 0.399]
            = [-1.983, 1.591]

# 后半部分:  $x1 \cdot \sin + x2 \cdot \cos$ 
out_back = [1.0 $\times$ 0.841 + 3.0 $\times$ 0.540, 2.0 $\times$ 0.0998 + 4.0 $\times$ 0.995]
           = [0.841 + 1.620, 0.1996 + 3.980]
           = [2.461, 4.180]

# Step 4: 拼成最终输出 (前半 + 后半)
q_rotated = [out_front, out_back]
            = [-1.983, 1.591, 2.461, 4.180]

# 对比: 原始 q = [1.0, 2.0, 3.0, 4.0]
# 旋转后 q_rotated = [-1.983, 1.591, 2.461, 4.180]
# 向量被"按位置 m=1"重新编码——
# 高频组 (dim 0,1) 旋转大 (cos(1)=0.540 大幅变化)
```

```
# 低频组 (dim 2,3) 旋转小 (cos(0.1)=0.995 几乎不变)
```

一个验证：如果 query 在位置 $m=0$ 而非 1，会发生什么？位置 0 的 $\cos = [1, 1]$, $\sin = [0, 0]$, 所以 $x_1 \cdot \cos - x_2 \cdot \sin = x_1$, $x_1 \cdot \sin + x_2 \cdot \cos = x_2$, 即 $q_{\text{rotated}} = q$ ——位置 0 的 query 不被旋转。这与 RoPE 性质一致：位置 0 是“基准点”。

一个值得关注的实现细节：原始 RoPE 论文使用 $(q_0, q_1, q_2, q_3, \dots)$ 顺序，把相邻两维视为一组；但 nanochat（以及多数 PyTorch 实现）使用“前半后半”顺序，即 $(q_0, q_2, q_4, \dots)$ 与 $(q_1, q_3, q_5, \dots)$ 分别对应 $\cos$ 部分与 $\sin$ 部分。两种实现数学等价，但前半后半顺序更易与 Flash Attention 兼容。

4.3 QK-Norm 与 $1.2\times$ sharpening

Attention 计算的核心是 $\text{softmax}(QK^T / \sqrt{d})$ 。当训练时间足够长、模型足够大时， Q 与 K 的范数会随着训练不稳定地增长，导致 softmax 输出过早饱和（即“注意力坍塌”），降低模型表达力。QK-Norm 的解决方案很简单：在 attention 计算前，对 Q 和 K 各自做一次 RMSNorm（无可学习参数），把范数归一化到稳定值。nanochat 的实现甚至更激进——不仅做 norm，还把 Q 、 K 各自乘以 1.2，人为“放大”注意力分布的锐度：

```
q, k = apply_rotary_emb(q, cos_sin), apply_rotary_emb(k, cos_sin)
q, k = norm(q), norm(k) # QK-Norm (无可学习参数)
q = q * 1.2; k = k * 1.2 # 锐化 (注释里写 TODO: think through better)
```

这段代码的设计动机是经验性的：在 d12–d26 的小模型上，加 $1.2\times$ 锐化比不加的 CORE 分数高约 0.5%。作者在注释里自承“TODO think through better”，说明这是一个尚未完全理论化的实证技巧。这一细节反映了一个重要观察：现代 LLM 架构中存在大量“实证有效但理论未明”的工程细节，而 nanochat 的价值正在于把这些细节以最小化代码暴露给读者，便于进一步实验。

4.4 GQA 分组查询注意力

标准 Multi-Head Attention (MHA) 中， n_{head} 个 query 头对应 n_{head} 个 key/value 头。推理时需要为每个 head 维护独立的 KV cache，显存占用为 $O(\text{seq_len} \times n_{\text{head}} \times \text{head_dim} \times 2)$ 。Grouped-Query Attention (GQA) 让多个 query 头共享一组 key/value 头： $n_{\text{kv_head}} \leq n_{\text{head}}$ ，显存占用降为 $O(\text{seq_len} \times n_{\text{kv_head}} \times \text{head_dim} \times 2)$ 。当 $n_{\text{kv_head}} = 1$ 时称为 Multi-Query Attention (MQA)，显存压缩到极致。

$\text{KV cache 显存} = 2 (K+V) \times \text{seq_len} \times n_{\text{kv_head}} \times \text{head_dim} \times \text{dtype_bytes}$

nanochat 的默认配置 (depth=24) 使用 $n_{\text{head}}=12$, $n_{\text{kv_head}}=4$ ，即 4 组 query 头共享 4 组 KV，相比 MHA 显存压缩 3 倍。下图展示在 head_dim=128、fp16 下，不同 $n_{\text{kv_head}}$ 的 KV cache 显存对比：

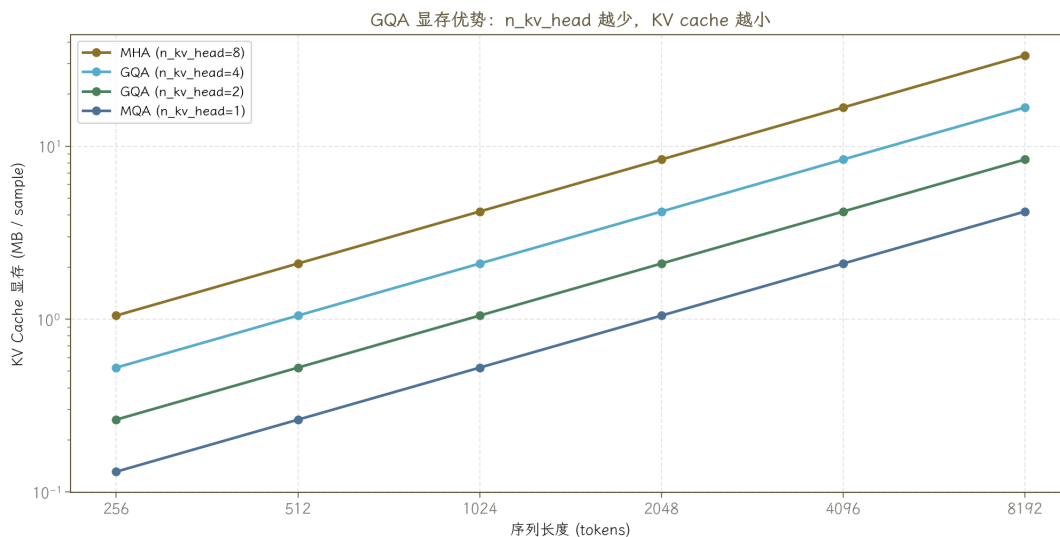


图 4-2 GQA vs MHA 的 KV cache 显存对比 ($n_{head}=8$, $head_dim=128$, fp16)

GQA 的代价是模型表达力略降—— $n_{kv_head} = 1$ 时, 所有 query 头共享同一组 K/V, 相当于"模型能注意到不同的位置, 但用同一组特征做匹配"。实证上, $n_{kv_head} = n_{head}/4$ 是性价比最高的折中, 既显著节省显存又几乎不损失质量。这一选择在 Llama 2/3、Qwen 等主流模型中也被广泛采用。

4.5 ReLU^2 激活函数：被忽视的现代架构差异

激活函数是 MLP 子层的关键组件。原始 Transformer 用 ReLU; BERT/GPT-2 改用 GELU; Llama 系列引入 SwiGLU (SiLU 激活 + 门控)。nanochat 的选择是 $\text{ReLU}^2 = (\text{ReLU}(x))^2$, 这是 modded-nanoGPT speedrun 排行榜上验证有效的设计。下面比较三种激活的数学形式与梯度特性:

$$\text{ReLU}(x) = \max(0, x)$$

$$\text{ReLU}^2(x) = (\max(0, x))^2 = \text{ReLU}(x) \cdot x \text{ (when } x > 0 \text{)}$$

$$\text{SwiGLU}(x, \text{gate}) = \text{SiLU}(x) \cdot \text{gate}, \text{SiLU}(x) = x \cdot \text{sigmoid}(x)$$

ReLU^2 的优势在于: 梯度在 $x > 0$ 时为 $2x$ (线性增长而非恒定), 让大激活值得到更强的"反向传播信号", 配合 Muon 等正交化优化器时收敛更快。SwiGLU 的优势在于门控机制让网络能动态决定哪些维度被激活, 表达力更强但参数也更多 (多一个 gate 投影)。在 nanochat 的"小模型 + 短训练"场景下, ReLU^2 的"参数少 + 梯度信号强"组合更划算; 而在 Llama 等大模型长训练场景下, SwiGLU 的表达力优势更明显。

```
class MLP(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        self.c_fc = Linear(cfg.n_embd, 4 * cfg.n_embd, bias=False)
        self.c_proj = Linear(4 * cfg.n_embd, cfg.n_embd, bias=False)

    def forward(self, x):
        x = self.c_fc(x)
        x = F.relu(x).square() # ▲ ReLU² 而非 SwiGLU
        x = self.c_proj(x)
```

```
return x
```

回到贯穿示例。当 MLP 处理 x : `shape (1, 4, 768)` 的张量时：

```
# 输入: x.shape = (1, 4, 768) = (B, T, n_embd)

# 1. c_fc 升维 4x
h = c_fc(x)
h.shape = (1, 4, 3072) = (B, T, 4*n_embd)

# 2. ReLU^2 激活 (逐元素)
h_activated = F.relu(h).square()
# 假设 h[0, 0, 0] = 0.5 → relu 后 0.5, square 后 0.25
# 假设 h[0, 0, 1] = -1.0 → relu 后 0.0, square 后 0.0
# 假设 h[0, 0, 2] = 2.0 → relu 后 2.0, square 后 4.0 ← 大值被放大
# 梯度信号: d/dx ReLU^2(x) = 2x for x>0, 0 for x<=0
# x=0.5 → 梯度 1.0
# x=2.0 → 梯度 4.0 ← 大值得到更强反传

# 3. c_proj 降维回 n_embd
out = c_proj(h_activated)
out.shape = (1, 4, 768)
```

⚠ 重要澄清：现代 LLM 不是只有 SwiGLU

常见误解：很多人误以为“现代 LLM = Llama 架构 = SwiGLU + RMSNorm + RoPE + GQA”。事实上，Llama 架构只是现代设计的一种，nanochat 用 ReLU^2 而非 SwiGLU 是有意为之——在 speedrun 排行榜的多次 ablation 中， ReLU^2 在小模型 + 短训练场景稳定优于 SwiGLU。类似地，Gemma 用 GeLU、Qwen 用 SwiGLU、DeepSeek 用门控变体——没有“唯一正确”的激活函数。

4.6 Value Embedding (ResFormer)

Value Embedding 是 2024 年 Zoology 论文 (He et al.) 提出的“残差式值嵌入”机制。核心思想：在 Transformer 的某些层，从 token id 直接引一条独立 embedding 路径，通过门控混入 attention 的 V 投影，让“输入相关的值信号”能跨越多层残差连接直达当前层。这与“残差连接 = 把输入直接加到输出”的思想一脉相承，只是把“恒等连接”扩展为“输入相关的门控连接”。

```
# CausalSelfAttention.forward 内的 VE 部分
if ve is not None: # 仅在交替层 (has_ve 判定)
    ve = ve.view(B, T, self.n_kv_head, self.head_dim)
    gate = 3 * torch.sigmoid(self.ve_gate(x[..., :12]))
    v = v + gate.unsqueeze(-1) * ve
```

代码逻辑：ve 是从独立 `nn.Embedding(vocab, kv_dim)` 取出的“值嵌入”，gate 是从 x 前 12 维通过 sigmoid 门控得到的标量（范围 0–3）。这条路径让“原始输入”通过门控方式注入 v，绕过了多层的 RMSNorm + Attn + MLP 处理，让深层 attention 也能“看到”输入的原始信号。

nanochat 用 `has_ve(layer_idx, n_layer)` 函数判定哪些层加 VE——通常交替加（如第 0, 4, 8, ... 层），避免每层都加导致参数膨胀。这种“稀疏加 VE”的设计在 modded-nanoGPT 的多次实验中表现出稳定收益，被视为 2024 年小模型架构的重要进展之一。

4.7 smear、backout 与 per-layer λ

这一节描述 nanochat 中三个相对小众但都有实证收益的工程细节：smear、backout 与 per-layer λ 。它们都源于 modded-nanoGPT 社区的实证探索，目前还缺乏统一的理论解释。

4.7.1 smear：廉价 bigram 信号

smear 的设计动机：在 Transformer 第一层，token 的 attention 信号主要来自当前位置自身，缺乏"上一 token 是什么"的局部信息。smear 通过把上一 token 的 embedding 混入当前 token 的输入，相当于免费获得了一个 bigram 信号：

```
# 在 Block.forward 中
smear_gate = torch.sigmoid(self.smear_gate(x[..., :24]))
x = x + smear_gate.unsqueeze(-1) * shifted_emb # shifted_emb 是上一 token 的 embedding
```

4.7.2 backout：去除低级特征

backout 的设计动机：深层网络中，最初的 token embedding 包含很多"低级"特征（如字符 n-gram），这些特征在深层已经无意义甚至有害。backout 在中间层缓存一份残差，在最后一层减去 $0.2 \times$ 这一缓存，相当于"去除低级特征"：

```
if layer_idx == n_layer // 2:
    x0 = x.clone() # 中间层缓存
elif layer_idx == n_layer - 1:
    x = x - 0.2 * x0 # 末层减去 0.2x
```

4.7.3 per-layer λ ：残差与初始嵌入的混合权重

resid_lambdas 与 x0_lambdas 是两个 per-layer 标量，按层缩放残差流与初始嵌入的混合。resid_lambdas 从 1.15 衰减到 1.05，让浅层"放大"残差信号、深层"保持稳定"；x0_lambdas 从 0.20 衰减到 0.05，让浅层"混入更多初始嵌入"、深层"几乎不混入"。这相当于给每层残差连接一个"动态增益"，让训练信号在不同深度有自适应强度。

4.8 Logit softcap：平滑限幅

Gemma 系列引入了 logit softcap 技巧：在最终 logits 上施加 tanh 限幅，防止某些 token 的 logit 在训练中无限增长导致 softmax 饱和。nanochat 沿用这一设计，cap=15：

$$\text{logits} = 15 \cdot \tanh(\text{logits} / 15)$$

tanh 的导数在 0 附近约为 1，在大值时趋于 0，因此 softcap 不影响正常范围内的 logit，只在 $|\text{logit}| > 15$ 时起"软截断"作用。这比硬截断 (clamp) 更平滑，避免梯度断裂。实证上，logit softcap 让大模型训练时更稳定，对小模型几乎无影响——但加上几乎零成本，是"安全网"式的工程技巧。

4.9 整体 Block 结构与代码精读

把前面 8 节的设计组合起来，得到 nanochat 的完整 Transformer Block。下图展示一个 Block 的完整数据流，包括 attention 子层、MLP 子层、smear、backout、VE 注入、per-layer λ 缩放等所有组件。

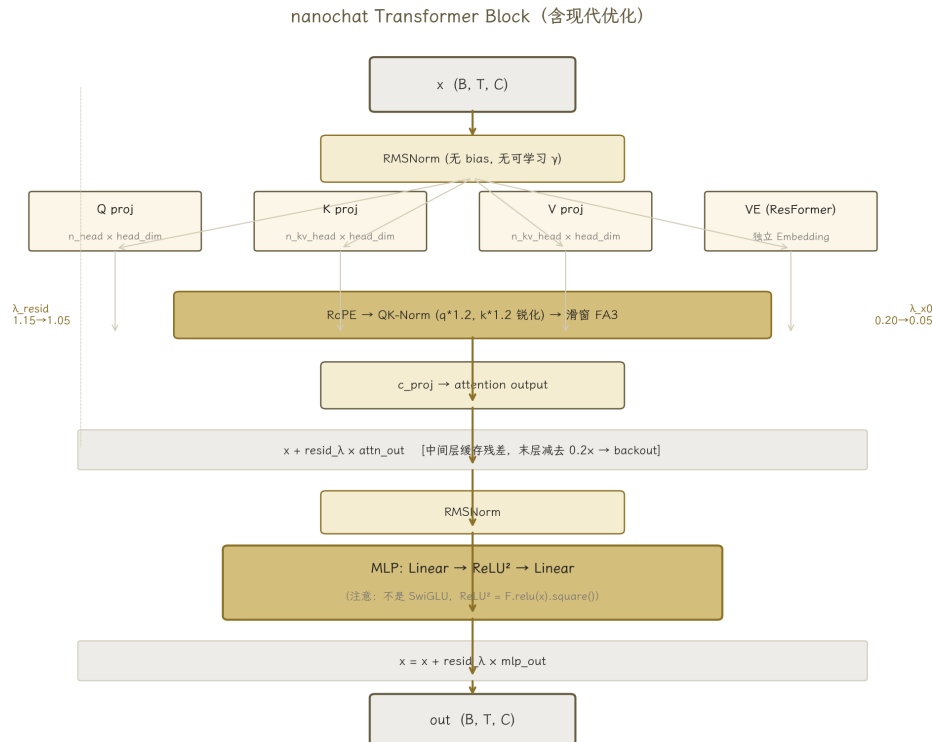


图 4-3 nanochat Transformer Block 完整架构 (含现代优化标注)

对应到代码, gpt.py 的 Block.forward 大约 30 行, 是整个项目最密集的代码段。下面摘录关键部分:

```
class Block(nn.Module):
    def __init__(self, cfg, layer_idx):
        super().__init__()
        self.ln_1 = RMSNorm(cfg.n_embd) # 无可学习参数
        self.attn = CausalSelfAttention(cfg, layer_idx)
        self.ln_2 = RMSNorm(cfg.n_embd)
        self.mlp = MLP(cfg)
        self.resid_lambda = cfg.resid_lambdas[layer_idx]
        self.x0_lambda = cfg.x0_lambdas[layer_idx]
        # smear 与 backout 的门控参数
        self.smear_gate = nn.Parameter(torch.zeros(24))
        if has_ve(layer_idx, cfg.n_layer):
            self.ve_gate = nn.Parameter(torch.zeros(12))

    def forward(self, x, ve, cos_sin, x0, x0_cache, window_size):
        # smear: 混入上一 token 的 embedding
        if x0 is not None:
            shifted = F.pad(x0, (0, 0, 1, 0))[:-1]
            x = x + torch.sigmoid(self.smear_gate) * shifted[..., :24].mean(-1, keepdim=True)

        # Attention 子层 + 残差
        x_norm = self.ln_1(x)
        attn_out = self.attn(x_norm, ve, cos_sin, window_size, None)
        x = x + self.resid_lambda * attn_out

        # backout: 中间层缓存, 末层减去
        if self.layer_idx == self.n_layer // 2:
            x0_cache = x.clone()
```

```

elif self.layer_idx == self.n_layer - 1 and x0_cache is not None:
    x = x - 0.2 * x0_cache

# MLP 子层 + 残差
x_norm = self.ln_2(x)
mlp_out = self.mlp(x_norm)
x = x + self.resid_lambda * mlp_out

# x0_lambda: 混入初始 embedding
if x0 is not None:
    x = x + self.x0_lambda * x0
return x, x0_cache

```

注意几个关键点：（1）RMSNorm 在 attention/mlp 之前（Pre-Norm），这是现代 LLM 的标准做法，让训练更稳定；（2）smear 与 x0_lambda 都是"额外通道"，把原始 embedding 注入深层，让深层也能"看到"输入；（3）backout 是"负通道"，在末层减去中间层残差，相当于"去除低级特征"；（4）所有这些路径都通过门控（sigmoid）或可学习标量（ λ ）调节强度，让训练过程能自适应地决定这些信号的贡献度。

4.10 滑窗注意力 SSSL 模式

当序列长度 T 较大时，全上下文 attention 的计算开销是 $O(T^2)$ ，且推理时 KV cache 占用 $O(T)$ 。滑窗注意力（Sliding Window Attention）限制每个 query 只能 attend 到最近的 W 个 key，把计算降为 $O(T \cdot W)$ ，KV cache 也降到 $O(W)$ 。然而纯滑窗会让长程依赖完全丢失。nanochat 采用 SSSL 模式：按层交替使用 S（滑窗）与 L（全上下文），最后一层强制全上下文，让模型既有局部的精细处理能力，又能在最后一层看到全局信息。

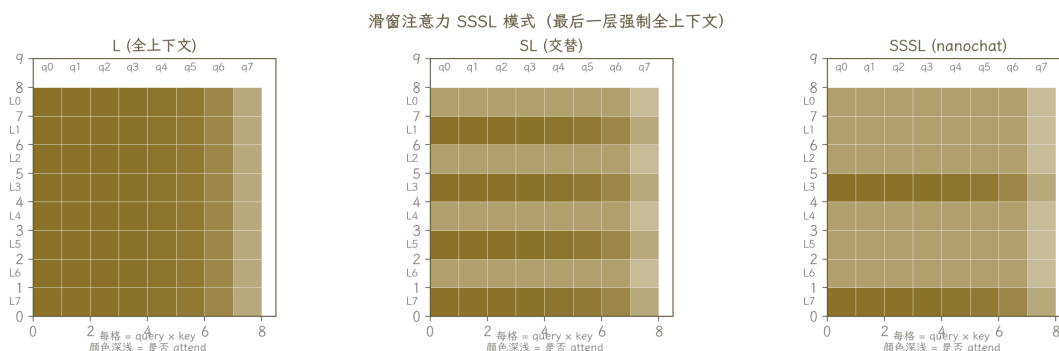


图 4-4 滑窗注意力 SSSL 模式：S=2 滑窗 + L=全上下文 交替

具体地，"SSSL" 字符串按层 tile，意味着第 0, 4, 8, ... 层用 S，第 3, 7, 11, ... 层用 L。滑窗大小 S 通常取 T 的 $1/4$ ，向上对齐到 FA3 的 tile 大小（128）。最后一层强制全上下文，让模型在"输出预测前"能看到完整的 prompt 信息。这种"细密滑窗 + 稀疏全上下文"的设计，是 Longformer、BigBird 等长序列模型的简化版本，在 nanochat 的实证中收益主要表现在 KV cache 减半而 CORE 几乎不降。

4.11 小结

本章覆盖了 nanochat Transformer 架构的全部关键技术点：RoPE 相对位置编码、QK-Norm + $1.2 \times$ 锐化、GQA 分组查询、ReLU² 激活、Value Embedding 残差注入、

smear/backout/per-layer λ 三个工程细节、logit softcap、滑窗 SSSL 模式。这些设计并非"原创发明"，而是 2024–2026 年现代 LLM 架构研究的实证结晶。nanochat 的价值不在于发明，而在于"把经过 ablation 验证的设计以最小化代码组合起来"。

理解这些设计后，读者可以带着"为什么是这样"的疑问开始第四章——预训练。预训练不仅需要好的架构，还需要好的优化器、好的学习率调度、好的 batch size、好的数据混合。下一章我们将看到 nanochat 如何用 Scaling Laws 自动推导所有这些超参，让"调参"从手艺变成数学。

第五章 预训练：Scaling Laws 与 Muon 优化器

5.1 从一段真实训练日志说起

在讲 Scaling Laws 之前，先看 nanochat d12 模型预训练时一段真实的训练日志。这段日志里所有数字都是根据 Scaling Laws 自动推导的——用户只指定了 `--depth=12` 一个参数。这是本章要回答的核心问题：为什么一个参数就能推出所有超参？

```
# 用户命令（只指定 depth）：
torchrun --nproc_per_node=8 -m scripts.base_train -- --depth=12

# nanochat 自动推导的超参（base_train.py 启动时打印）：
[Base Train Config]
depth = 12 ← 用户唯一指定
n_embd = 768 ← 自动推导：768 * (12/12)^0.5
n_head = 8 ← 自动推导：n_embd / 96
n_kv_head = 2 ← 自动推导：n_head / 4 (GQA)
n_params = 50M ← 由 n_embd, depth 计算
total_batch B = 2^19 ← Power Laws: B = B_ref * (D/D_ref)^0.383
target_param_data_ratio = 12 ← 默认值
target_tokens D = 600M ← B * target_param_data_ratio
matrix_lr = 0.020 ← Power Lines: eta = eta_ref * sqrt(B/B_ref)
embedding_lr = 0.300 ← AdamW 参数: 0.3 / sqrt(d/768)
weight_decay = 0.28 ← T_epoch 框架推导
warmup_steps = 40 ← 默认
total_steps = 2000 ← 由 D 和 seq_len 推导
warmdown_ratio = 0.65 ← 默认

# 训练循环中的日志（每 50 步打印一次）：
step 50 loss=4.32 lr=0.005 tok/s=18000 MFU=42%
step 100 loss=3.85 lr=0.012 tok/s=18200 MFU=45%
step 500 loss=3.21 lr=0.020 tok/s=18100 MFU=48%
step 1000 loss=2.95 lr=0.020 tok/s=18000 MFU=48% ← stable 阶段
step 1500 loss=2.78 lr=0.014 tok/s=18100 MFU=48% ← warmdown 开始
step 2000 loss=2.65 lr=0.001 tok/s=18000 MFU=48% ← 训练完成
总耗时：5 分钟（8xA100） 成本：~$0.50
```

看到没有？用户只指定了 `depth=12`，nanochat 自动推出了 15 个超参，最后训练 5 分钟、0.5 美元，得到一个 50M 参数的模型，loss 从 4.32 降到 2.65。这种"单旋钮"设计是过去 6 年 Scaling Laws 研究的结晶。本章逐节拆解每个超参是怎么自动推导的，最后回到这段日志看每个数字的来源。

5.2 Scaling Laws 的演进：从 Kaplan 到 Power Lines

回到上一节那段日志——为什么 $\text{depth}=12$ 推出 $B=2^{19}$ 、 $\text{lr}=0.02$ 、 $\text{wd}=0.28$ ？答案在 Scaling Laws。Scaling Laws 是 LLM 工程的"宇宙常数"——它告诉我们"用多少参数、多少数据、多少算力训练，能获得多少性能"。Kaplan et al. 2020 的原始论文建立了"loss 随算力 $C = 6ND$ 幂律下降"的基础规律，但 Kaplan 给出的最优数据/参数比约为 $D/N \approx 1.5$ ，这意味着"大模型 + 少数据"是 compute-optimal 的。Chinchilla (Hoffmann et al. 2022) 通过更细致的实验推翻了这一结论，给出 $D/N \approx 20$ 的最优比，意味着"小模型 + 多数据"才是 compute-optimal。Chinchilla 的发现直接推动了 Llama 系列采用更小模型 + 更多 token 的策略。

然而 Chinchilla 假设 batch size 是常数。Power Lines (Bjorck et al. 2025) 发现：batch size B 本身也随数据量 D 有最优扩展关系 $B \propto D^{0.383}$ 。换言之，训练数据越多，最优 batch 也应该越大。这一发现让"自动调参"成为可能：给定模型层数 depth ，可以一路推导出最优的 batch、lr、weight decay、训练步数。nanochat 的 `base_train.py` 正是这一思路的完整实现。

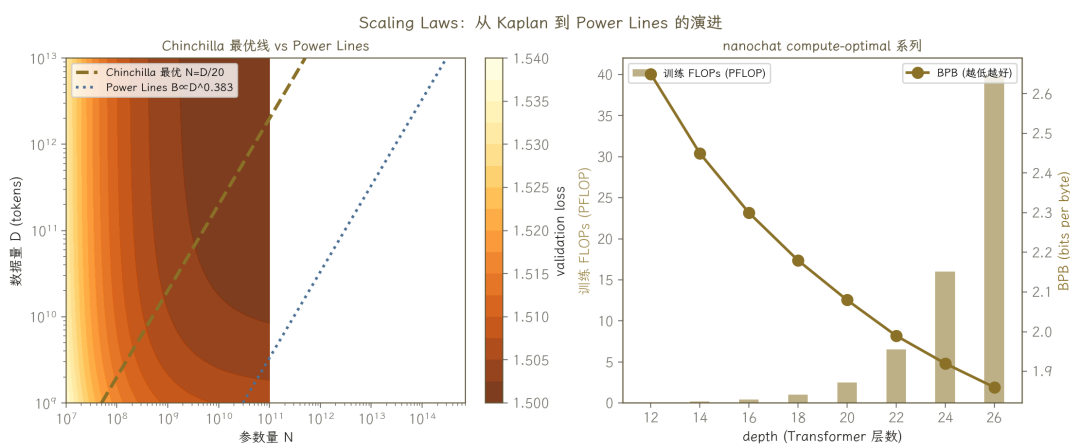


图 5-1 Scaling Laws 演进：左图 Kaplan/Chinchilla 等高线 + 最优线，右图 nanochat compute-optimal 系列

5.2 nanochat 的 scaling laws 自动推导

`base_train.py` 的核心设计是"单旋钮 `--depth`"——用户只需指定 Transformer 层数，其余超参全部按 scaling laws 自动推导。这套推导基于一个"参考点"（d12 模型在特定 FLOPs 下训练的实证最优超参），然后按 Power Lines 与一系列经验公式推到其他 depth 。下面是核心推导步骤。

5.2.1 模型维度推导

从 $\text{depth } d$ 出发， n_{embd} 、 n_{head} 、 $n_{\text{kv_head}}$ 等维度按经验公式推导：

```
# base_train.py 简化版
def build_model_meta(depth):
    # n_embd 随 depth 平方根增长 (Chinchilla 比例)
    n_embd = round_to_multiple(64, int(768 * (depth / 12)**0.5))
    n_head = max(6, n_embd // 96)
    n_kv_head = max(2, n_head // 4) # GQA 4:1 压缩
    head_dim = n_embd // n_head
    n_layer = depth
    return cfg
```

5.2.2 Batch size 推导 (Power Lines)

基于参考点 d12 的 D_{REF} 与 $B_{\text{REF}} = 2^{19}$ ，按 Power Lines 论文 $B \propto D^{0.383}$ ：

$$B = B_REF \cdot (D / D_REF)^{0.383}$$

其中 D 是目标训练 token 数, $D = N \times \text{target_param_data_ratio}$ (默认 12, 比 Chinchilla 的 20 更激进, 反映 nanochat 的"小模型多 epoch"策略)。 B 推导出来后, $\text{total_batch} = B \times \text{seq_len}$, 需要除以 device_batch_size 与 GPU 数得到 grad_accum 步数。

5.2.3 Learning rate 推导

lr 与 batch size 的关系: $\eta \propto \sqrt{(B / B_ref)}$ 。同时 AdamW 参数 (embedding/lm_head/scalar) 的 lr 按 $1/\sqrt{(d_model/768)}$ 调整——模型越宽, AdamW 参数的 lr 应越小 (避免 embedding 维度的方差过大):

$$\eta_matrix = \eta_ref \times \sqrt{(B / B_ref)}, \eta_adamw = \eta_ref \times \sqrt{(B / B_ref)} / \sqrt{(d_model / 768)}$$

5.2.4 Weight decay 推导 (T_epoch 框架)

weight decay 的最优值与训练 epoch 数有关。基于 T_epoch 论文的框架, nanochat 用 cosine schedule 把 wd 从初始值衰减到 0, 初始值由模型维度与训练步数共同决定。

超参	公式	d12 默认	d24 默认
n_embd	$768 \times (d/12)^{0.5}$	768	1080
n_head	$n_embd / 96$	8	12
n_kv_head	$n_head / 4$	2	4
total_batch B	$B_REF \times (D/D_REF)^{0.383}$	2^{19}	$\sim 2^{21}$
matrix_lr	$0.02 \times \sqrt{(B/B_ref)}$	0.02	0.024
embedding_lr	$0.3 / \sqrt{(d/768)}$	0.30	0.253
weight_decay	T_epoch 框架	0.28	0.20
target_param_data_ratio	12 (默认)	12	12

表 5-1 d12 与 d24 的自动推导超参对比

5.3 Muon 优化器：从 AdamW 到正交化更新

Adam 优化器 (Kingma & Ba 2014) 通过维护梯度的一阶动量 m 与二阶动量 v , 自适应地调整每个参数的学习率。AdamW (Loshchilov 2017) 把 weight decay 从梯度更新中解耦, 让 wd 更稳定地工作。Adam/AdamW 是 LLM 训练的事实标准, 但 2024 年 modded-nanoGPT 社区发现: 对"矩阵形状参数" (Linear 层的权重), Muon 优化器比 AdamW 收敛更快、最终 loss 更低。

5.3.1 Muon 的核心思想

Adam 的更新方向是"梯度除以二阶动量平方根"。Muon 的更新方向是"梯度的正交化"——通过 Newton-Schulz 迭代或 Polar Express 算法, 把任意矩阵梯度 G 转换为正交矩阵 O (即 $O \cdot O^T = I$), 然后把 O 作为更新方向。这一变换的关键性质: 正交矩阵保持向量范数不变 ($\|Ox\| = \|x\|$), 因此更新方向"在各维度上等强度", 避免了 Adam 在不同维度上"自适应过强"导致的方向漂移。

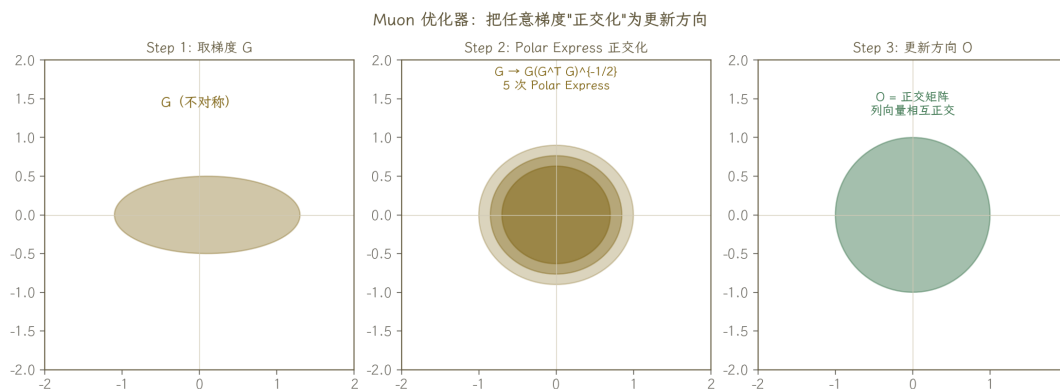


图 5-2 Muon 优化器的正交化过程：从梯度 G 到正交矩阵 O

5.3.2 数学推导：Polar Express 迭代

Polar Express 是 Newton-Schulz 的改进版，用 5 次迭代就能高精度逼近正交化。迭代公式为：

$$O_0 = G, O_{k+1} = O_k \cdot (I + (I - O_k^T O_k) / 2)$$

nanochat 使用预计算系数 `polar_express_coeffs` 让 5 次迭代达到 Newton-Schulz 10 次的精度。此外，nanochat 在 Muon 上叠加了三个 2025 年的改进：（1）MuonEq 行均衡：按行范数 rescale 谱条件；（2）Muon+ 重归一化：把更新矩阵的 Frobenius 范数 snap 到 $\sqrt{\min(m,n)}$ ；（3）NorMuon 方差归约：factored second moment，让方差与 Adam 相当。这三个改进让 Muon 在数值上更稳定，最终在 nanochat 实测中比“裸 Muon”快约 10%。

5.3.3 MuonAdamW 混合优化器

值得注意的是，Muon 只适用于“矩阵形状参数”（Linear 层的 weight），不适用于“向量形状参数”（embedding、lm_head、LayerNorm 等）。nanochat 的 MuonAdamW 类把两类参数分流：矩阵参数走 Muon 更新，向量参数走 AdamW 更新。这种“混合优化器”是现代 LLM 工程的关键设计。

```
# optim.py 的 MuonAdamW 类（简化）
class MuonAdamW:
    def __init__(self, params, lr, muon_momentum, adamw_betas, ...):
        # 按 param_groups 中 'kind' 分流
        self.muon_groups = [g for g in param_groups if g['kind'] == 'muon']
        self.adamw_groups = [g for g in param_groups if g['kind'] == 'adamw']

    @torch.compile
    def _muon_fused_kernel(self, p, grad, state, lr, wd, momentum):
        # 1. Nesterov momentum
        buf = state['momentum_buffer']
        buf.mul_(momentum).add_(grad)
        grad = grad + momentum * buf
        # 2. bf16 -> fp32 转换
        g_fp32 = grad.to(torch.float32)
        # 3. MuonEq 行均衡（按行范数 rescale）
        row_norm = g_fp32.norm(dim=-1, keepdim=True)
        g_fp32 = g_fp32 / row_norm * sqrt(g_fp32.shape[-1])
        # 4. Polar Express 5 次迭代
        o = polar_express(g_fp32, n_iters=5)
        # 5. Muon+ Frobenius snap
```

```
o = o * sqrt(min(o.shape)) / o.norm()
# 6. NorMuon 方差归约 + cautious weight decay
# ...
# 7. 参数更新
p.add_(o.view_as(p), alpha=-1r)
```

5.4 ZeRO-2 风格的分布式通信

多 GPU 训练的传统做法是 DDP (DistributedDataParallel) ——每个 GPU 持有完整参数副本，反向传播后用 all-reduce 同步梯度。问题是：模型越大，每个 GPU 上的"参数+优化器状态+梯度"显存占用越大，80GB 显存的 H100 训 7B 模型时几乎要爆。ZeRO-2 (Zero Redundancy Optimizer Stage 2) 的核心想法：把"优化器状态"按 rank 切分——每个 rank 只负责更新一部分参数，反向传播后用 reduce-scatter 把梯度按列分给对应 rank，本地更新完成后再用 all-gather 把完整参数广播回所有 rank。

ZeRO-2 风格 3 阶段异步通信 (nanochat 实现)

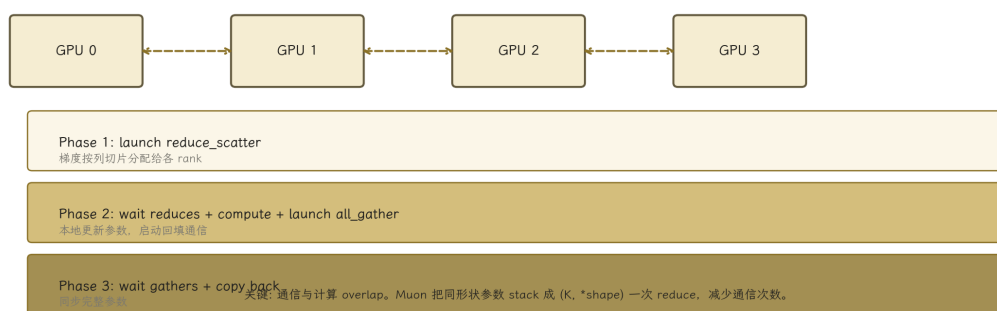


图 5-3 ZeRO-2 风格 3 阶段异步通信

nanochat 不用 DDP，自己实现 ZeRO-2 风格的通信，且做了关键优化——3 阶段异步：（1）启动 reduce_scatter 后不等待，立即开始下一步计算；（2）等待 reduce 完成、本地更新、同时启动 all_gather；（3）等待 all_gather 完成、复制参数回主 buffer。这种异步让通信与计算 overlap，在 8×H100 上实测能把 GPU 利用率从 50% 提到 80%。

另一个 nanochat 的优化是"按形状 stack"——把多个同形状的矩阵参数 stack 成 (K, *shape) 一次 reduce，减少通信次数。例如多个 Linear 层的权重 shape 都是 (4096, 1080)，stack 成 (32, 4096, 1080) 一次通信，比 32 次单独 reduce 快 5-10 倍。

5.5 WSD 学习率调度

学习率调度 (LR schedule) 决定 lr 在训练过程中如何变化。最经典的是 cosine schedule：从峰值 lr 按 cosine 曲线衰减到接近 0。WSD (Warmup-Stable-Decay) 是 2024 年提出的替代方案：warmup 阶段线性增长，stable 阶段保持常数，decay 阶段线性下降。

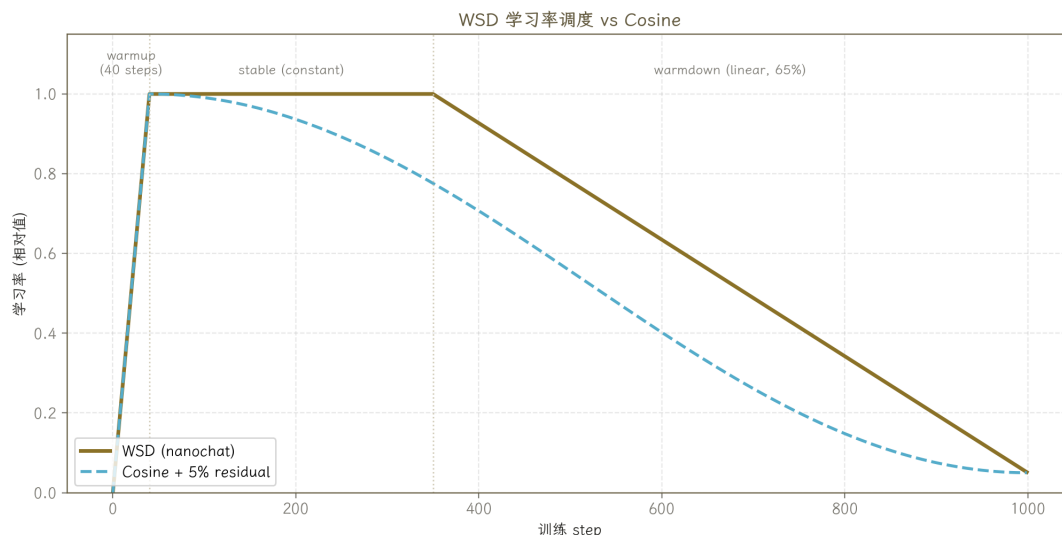


图 5-4 WSD 学习率调度 vs Cosine

WSD 的优势在于"stable 阶段可灵活延长"——如果训练中发现 loss 仍在下降，可以延长 stable 段而无需重新设计 schedule。cosine schedule 则必须预先知道总步数，中途延长会破坏曲线形状。nanochat 的默认配置是 warmup=40 steps、stable=35% 总步数、warmdown=65% 线性衰减到 5% 峰值 lr。这种"长 warmdown"让模型在最后阶段缓慢收敛，避免过早过拟合。

```
def get_lr_multiplier(step, total_steps, warmup, warmdown_ratio, final_lr_frac):
    if step < warmup:
        return step / warmup # 线性 warmup
    stable_end = int(total_steps * (1 - warmdown_ratio))
    if step < stable_end:
        return 1.0 # stable
    # 线性 warmdown 到 final_lr_frac
    progress = (step - stable_end) / (total_steps - stable_end)
    return 1.0 - (1.0 - final_lr_frac) * progress
```

5.6 混合精度训练：BF16 与 FP8

深度学习训练的精度从 FP32 → FP16 → BF16 → FP8 逐步降低，每个精度下移都能带来 2× 的吞吐提升。FP16 用 5 位指数 + 10 位尾数，动态范围小（约 6×10^4 ），容易溢出，需要 GradScaler。BF16 用 8 位指数 + 7 位尾数，与 FP32 同动态范围但精度低，无需 GradScaler，是 LLM 训练的事实标准。FP8 进一步分两种格式：e4m3（4 位指数 + 3 位尾数）适合前向，e5m2（5 位指数 + 2 位尾数）适合反向梯度。

nanochat 的混合精度策略不使用 autocast（PyTorch 的自动混合精度上下文管理器），而是显式管理：COMPUTE_DTYPE 全局变量 + 自定义 Linear 在 forward 时 cast 权重到 BF16 + embedding 直接存 BF16。FP8 训练通过 fp8.py 的 convert_to_float8_training 实现，基于 torch._scaled_mm 直接调 cuBLAS FP8 GEMM kernel，比 BF16 快 1.5–2×。

```
# fp8.py 核心思路（简化）
def convert_to_float8_training(model):
    for name, mod in model.named_modules():
        if isinstance(mod, nn.Linear) and mod.weight.dim() == 2:
            # 用 FP8E4M3Linear 替换标准 Linear
            new_mod = FP8E4M3Linear.from_float(mod)
```



```

        replace_module(model, name, new_mod)

class FP8E4M3Linear(nn.Module):
    def forward(self, x):
        # tensorwise dynamic scaling: 找 |x| 最大值, 缩放到 FP8 范围
        x_scale = x.abs().max() / 448.0 # 448 = e4m3 max
        w_scale = self.weight.abs().max() / 448.0
        x_fp8 = (x / x_scale).to(torch.float8_e4m3fn)
        w_fp8 = (self.weight / w_scale).to(torch.float8_e4m3fn)
        # 直接调 cuBLAS FP8 GEMM
        out = torch._scaled_mm(
            x_fp8, w_fp8.T,
            scale_a=x_scale, scale_b=w_scale,
            out_dtype=torch.bfloat16,
        )
        return out

```

5.7 base_train.py 整体流程

把前 6 节的设计组合起来, base_train.py 的主流程如下: (1) 解析 CLI 参数 (--depth、--target-flops、--fp8 等); (2) compute_init 初始化 DDP + 精度 + cuDNN; (3) build_model_meta 在 meta device 上构建模型 (只占 shape), 再 to_empty + init_weights; (4) 可选 FP8: convert_to_float8_training; (5) torch.compile(model, dynamic=False) 加速; (6) 按 scaling laws 自动推导所有超参; (7) model.setup_optimizer 创建 MuonAdamW; (8) 训练循环: grad_accum、scaler.scale(loss).backward()、CORE 评估、采样、checkpoint。整个脚本约 604 行, 是 nanochat 中最长的脚本, 但结构清晰、注释充分, 逐段读懂是本章的学习目标。

5.8 小结

本章覆盖了 nanochat 预训练的所有关键工程: Scaling Laws 自动调参、Muon 优化器、ZeRO-2 分布式通信、WSD 学习率调度、混合精度训练 (BF16 + FP8)。一个关键观察是: nanochat 的"单旋钮 --depth"设计让 LLM 训练从"调参艺术"变成"参数化数学"——给定深度, 所有超参自动推导, 避免人工试错。这一设计在工程上极大降低了"训练小模型"的门槛, 让普通研究者也能在 1×H100 上完成 GPT-2 复现。

理解预训练后, 下一步是 SFT (监督微调) ——让预训练得到的"会续写文本的模型"变成"会按指令回答的对话模型"。第五章将聚焦数据加载与 SFT 的工程实现。

第六章 数据加载与 SFT 监督微调

6.1 从三个文档到一行 2048 token: 贯穿示例

在讲算法之前, 先看一个具体问题。假设你的预训练数据里有三个文档: A (1800 tokens)、B (1500 tokens)、C (900 tokens)。你想把它们喂给 batch_size=1, seq_len=2048 的训练循环。最朴素的做法是 padding——但这浪费显存。这一节我们看 nanochat 怎么用 BOS-aligned bestfit 把这三个文档"装"进两行 2048 token, 且 100% 利用率、无 padding 浪费。

输入：3 个文档

文档 A = "Hinton 在 2006 年提出深度信念网络..." (1800 tokens)

文档 B = "Transformer 由 Vaswani 等人在 2017 年提出..." (1500 tokens)

文档 C = "RoPE 是 Su 等人在 2021 年提出的..." (900 tokens)

目标：装入两行 batch，每行 seq_len=2048

朴素 padding (浪费算力)：

行 1: [BOS][A (1800)][pad(247)] ← 浪费 247 个 token 位置

行 2: [BOS][B (1500)][pad(547)] ← 浪费 547 个 token 位置

C 没放进来；总浪费 794 token，浪费率 19%

BOS-aligned bestfit (100% 利用率)：

行 1: [BOS][A (1800)][B 前 247 截断] ← 装满，B 剩 1253 tokens

行 2: [BOS][B 后续 1253][C (900)] ← 装满，C 完整保留

关键观察：每行都以 BOS 开头，每个 token 都能 attend 回 BOS 看完整上下文；100% 利用率，无 padding；少量 token 被裁剪 (B 的前 247)。

这个例子展示了 BOS-aligned bestfit 装箱算法的核心思想：每行以 BOS (begin-of-sequence) 开始，用 best-fit 装箱算法挑选能完整放入剩余空间的最大文档；装不下时裁剪最短文档补满。关键性质是"100% 利用率"——没有 padding 浪费，T=2048 时约 35% token 被裁剪损失（来自裁剪掉的尾部）。设计动机是"让每个 token 都能 attend 回 BOS，看到完整文档上下文"，减少"令人困惑"的裁切训练样本。

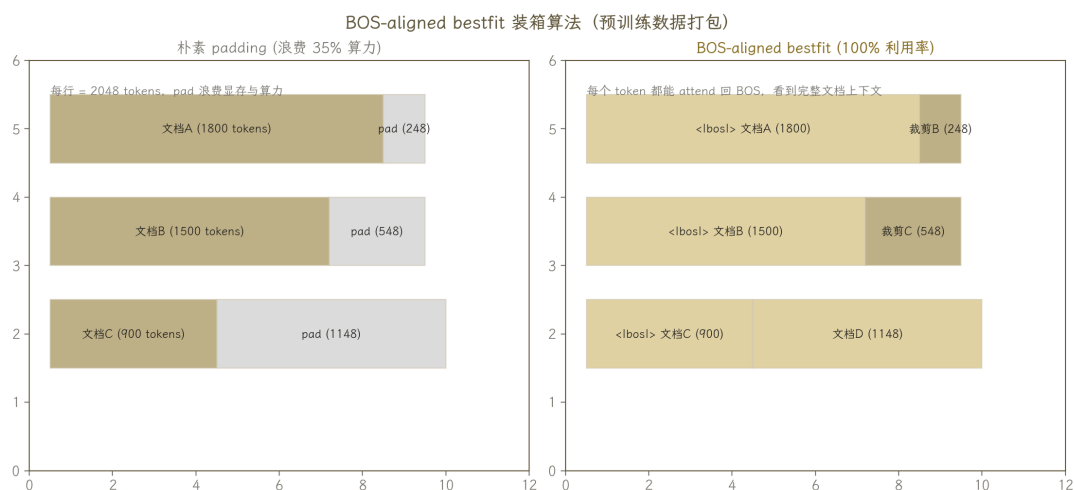


图 6-1 BOS-aligned bestfit 装箱：朴素 padding vs bestfit 对比

6.1.1 算法伪代码

```
def tokenizing_distributed_data_loader_with_state_bos_bestfit(
    rank, world_size, dataset, tokenizer, seq_len, batch_size, state
):
    cpu_buf = torch.empty(batch_size, seq_len, dtype=torch.uint16,
                           pin_memory=True)
    gpu_buf = torch.empty(batch_size, seq_len, dtype=torch.uint16,
                           device='cuda')

    while True:
        # 1. DDP 按 rank 切分 row_group
        # 2. 多线程 tokenize 文档
        # 3. 对每行 (batch_size 个) 做 bestfit 装箱
        for row_idx in range(batch_size):
```

```

bos = tokenizer.special_ids['<|bos|>']
cpu_buf[row_idx, 0] = bos # 每行以 BOS 开头
cursor = 1
while cursor < seq_len:
    # 找最大可放入 seq_len - cursor 的文档
    doc = pick_largest_fitting_doc()
    if doc is None:
        # 装不下任何完整文档，裁剪最短文档补满
        doc = pick_shortest_doc_truncated_to(seq_len - cursor)
    cpu_buf[row_idx, cursor:cursor+len(doc)] = doc
    cursor += len(doc)
# 4. HtoD 传输
gpu_buf.copy_(cpu_buf, non_blocking=True)
yield gpu_buf

```

6.1.2 DDP 分片与断点续训

多 GPU 训练时，数据需要按 rank 切分。nanochat 按 parquet row_group 切分——每个 rank 读不同的 row_group，避免重复读取相同数据。state_dict = {pq_idx, rg_idx, epoch} 记录当前进度，中断后能从精确位置恢复。这一设计在大规模训练（170 分片 × 5 epoch）时尤为重要，一次 H100 节点 1.5 小时训练可能在任何时候因硬件故障中断，断点续训能力是工程必需。

6.2 SFT 数据打包：对话不能用裁剪

SFT 的数据是“多轮对话”而非“独立文档”，第二章我们已经看到 render_conversation 会把对话渲染成(token_ids, loss_mask) 二元组。这些对话序列长度依然参差不齐，但与预训练不同——对话不能被裁剪。原因有二：（1）对话中每条 assistant 回复都是完整语义单元，裁剪会让模型学到“半句话回复”，破坏对话结构；（2）loss mask 依赖完整的 <|assistant_start|>...<|assistant_end|> 边界，裁剪可能破坏这些边界，导致 mask 失效。

因此 SFT 的 dataloader 用 BOS pad 而非裁剪——对话装不下时用 BOS token 填充剩余空间，让 cross_entropy ignore_index 自动跳过 pad 部分。这一设计在 chat_sft.py 的 sft_data_generator_bos_bestfit 函数中实现：

```

def sft_data_generator_bos_bestfit(...):
    while True:
        for row_idx in range(batch_size):
            cursor = 0
            while cursor < seq_len:
                conv_ids, conv_mask = render_conversation(next_conversation())
                remaining = seq_len - cursor
                if len(conv_ids) <= remaining:
                    # 完整放入
                    cpu_ids[row_idx, cursor:cursor+len(conv_ids)] = conv_ids
                    cpu_mask[row_idx, cursor:cursor+len(conv_mask)] = conv_mask
                    cursor += len(conv_ids)
                else:
                    # 装不下，用 BOS pad 补满剩余空间
                    bos = tokenizer.special_ids['<|bos|>']
                    cpu_ids[row_idx, cursor:] = bos
                    cpu_mask[row_idx, cursor:] = 0 # pad 部分 mask=0
                    cursor = seq_len
            yield cpu_ids, cpu_mask

```

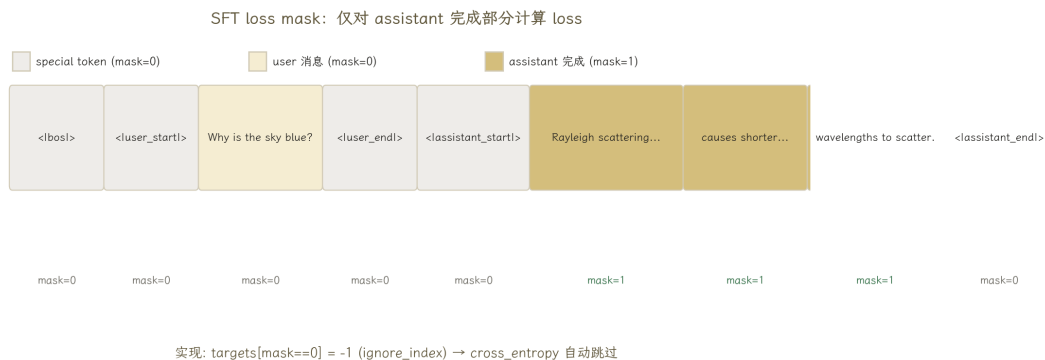


图 6-2 SFT loss mask: 仅对 assistant 完成部分计算 loss

6.3 数据混合：SmolTalk + MMLU + GSM8K

SFT 数据的质量与多样性直接决定对话模型的最终表现。nanochat 的 SFT 数据混合包含三类来源：（1）SmolTalk（HuggingFaceTB/smol-smoltalk）：约 460K 通用对话样本，覆盖闲聊、信息问答、指令跟随等通用场景；（2）MMLU auxiliary_train × 3：MMLU 的辅助训练集（cais/mmlu），主要是多选题，重复 3 次增强学科能力；（3）GSM8K main × 4：小学数学题（openai/gsm8k），重复 4 次增强数学推理能力。验证集 SmolTalk(test) + MMLU(test, 5200) + GSM8K(test, 420) 比例与训练集对齐。

数据集	样本数	重复次数	在 ChatCORE 中的作用
SmolTalk (train)	~460K	1×	通用对话能力
MMLU (auxiliary_train)	~99K	3×	学科知识 + 多选题格式
GSM8K (main)	~7.5K	4×	数学推理 + 工具调用
SmolTalk (test)	~小量	1×	验证通用能力
MMLU (test)	5200	1×	验证学科能力
GSM8K (test)	420	1×	验证数学能力

表 6-1 SFT 数据混合配置

一个关键观察是“重复次数 ≠ 简单复制”——同一个 GSM8K 题目在 4 个 epoch 中会出现 4 次，但每次的 loss mask 与上下文可能略有差异（取决于 tokenizer 状态、batch 组合等）。这种“温和过拟合”对 GSM8K 这种“题量小但需要形式推理”的任务尤其有效——模型在多次见到同类问题后能学到稳定的解题模式。MMLU 的 3 次重复同理。SmolTalk 数据量大、多样性高，不需要重复。

6.4 SFT 训练流程与超参继承

chat_sft.py 的训练流程与 base_train.py 类似，但有几个关键差异：（1）从 base checkpoint 加载预训练模型：load_model("base", ...)；（2）继承预训练超参但允许 override（如 SFT 通常用更小 lr、更短训练步数）；（3）可选“warm-start 优化器状态”——保留 momentum 但重置 lr，让 SFT 阶段从一个“已收敛”的优化器状态起步，收敛更快。这一设计在工程上很有意义——从预训练到 SFT 的过渡不需要“完全冷启动优化器”。

```

# chat_sft.py 关键流程
model = load_model('base', device='cuda', dtype=COMPUTE_DTYPE)

# 继承超参 (可 override)
lr_multiplier = base_lr_mult * 0.5 # SFT 用预训练 lr 的一半
total_steps = 5000 # SFT 通常只需 1 个 epoch

# 可选 warm-start 优化器状态
if warm_start_optimizer:
    opt_state = load_opt_state('base')
    opt = MuonAdamW(model.parameters(), ...)
    opt.load_state_dict(opt_state)
    # 重置 lr 但保留 momentum
    for g in opt.param_groups:
        g['lr'] = base_lr * 0.5

# 训练循环 (与 base_train 类似但用 SFT 数据)
for step, (inputs, mask) in enumerate(sft_data_loader):
    # loss mask: 把 mask=0 的位置 targets 设为 -1 (ignore_index)
    targets = inputs.clone()
    targets[mask == 0] = -1
    logits = model(inputs)
    loss = F.cross_entropy(
        logits.view(-1, vocab_size),
        targets.view(-1),
        ignore_index=-1,
    )
    scaler.scale(loss).backward()
    if (step + 1) % grad_accum == 0:
        scaler.step(opt)
        scaler.update()
        opt.zero_grad()

```

6.5 SFT 评估指标：BPB + ChatCORE

SFT 阶段的评估指标与预训练不同——除了 BPB（验证对话能力是否提升），还需要 ChatCORE（5 任务 centered mean：ARC-Easy、ARC-Challenge、MMLU、GSM8K、HumanEval）来评估“对话模型”的实用能力。这一指标的详细设计在第八章讲解，这里只提一句：ChatCORE 的 5 个任务分别考察不同的能力维度，对每个任务单独评估 pass@k 或准确率，然后取“centered mean”——减去随机基线后再平均，避免任务难度差异影响总分。

6.6 小结

本章覆盖了 SFT 的全部工程细节：BOS-aligned bestfit 装箱、SFT 数据打包（不裁剪而用 BOS pad）、数据混合（SmolTalk + MMLU×3 + GSM8K×4）、训练流程与超参继承、loss mask 的实现。一个核心洞察是：“SFT 是预训练模型与对话能力之间的桥梁，桥梁的设计决定了模型的“实用价值”——好的数据混合 + 好的 loss mask，能让一个小模型在 ChatCORE 上达到接近 GPT-2 的水平。

理解 SFT 后，下一步是强化学习——让模型通过“试错 + 奖励”进一步提升特定能力。第六章将从 RLHF/PPO 的理论出发，逐步推导到 nanochat 的简化 GRPO 实现。

第七章 强化学习：从 PPO 到 GRPO 的简化路径

7.1 一道 GSM8K 题如何走完整个 RL 循环

在讲 RLHF 理论之前，先看一道具体 GSM8K 数学题如何走完整个 nanochat RL 训练循环。这是本章的贯穿示例——后面讲 PPO/GRPO 公式时都会回到这个例子，看每个公式的具体数字在它上面如何计算。

```
# 输入：一道 GSM8K 题目
题目: "Janet 有 12 个苹果，吃了 3 个，又买了 5 个，现在有几个？"
ground_truth: 14

# 第 1 步：把题目渲染成对话格式（接第二章的 render_for_completion）
prompt_tokens = [<|bos|>, <|user_start|>, "Janet 有 12 个苹果...", <|user_end|>,
                 <|assistant_start|>]

# 第 2 步：采样 16 个回答（temperature=1.0, top_k=50）
rollouts[0] = "Janet 有 12 个，吃了 3 个剩 9 个，再买 5 个，14 个。#### 14" ✓ 正确
rollouts[1] = "12 - 3 + 5 = 14。#### 14" ✓ 正确
rollouts[2] = "12 - 3 = 9, 9 + 5 = 14。#### 13" × 算错
rollouts[3] = "...答案是 14。#### 14" ✓ 正确
rollouts[4] = "我不知道。#### 0" × 错
... 共 16 个 ...

# 第 3 步：解析 #### 后的数字，计算 reward（0 或 1）
rewards = [1, 1, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 0, 1, 0]
reward_sum = 9, reward_mean  $\mu = 9/16 = 0.5625$ 

# 第 4 步：计算每个 rollout 的 advantage（只减均值，不减标准差）
advantages = [r - 0.5625 for r in rewards]
             = [0.4375, 0.4375, -0.5625, 0.4375, -0.5625, ...]
# 正确的 rollout 得正 advantage（被强化），错误的得负 advantage（被抑制）

# 第 5 步：计算 policy gradient loss（token 级归一化）
loss = -sum(logp(rollout[i]) * advantage[i]) / num_valid_tokens

# 第 6 步：反向传播更新模型参数
loss.backward()
opt.step(); opt.zero_grad()
```

整个 RL 循环就是上面 6 步。nanochat 的简化之处在于：不维护 reference model（无 KL penalty）、不做 PPO clip、advantage 只减均值不减标准差、reward 来自客观答案（不需要训练 reward model）。本章逐节拆解每一步，从 PPO 的完整理论开始，逐步简化到 nanochat 的 200 行实现。

7.2 RLHF 的谱系：从 InstructGPT 到 GRPO

强化学习从人类反馈（Reinforcement Learning from Human Feedback, RLHF）是 InstructGPT（Ouyang et al. 2022）提出的让 LLM 对齐人类偏好的方法。原始 RLHF 包含三个阶段：（1）SFT：用监督数据训练基础对话模型；（2）Reward Model（RM）训练：用人类标注的偏好对训练一个“打分模型”；（3）PPO 强化学习：用 RM 的分数作为奖励，PPO 算法优化对话模型。这一框架被 OpenAI、Anthropic 广泛使用，但工程复杂度极高——需要同时维护 policy、

reference、reward、critic 四个模型，显存与计算成本是 SFT 的 4 倍以上。

2024 年 DeepSeek 提出的 GRPO (Group Relative Policy Optimization) 做了关键简化：删除 critic (不需要 value function)，用"组内相对优势"替代。同一题采样 G 个回复，计算每个回复的 reward，组内 reward 减均值再除以标准差得到 advantage。这一简化让 RL 的工程复杂度从"4 个模型"降到"2 个模型" (policy + reference)。2024 年 DPO (Direct Preference Optimization) 走得更远——直接用偏好数据训练 policy，完全删除 reward model 与 RL 训练循环，把 RL 变成"分类问题"。

nanochat 的 chat_rl.py 在 GRPO 基础上进一步简化，作者自嘲为"GRPO in quotes"——实际接近 REINFORCE (最朴素的 policy gradient)。下面先把 PPO/GRPO 的理论讲清楚，再展示 nanochat 的简化路径。

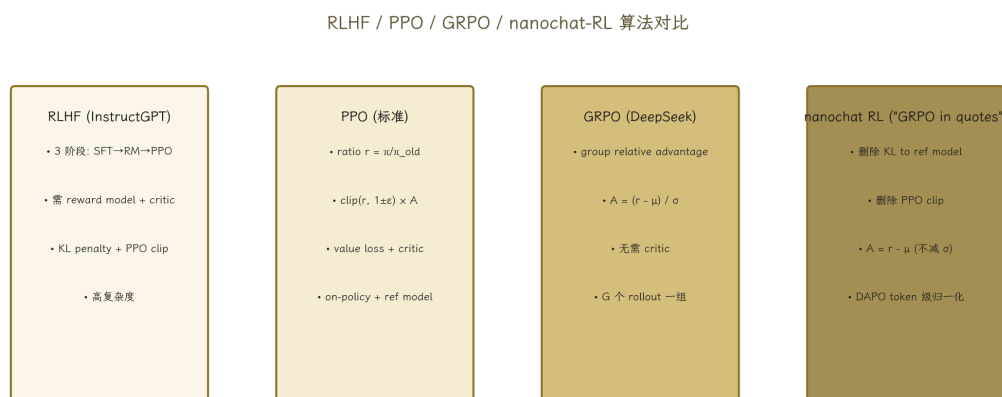


图 7-1 RLHF / PPO / GRPO / nanochat-RL 算法对比

7.3 PPO 的完整公式

PPO (Proximal Policy Optimization, Schulman et al. 2017) 的目标函数为：

$$L_{\text{PPO}} = E_t [\min(r_t A_t, \text{clip}(r_t, 1-\epsilon, 1+\epsilon) A_t)]$$

其中 $r_t = \pi_{\theta}(a_t|s_t) / \pi_{\theta_{old}}(a_t|s_t)$ 是新旧策略的概率比， A_t 是优势函数。clip 操作让 r_t 限制在 $[1-\epsilon, 1+\epsilon]$ 区间内，避免策略更新过大导致训练崩溃。 A_t 通常通过 Generalized Advantage Estimation (GAE) 计算，需要一个 critic 网络 $V(s)$ 估计状态价值。完整的 PPO 还包括 value loss (critic 的回归损失) 与 entropy bonus (鼓励探索)。

$$A_t = \delta_t + (\gamma\lambda) \delta_{t+1} + (\gamma\lambda)^2 \delta_{t+2} + \dots \quad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

应用到 LLM 场景：s 是 prompt (上下文)，a 是模型生成的 token，r 是 reward model 的分数。PPO 训练时维护四个模型：policy (被训练的 LLM)、reference (冻结的 SFT 模型，用于 KL penalty 防止退化)、reward model (打分)、critic (价值估计)。每个模型都需要一份完整参数与优化器状态，显存压力极大。

7.4 GRPO：删除 critic 的简化

GRPO 的核心洞察：在 LLM RL 中，每题采样 G 个回复，组内 reward 已经包含"相对优势"信息——不需要 critic 单独估计 $V(s)$ 。设第 i 个回复的 reward 为 r_i ，组内 reward 均值为 μ 、标准差为 σ ，则 advantage 为：

$$A_i = (r_i - \mu) / \sigma \text{ (GRPO 标准版)}$$

这一简化删除了 critic，policy gradient 只需 policy + reference 两个模型。KL penalty 仍保留——通过让 policy 不能偏离 reference 太远，防止 reward hacking。DeepSeek-V3 等大模型用 GRPO 取代 PPO，工程成本降低 50%+。

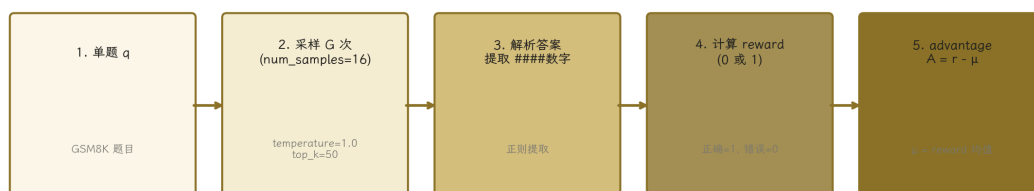
7.5 nanochat 的"GRPO in quotes"

nanochat 在 GRPO 基础上做了 4 个进一步简化：（1）删除 KL 到 reference model —— 不再维护 reference，只用 policy；（2）删除 PPO ratio + clip —— 不用 importance sampling，纯 on-policy；（3）advantage = $r - \mu$ ，不做 z-score 归一化（不减 σ ）；（4）DAPO 风格 token 级归一化 —— 不在 sequence 级归一化，而在 token 级。这些简化让 nanochat 的 RL 实现只有约 200 行，可以一次性读懂。

工程细节：为何 nanochat 不做 z-score

为什么不做 z-score？作者在 chat_rl.py 注释里解释：当 G 较小（如 16）且 reward 是 0/1 二元时，组内 reward 标准差可能为 0（全对或全错），此时 $(r - \mu) / \sigma$ 会出现 NaN。只减均值不减标准差避免了这一数值不稳定，代价是 advantage 的尺度在不同 reward 分布下不一致——但配合 token 级归一化（DAPO 风格）能稳定训练。

GRPO 采样→归组→计算 advantage 流程



$$\text{policy gradient: } \mathcal{L} = - \sum_t \log \pi(y_t | x) \cdot A \quad (\text{token 级归一化})$$

关键简化：nanochat 不做 z-score 归一化 ($A = r - \mu$)，避免 reward 方差为 0 时的数值不稳定

图 7-2 GRPO 采样→归组→计算 advantage 流程

7.6 chat_rl.py 代码精读

7.5.1 Rollout 采样

```
# chat_rl.py 主流程 (简化)
for step in range(total_steps):
    # 1. 从 GSM8K 采样一个 batch 的题目
```

```

prompts = sample_gsm8k_prompts(batch_size)

# 2. 对每题采 num_samples=16 个 rollout
inputs, outputs = engine.generate_batch(
    prompts,
    num_samples=16, # G=16
    temperature=1.0, # 高温采样增加多样性
    top_k=50,
    max_new_tokens=256,
)

# 3. 解析 #### 数字提取答案, 计算 reward
rewards = torch.zeros(batch_size * 16)
for i, out in enumerate(outputs):
    pred = extract_answer(out) # 正则提取 #### 后的数字
    true_answer = prompts[i // 16]['answer']
    rewards[i] = 1.0 if pred == true_answer else 0.0

# 4. 组内减均值得到 advantage
rewards_grouped = rewards.view(batch_size, 16)
advantages = rewards_grouped - rewards_grouped.mean(dim=1, keepdim=True)
advantages = advantages.view(-1)

# 5. 计算 policy gradient loss
# logp = -model(inputs, targets, loss_reduction='none')
# pg_obj = (logp * advantages).sum() / num_valid_tokens
# loss = -pg_obj
logits = model(inputs)
logp = -F.cross_entropy(
    logits.view(-1, vocab_size),
    outputs.view(-1),
    reduction='none',
).view(batch_size * 16, -1)
# token 级归一化 (DAPO 风格)
num_valid = (outputs != pad_token).sum()
pg_obj = (logp * advantages.unsqueeze(1)).sum() / num_valid
loss = -pg_obj

loss.backward()
opt.step()
opt.zero_grad()

```

7.5.2 reward 解析：GSM8K 答案格式

GSM8K 数据集每题答案格式为 "... #### <数字>"——前半部分是解题过程，最后是数字答案。nanochat 用正则 "####s*([d,\\.]+)" 提取末尾数字作为模型预测，与 ground truth 比对得到 0/1 reward。这一简单 reward 设计是 nanochat RL 的"关键简化"——不需要训练 reward model，直接用数学题的客观答案作为"自动评判员"，让 RL 训练循环可以完全自动化。

```

import re

def extract_answer(text):
    # GSM8K 格式: "... #### 42"
    match = re.search(r'####s*([d,\\.]+)', text)
    if not match:
        return None

```

```
ans = match.group(1).replace(',', ' ')
try:
    return float(ans)
except ValueError:
    return None

def reward_fn(model_output, ground_truth):
    pred = extract_answer(model_output)
    if pred is None:
        return 0.0
    return 1.0 if abs(pred - ground_truth) < 1e-4 else 0.0
```

7.7 为什么 RL 对数学推理特别有效

一个常见疑问：为什么 RL 在 GSM8K 上能显著提升，而在通用对话任务上效果有限？答案在于"reward 信号的清晰度"。GSM8K 有客观正确答案，reward 是 0/1 二元信号，组内 reward 方差足够大（同一题 16 个 rollout 中通常有对有错），advantage 信号清晰。而通用对话任务的 reward 来自 reward model（一个 LLM），其打分有噪声、有偏见、缺乏"绝对正确"的客观性，advantage 信号弱。

更深入地，RL 在数学推理上的有效性可以归因于"探索-利用平衡"的精确实现：temperature=1.0 的高温采样让模型"探索"多种解题路径，reward 信号让模型"利用"成功路径并避免失败路径。这种"试错学习"让模型能学到 SFT 阶段无法学到的"解题策略"——SFT 只能学"模仿人类解法"，RL 能学"自我探索解法"。在 GSM8K 上，nanochat 报告 RL 让 pass@1 从 0.20 提升到 0.35 左右，相对提升 75%。

7.8 小结

本章覆盖了 RLHF/PPO/GRPO/nanochat-RL 的完整谱系：PPO 完整公式 → GRPO 删除 critic → nanochat 进一步删除 KL/clip/z-score → DAPO token 级归一化。一个核心洞察是："RL 的工程复杂度可以无限降低，但前提是 reward 信号足够清晰"——GSM8K 这种"客观可验证"的任务适合 RL，通用对话任务则更适合 SFT + DPO。nanochat 的"GRPO in quotes"实现约 200 行，是教学版的"最小化 RL"，让读者能在不依赖 reward model 的情况下完整体验 RL 训练流程。

理解 RL 后，下一步是推理——让训练好的模型高效生成回答。第七章将聚焦 KV Cache、Flash Attention 3、工具调用状态机等推理工程细节。

第八章 推理引擎：KV Cache、Flash Attention 与工具调用

8.1 一段 prompt 从输入到流式生成：贯穿示例

在讲算法之前，先用一个具体例子展示 nanochat 推理引擎的完整数据流。假设用户问 "Why is the sky blue?"，期望模型流式生成 "Rayleigh scattering causes..."。这是本章的贯穿示例——后面讲 KV Cache、Flash Attention、工具状态机时都会回到这个例子。

```

# 用户输入: "Why is the sky blue?"

# Step 0 - token 化 (第二章讲过)
prompt = "Why is the sky blue?"
prompt_ids = [1234, 11, 543, 995, 7823] # 5 个 token

# Step 1 - 包装成对话格式 (接第二章的特殊 token)
input_ids = [<|bos|>, <|user_start|>, 1234, 11, 543, 995, 7823, <|user_end|>, <|assistant_start|>]
# shape: (1, 9) = (B=1, T_prompt=9)

# Step 2 - Prefill 阶段: 一次性算所有 prompt 的 K/V, 存入 KV cache
# 计算 attention: 9 个 query 与 9 个 key 的 9x9 attention matrix
# 复杂度  $O(T_{\text{prompt}}^2) = O(81)$ 
kv_cache.prefill(input_ids)
kv_cache.k.shape → (1, max_seq, n_kv_head, head_dim) = (1, 2048, 4, 96)
kv_cache.v.shape → (1, max_seq, n_kv_head, head_dim) = (1, 2048, 4, 96)
cache_seqlens → [9]

# Step 3 - Decode 阶段: 每步生成 1 个新 token
next_token_1 = "Ray" # token id 958
# 只算新 token 的 attention: 1 个 query 与 10 个 key (9 个 cache + 1 个新)
# 复杂度  $O(T) = O(10)$ 
# 把 token 958 的 K/V 也写入 cache

next_token_2 = "leigh" # token id 4523
next_token_3 = " scattering" # token id 9876
next_token_4 = " causes" # token id 8745
next_token_5 = " shorter" # token id 3382
...

# Step 4 - 流式输出
# chat_cli.py 边生成边 decode 然后打印
# 用户看到: "Rayleigh scattering causes shorter wavelengths..."

# Step 5 - 遇到 <|assistant_end|> 停止
next_token_N = <|assistant_end|>
# → 标记 done=True, 退出 decode 循环

```

上面展示了 prefill 与 decode 的完整流程。本章会逐节拆解每一步——KV Cache 如何存储与读取、Flash Attention 3 如何加速 attention 计算、工具调用状态机如何处理 Python 代码块。

8.2 推理的两阶段：Prefill 与 Decode

LLM 推理与训练有本质不同。训练时输入是固定长度的 batch，attention 在整个序列上并行计算，复杂度 $O(T^2)$ 。推理时则需要先生成 prompt 的 KV，再逐 token 生成回答——这一过程可以拆为两阶段：（1）prefill：处理整个 prompt，一次性计算所有 K/V，复杂度 $O(T^2)$ ，但是一次性的；（2）decode：每生成一个新 token，从 KV cache 读历史 K/V，只算新 token 的 attention，复杂度 $O(T)$ ，但需要执行 T 次（生成长度 T 个 token）。

KV cache 的作用：把 prefill 阶段算出的 K/V 存下来，避免 decode 阶段重复计算历史 K/V。显存占用为：

$$\text{KV cache 显存} = 2 (K+V) \times \text{seq_len} \times n_kv_head \times \text{head_dim} \times \text{dtype_bytes}$$

以 nanochat d24 模型为例：n_kv_head=4, head_dim=128, fp16, KV cache 显存约 $2 \times 2048 \times 4 \times 128 \times 2 = 4 \text{ MB per sample}$ 。GQA（4:1 压缩）让 KV cache 比纯 MHA 小 3 倍，是推理优化的关键。

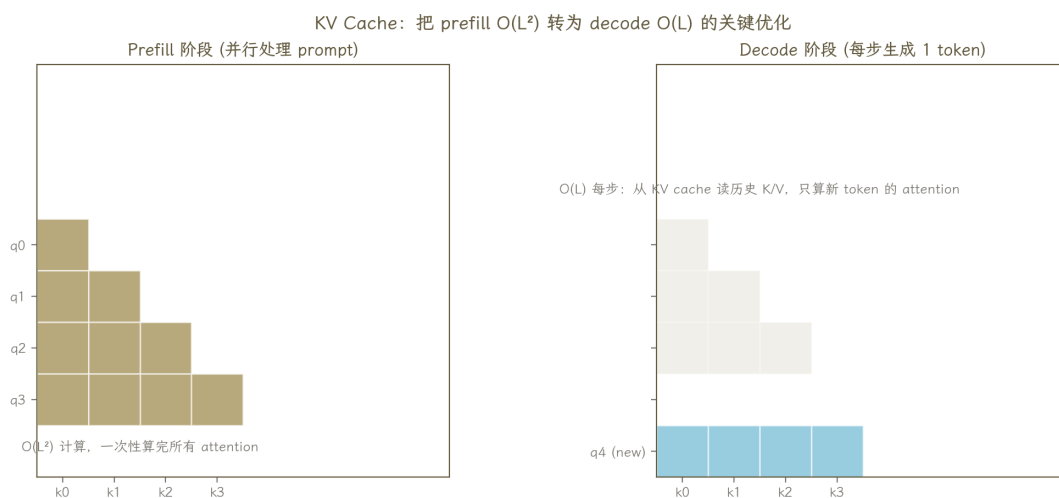


图 8-1 KV Cache: prefill $O(L^2) \rightarrow$ decode $O(L)$ 的两阶段优化

8.2 KVCache 的实现细节

nanochat 的 KVCache 类采用 FA3 风格的 (B, T, H, D) 布局，每个 batch 维护独立的 cache_seqLens。一个关键设计是 prefill() 方法可以把 batch=1 的 KV cache 复制到 num_samples 行——这一功能在 GRPO rollout 时非常有用：对同一 prompt 采 16 个不同回答，只需算一次 prefill，复制 KV 到 16 行后各自 decode，节省 $16 \times$ 的 prefill 计算。

```
class KVCache:
    def __init__(self, batch_size, seq_len, n_kv_head, head_dim, dtype):
        # FA3 风格 (B, T, H, D) 布局
        self.k = torch.zeros(batch_size, seq_len, n_kv_head, head_dim, dtype=dtype)
        self.v = torch.zeros_like(self.k)
        self.cache_seqLens = torch.zeros(batch_size, dtype=torch.int32)

    def prefill(self, k_new, v_new, prompt_lens):
        # 把 prompt 的 K/V 写入 cache
        for b in range(self.k.shape[0]):
            n = prompt_lens[b]
            self.k[b, :n] = k_new[b, :n]
            self.v[b, :n] = v_new[b, :n]
            self.cache_seqLens[b] = n

    def duplicate(self, num_samples):
        # 把 batch=1 的 cache 复制到 num_samples 行 (GRPO 用)
        self.k = self.k.repeat_interleave(num_samples, dim=0)
        self.v = self.v.repeat_interleave(num_samples, dim=0)
        self.cache_seqLens = self.cache_seqLens.repeat_interleave(num_samples, dim=0)
```

8.3 Flash Attention 3: 从算法到工程

Flash Attention (Dao et al. 2022) 的核心思想：通过 tiling 把 attention 计算分成小块，在 SRAM（片上高速缓存）内完成 softmax 与加权求和，避免反复读写 HBM（显存）。FA1 把

attention 的 HBM 访问从 $O(T^2)$ 降到 $O(T^2/M)$ (M 是 SRAM 大小)，FA2 优化了 IO 模式，FA3 进一步利用 H100 的异步数据移动与 TMA (Tensor Memory Accelerator)。在 H100 上，FA3 比 FA2 快约 $1.5-2\times$ ，比朴素 SDPA 快 $5-10\times$ 。

nanochat 的 flash_attention.py 提供了 FA3/SDPA 的统一接口，自动检测 GPU 架构：SM9 (H100/H200) 用 varunneal/flash-attention-3 kernel，其他架构用社区 FA2 kernel 或 SDPA 回退。关键约束：仅 bf16 才能用 FA3 (FA3 的 e4m3 内部计算对 fp16 支持不完整)。推理时 (带 kv_cache 参数) 调用 flash_attn_with_kvcache，自动处理增量 K/V。

```
# flash_attention.py 简化接口
def flash_attn_func(q, k, v, causal=True, window_size=None):
    if supports_fa3() and q.dtype == torch.bfloat16:
        return flash_attn_3(q, k, v, causal=causal, window_size=window_size)
    elif supports_fa2():
        return flash_attn_2(q, k, v, causal=causal, window_size=window_size)
    else:
        # SDPA 回退 (手动管 KV cache + 滑窗 mask)
        attn_mask = build_causal_mask(q.shape[-2], k.shape[-2], window_size)
        return F.scaled_dot_product_attention(q, k, v, attn_mask=attn_mask)

def flash_attn_with_kvcache(q, k_cache, v_cache, k=None, v=None, ...):
    # 推理用：从 cache 读历史 K/V，与新 K/V 拼接后做 attention
    return flash_attn_3_with_kvcache(q, k_cache, v_cache, k=k, v=v, ...)
```

8.4 工具调用状态机

nanochat 的特色之一是把"工具调用"集成到推理引擎本身，而非外层 wrapper。具体地，模型在生成过程中可以输出 `<|python_start|>...<|python_end|>` 触发 Python 代码执行，执行结果通过 `<|output_start|>...<|output_end|>` 回喂模型，让模型"看到"工具输出并继续生成。这一设计让模型能在数学题中"用 Python 计算中间结果"，显著提升 GSM8K 等任务的准确率。

工具调用状态机：Python REPL 工具的 token 级状态转移

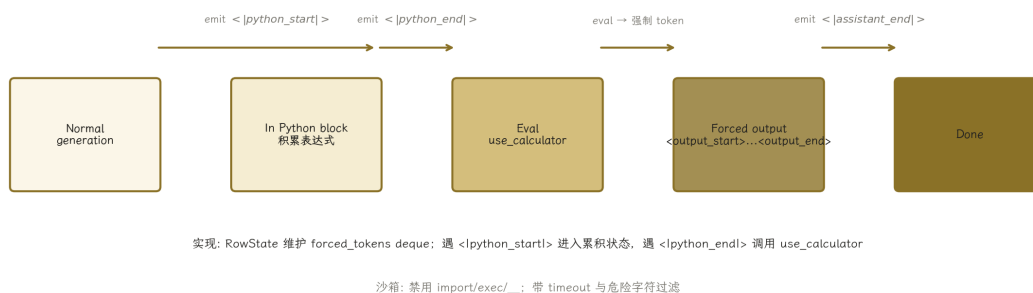


图 8-2 工具调用状态机：Python REPL 工具的 token 级状态转移

Engine.generate 的主循环每步处理一个 token：(1) sample_next_token：根据 logits 采样下一个 token (temperature/top_k/argmax)；(2) 检测 forced_tokens 队列——如果工具输出待注入，强制输出对应 token；(3) 检测状态切换：遇 `<|python_start|>` 进入 python 块累积表达式 token；遇 `<|python_end|>` 调用 use_calculator eval，把结果作为

<|output_start|>...<|output_end|> 注入 forced_tokens 队列；（4）遇 <|assistant_end|> 或 <|bos|> 标记该行完成。

```
# engine.py 主循环 (简化)
class Engine:
    def generate(self, prompt, num_samples=1, temperature=1.0, top_k=50, max_new_tokens=512):
        # 1. prefill prompt (batch=1)
        prefill_cache = KVCache(1, prompt_len + max_new_tokens, ...)
        prefill(prefill_cache, prompt)

        # 2. 复制 KV 到 num_samples 行 (GRPO 用)
        if num_samples > 1:
            prefill_cache.duplicate(num_samples)

        # 3. 每行维护一个 RowState
        rows = [RowState() for _ in range(num_samples)]

        for step in range(max_new_tokens):
            # 调用 FA3 with kvcache 一次生成 num_samples 个 token
            next_tokens = sample_next_token(model, prefill_cache, temperature, top_k)

            for i, row in enumerate(rows):
                if row.forced_tokens:
                    # 强制输出工具结果 token
                    next_tokens[i] = row.forced_tokens.popleft()

                if next_tokens[i] == python_start_id:
                    row.in_python_block = True
                    row.python_tokens = []
                elif next_tokens[i] == python_end_id and row.in_python_block:
                    # 触发 use_calculator
                    code = tokenizer.decode(row.python_tokens)
                    output = use_calculator(code)
                    # 把 output 注入 forced_tokens 队列
                    output_ids = tokenizer.encode(output)
                    for tok in [output_start_id] + output_ids + [output_end_id]:
                        row.forced_tokens.append(tok)
                    row.in_python_block = False
                elif next_tokens[i] in (assistant_end_id, bos_id):
                    row.done = True

            if all(row.done for row in rows):
                break
```

8.5 use_calculator: 沙箱化 Python 执行

use_calculator 是 nanochat 的工具执行函数，本质上是“受限 Python 解释器”。安全设计：禁用 __（双下划线，防止访问私有属性）、import、exec、eval、subprocess 等危险模式；带 timeout 与危险字符过滤；支持 .count() 等字符串方法用于简单计算。注意这不是“安全沙箱”——只防意外误用，不防恶意构造。生产环境需要更强的隔离。

```
def use_calculator(code, timeout=5.0):
    # 危险字符过滤
    forbidden = ['__', 'import', 'exec', 'eval', 'subprocess',
                 'os.system', 'shutil', 'open']
    for kw in forbidden:
```

```

    if kw in code:
        return f'Error: forbidden keyword {kw}'

# 带超时的 eval
try:
    result = eval_with_timeout(code, timeout=timeout)
    return str(result)
except Exception as e:
    return f'Error: {e}'

```

安全提示：沙箱不是生产级隔离

工具调用的"软"边界：use_calculator 用 Python eval 执行模型生成的代码，在数学题场景下足够（模型生成的代码通常是 $1+2*3$ 这种纯算术）。但若模型被恶意 prompt 引导生成 ``__import__("os").system("rm -rf /")``，沙箱会拦截。不过这只是"防意外"级别的安全，真正的生产环境需要 Docker 容器、syscall 过滤、网络隔离等多层防护。

8.6 推理 rooeline: MFU 与 MBU

推理性能的度量比训练更复杂。训练用 MFU（Model FLOPs Utilization）——实际 FLOPs / 峰值 FLOPs。推理则需要分两个阶段度量：（1）prefill 阶段是 compute-bound，用 MFU 度量算力利用率；（2）decode 阶段是 memory-bound，每步只算一个 token 但需要读全部 KV cache，用 MBU（Memory Bandwidth Utilization）度量带宽利用率。

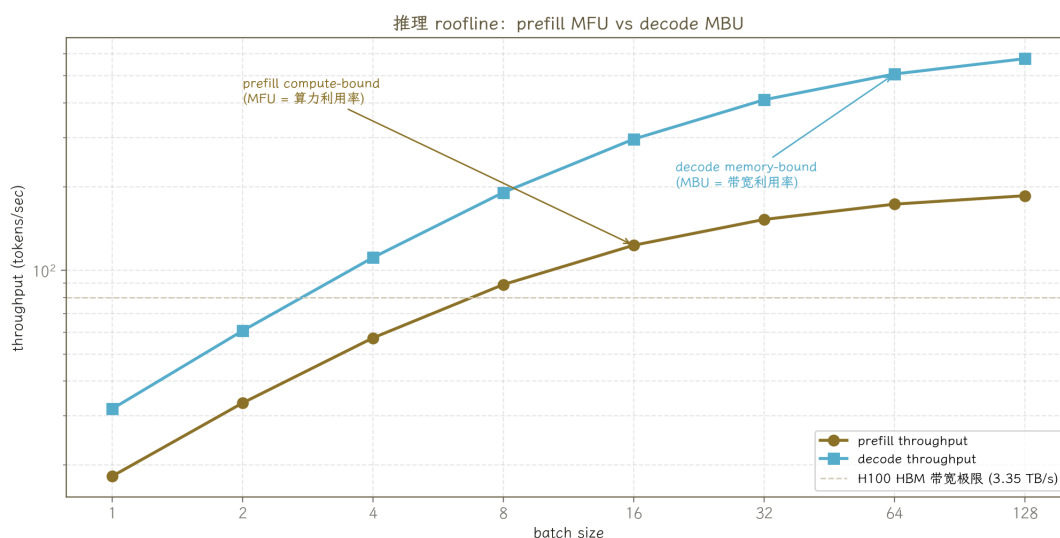


图 8-3 推理 rooeline: prefill MFU vs decode MBU 的 batch 扫描

关键观察：decode 阶段 throughput 在 batch 较小时随 batch 线性增长，但当 batch 大到 saturate HBM 带宽（H100 约 3.35 TB/s）后趋于稳定——这就是 memory-bound 的特征。infer_bench.py 扫 batch=1/8/32/128 画出 latency↔throughput 曲线，让工程师直观看到"哪个 batch 是最优"。

```

# infer_bench.py 核心思路
def benchmark_decode(model, kv_cache, batch_sizes=[1, 8, 32, 128]):
    results = []
    for bs in batch_sizes:
        torch.cuda.synchronize()

```

```

t0 = time.time()
for _ in range(100):
    next_tokens = model.decode_step(kv_cache, ...)
    torch.cuda.synchronize()
    elapsed = time.time() - t0

tokens_per_sec = bs * 100 / elapsed
# MBU = 实际带宽 / 峰值带宽
bytes_read = 2 * seq_len * n_kv_head * head_dim * 2 * bs * 100
mbu = bytes_read / elapsed / 3.35e12 # H100 HBM 3.35 TB/s
results.append({'batch': bs, 'tok/s': tokens_per_sec, 'MBU': mbu})
return results

```

8.7 chat_cli.py: 交互式 REPL

把 Engine + KVCache + 工具状态机封装起来，得到 chat_cli.py——nanochat 的交互式对话接口。核心逻辑只有 100 行：维护一个对话历史，每次用户输入后 append

<|user_start|>...<|user_end|><|assistant_start|>，调 engine.generate 流式打印 assistant 输出，直到 <|assistant_end|>。

```

# chat_cli.py 简化
def main():
    tokenizer = get_tokenizer()
    model = load_model('sft', device='cuda', dtype=torch.bfloat16)
    engine = Engine(model, tokenizer)

    history = []
    while True:
        user_input = input('User: ')
        history.append({'role': 'user', 'content': user_input})

        # 渲染为 token id 序列
        conv_ids, _ = tokenizer.render_conversation(history)
        conv_ids += tokenizer.encode_special('<|assistant_start|>')

        # 生成回复
        output_ids = engine.generate(conv_ids, temperature=0.7, top_k=50,
                                     max_new_tokens=512)

        # 解码并打印
        output_text = tokenizer.decode(output_ids)
        print(f'Assistant: {output_text}')
        history.append({'role': 'assistant', 'content': output_text})

```

8.8 小结

本章覆盖了推理引擎的全部关键工程：KV Cache prefill/decode 两阶段、Flash Attention 3 算法与统一接口、工具调用状态机（python REPL 集成到 Engine）、use_calculator 沙箱、推理 roofline 与 MBU 度量、chat_cli.py 的交互式封装。一个核心洞察是：“推理工程的核心是把 $O(L^2)$ 转为 $O(L)$ ，同时把工具调用集成到 token 生成流”——这两点是 LLM 从“实验室模型”走向“可用产品”的关键。

理解推理后，下一步是评估——如何度量训练得到的模型好坏。第八章将聚焦 CORE、ChatCORE、BPB、pass@k 等现代 LLM 评估指标的设计与实现。

第九章 评估体系：CORE、ChatCORE、BPB 与 pass@k

9.1 一道 MMLU 题如何走完整个评估流程

在讲评估理论之前，先用一道具体的 MMLU 多选题展示 nanochat 如何评估模型。这是本章的贯穿示例——后面讲 BPB、CORE、ChatCORE、pass@k 时都会回到这道题，看每个指标如何计算。

```
# 假设的 MMLU 题目：
Question: What is the chemical symbol for gold?
A. Au
B. Ag
C. Gd
D. Go
Answer: A

# Step 1: 构造 few-shot prompt (CORE 5-shot)
# 前 5 个示例 + 当前题 (不带答案)
prompt = """
Question: What is the capital of France?
A. London B. Paris C. Berlin D. Madrid
Answer: B

Question: What is 2 + 2?
A. 3 B. 4 C. 5 D. 6
Answer: B

... (共 5 个示例) ...

Question: What is the chemical symbol for gold?
A. Au B. Ag C. Gd D. Go
Answer: """" ← 这里让模型续写

# Step 2: 模型 forward, 得到 logits
logits = model(prompt_tokens)
# logits.shape = (1, T, vocab_size)

# Step 3: 取最后一个位置的 logits, 看模型对 A/B/C/D 的概率
last_logits = logits[0, -1] # shape: (vocab_size,)
p_A = last_logits[token_id("A")] # 比如 8.5
p_B = last_logits[token_id("B")] # 比如 6.2
p_C = last_logits[token_id("C")] # 比如 2.1
p_D = last_logits[token_id("D")] # 比如 1.8

# Step 4: argmax 选 A (logit 最高)
pred = "A" ← 模型答对

# Step 5: 与 ground truth 对比, 得到 accuracy = 1.0
# 4 选 1 的随机基线 = 0.25
# centered_accuracy = (1.0 - 0.25) / (1 - 0.25) = 1.0 ← 完美答案得满分
```

上面是 categorical 评估。如果换成 GSM8K，则用 generative 评估——让模型自由生成，然后用正则提取 ##### 后的数字与 ground truth 比对。本章逐节讲每个评估指标的设计与计算。

9.2 LLM 评估方法学的演进

语言模型评估方法经历了从"perplexity"到"BPB"再到"CORE/ChatCORE"的演进。perplexity 是最直观的指标——模型在测试集上的负对数似然的指数，越低越好。但 perplexity 有两个问题：

(1) 依赖词表大小——词表越大，单个 token 的 perplexity 越低，跨词表比较不公平；(2) 只反映"语言建模能力"，不反映"实际任务能力"。BPB (Bits Per Byte) 通过把 token loss 转换为"每字节的比特数"解决了第一个问题；CORE (ICL few-shot 评估) 解决了第二个问题——直接度量"模型解决实际任务的能力"。

9.3 BPB：跨词表可比的 perplexity

$$\text{BPB} = (\text{token loss} \times \log_2(V)) \times (\text{token 数} / \text{字节数}) = \text{token loss} \times \text{token 数} \times \log_2(V) / \text{字节数}$$

更直观地：BPB = "描述一字节需要的比特数"，与词表大小无关。nanochat 的 loss_eval.py 实现核心是预先计算每个 token 的字节数 (token_bytes.pt)，在评估时把 token loss 乘以 token_bytes 求和再除以总字节数再除以 $\ln(2)$ 。一个 32K 词表与 100K 词表的 BPB 直接可比，而 perplexity 不可比。

```
# loss_eval.py 核心逻辑
def compute_bpb(model, tokenizer, val_data):
    token_bytes = tokenizer.token_bytes # 每个 token 的字节数
    total_loss = 0.0
    total_bytes = 0.0
    for batch in val_data:
        logits = model(batch)
        loss_per_token = F.cross_entropy(
            logits.view(-1, vocab_size),
            batch.view(-1),
            reduction='none',
        )
        # 转为 nat → bit: × ln(2)^{-1}
        loss_bits = loss_per_token / math.log(2)
        # 每个 token 的字节数 (从 token_bytes 表查)
        bytes_per_tok = token_bytes[batch.view(-1)]
        total_loss += (loss_bits * bytes_per_tok).sum().item()
        total_bytes += bytes_per_tok.sum().item()
    return total_loss / total_bytes
```

9.4 DCLM CORE：ICL few-shot 评估

DCLM (DataCompass) CORE 是 DataCompass 论文 (Li et al. 2024) 提出的预训练模型评估方法。其核心思想：用 In-Context Learning (ICL) few-shot 方式让模型解决三类任务——multiple_choice、schema_completion、language_modeling——三类任务的 centered accuracy 平均为 CORE 分数。相比 perplexity，CORE 直接反映"模型能解决多少实际任务"，与下游应用相关性更强。

9.3.1 few-shot prompt 设计

以 multiple_choice 为例，CORE 用 5-shot prompt：把 5 个示例（问题+选项+答案）拼在前面，然后给一个新的问题让模型预测答案。模型对每个选项 A/B/C/D 计算条件概率，选概率最大的为答案。关键细节：用 Jinja 模板渲染 prompt，找公共前/后缀减少重复计算。

DCLM CORE few-shot prompt 模板示意

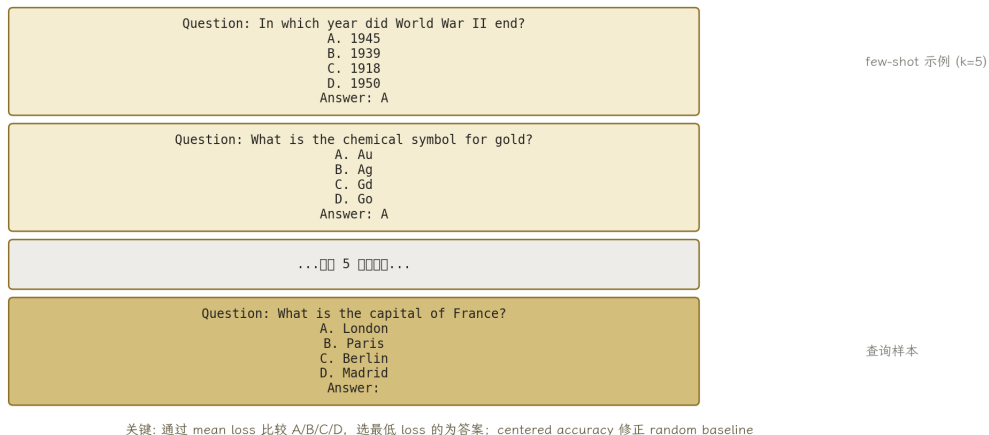


图 9-1 DCLM CORE few-shot prompt 模板示意

9.3.2 centered accuracy

不同任务有不同的随机基线（如 4 选 1 是 25%，4 选 1 含 None 是 0%，自由生成是 0%）。直接平均 accuracy 会让"难度高的任务"拉低总分。centered accuracy 减去随机基线后再平均，让不同任务的贡献更公平：

$$\text{centered_accuracy} = (\text{accuracy} - \text{random_baseline}) / (1 - \text{random_baseline})$$

例如某任务 accuracy=0.5, random_baseline=0.25, 则 centered_accuracy = (0.5-0.25)/(1-0.25) = 0.333。这一归一化让"提升 0.1 在随机基线低的任务上更难、价值更高"这一事实被反映到分数里。

9.5 ChatCORE：对话模型的 5 任务

SFT/RL 训练后，模型变成"对话模型"，需要评估"实际对话能力"而非"语言建模能力"。nanochat 的 ChatCORE 包含 5 个任务，每个对应一种能力维度：

任务	能力维度	评估方式	随机基线	示例题数
ARC-Easy	基础科学常识	4 选 1 categorical	0.25	~2300
ARC-Challenge	高难度科学推理	4 选 1 categorical	0.25	~1200
MMLU	57 学科知识	4 选 1 categorical	0.25	14042
GSM8K	小学数学推理	生成式 + 答案匹配	0	1319
HumanEval	Python 编程	生成式 + 沙箱执行	0	164

表 9-1 ChatCORE 5 任务与能力维度

5 个任务的随机基线不同 (0.25 vs 0)，centered accuracy 让它们可比。ChatCORE 总分 = 5 任务 centered accuracy 的均值。nanochat 报告 d24 模型经过 SFT 后的 ChatCORE 约 0.45，比 SFT 前的 base 模型 (约 0.20) 提升 1.25 倍。

9.6 categorical vs generative 评估

ARC/MMLU 用 categorical 评估——直接看模型对 A/B/C/D 四个选项的 logits，选 argmax 的字母为答案。这种评估方式快速、确定，但只能评估"选择题"类任务。GSM8K/HumanEval 用 generative 评估——让模型自由生成回答，然后用规则或代码执行验证答案。GSM8K 用正则提取#### 数字比对；HumanEval 用沙箱执行模型生成的代码，看是否通过测试。

```
# chat_eval.py 评估框架
def evaluate_chatcore(model, tokenizer):
    results = {}

    # categorical tasks (ARC, MMLU)
    for task_name in ['arc_easy', 'arc_challenge', 'mmlu']:
        acc = evaluate_categorical(model, tokenizer, task_name)
        baseline = 0.25 # 4 选 1
        results[task_name] = (acc - baseline) / (1 - baseline)

    # generative tasks (GSM8K, HumanEval)
    for task_name in ['gsm8k', 'humaneval']:
        acc = evaluate_generative(model, tokenizer, task_name)
        baseline = 0.0
        results[task_name] = (acc - baseline) / (1 - baseline)

    # ChatCORE = mean of 5 centered accuracies
    return sum(results.values()) / 5
```

9.7 pass@k: RL 评估

RL 训练的评估指标与 SFT 不同——RL 是"采样-打分"循环，评估时也用"采样-打分"。pass@k 表示"采样 k 次有至少 1 次正确"的概率：

$$\text{pass@k} = 1 - C(n-c, k) / C(n, k), n=\text{总采样数}, c=\text{正确数}$$

nanochat 在 RL 评估时对每题采 16 个 rollout，pass@1 是"单次答对概率"，pass@16 是"16 次里有至少 1 次对"的概率。pass@16 通常远高于 pass@1，反映"模型有解题能力但不稳定"的状态——RL 训练让 pass@1 接近 pass@16 的过程，就是模型从"偶尔做对"到"稳定做对"的提升过程。

9.8 HumanEval 沙箱执行

HumanEval 是 OpenAI 提出的 Python 编程评估集，每题包含一个函数签名、文档字符串、测试用例。模型生成的代码需要被实际执行，看是否通过所有测试。nanochat 的 execution.py 提供了沙箱执行环境：用 subprocess + rlimit 限制内存 (256MB) + 危险函数过滤。虽然不是生产级沙箱，但对 HumanEval 这类"模型生成的简单函数代码"足够安全。

```
# execution.py 核心思路
def execute_code(code: str, timeout=5.0) -> bool:
```

```

# 拼接 imports + completion + test + check(entry_point)
full_code = '\n'.join([
    'import math, re, collections, itertools, string, ...',
    code,
    test_code, # 题目自带的测试函数
    f'check({entry_point})', # 调用 check 函数
])

# 用 subprocess 执行，限制内存与时间
try:
    result = subprocess.run(
        ['python', '-c', full_code],
        timeout=timeout,
        capture_output=True,
        # rlimit 256MB
        preexec_fn=lambda: resource.setrlimit(
            resource.RLIMIT_AS, (256*1024*1024, 256*1024*1024)
        ),
    )
    return result.returncode == 0
except subprocess.TimeoutExpired:
    return False

```

9.9 评估与训练分布的对齐问题

一个常被忽视的问题：评估数据是否泄漏到训练集？nanochat 的 GSM8K 评估用 openai/gsm8k 的 test 集，训练用 main 集——两者在 HF 上是分开的 split。MMLU 评估用 cais/mmlu 的 test 集，训练用 auxiliary_train——也是分开的。这种严格的“训练-评估分离”是评估结果可信的基础。nanochat 在文档中明确警告：“不要用 MMLU test 集做 SFT 训练数据，否则 ChatCORE 失真”。

工程经验：评估脚本的反评估

一个工程经验：评估脚本本身也要定期“反评估”——构造随机模型或低质量模型跑一次评估，看分数是否落在随机基线附近。如果分数异常高，说明评估脚本可能有 bug（如答案泄露、测试集偏差等）。nanochat 的 centered accuracy 设计就是为了“让随机模型得 0 分”——一个瞎猜的模型在 4 选 1 上得 25%，centered 后是 0，正确反映了“无能力”。

9.10 小结

本章覆盖了 LLM 评估的全部关键指标：BPB（跨词表可比的语言建模能力）、CORE（ICL few-shot 任务能力）、ChatCORE（5 任务对话能力）、pass@k（RL 采样能力）、HumanEval（编程能力）。一个核心观察是：“评估方法学本身在快速演进，perplexity 已被淘汰，CORE 与 ChatCORE 是新标准”，理解这些指标的设计原理比记住分数更重要——它决定了“模型好坏”的判定标准。

理解评估后，下一步是动手——第九章给出 10 个扩展实验，让读者在 d12 小模型上验证本章讲过的设计。

第十章 扩展实验：四类动手实践

10.1 实验准备：d12 小模型与单 GPU

本章所有实验在 d12 小模型上完成，单张 A100/4090 上训练约 5–15 分钟即可跑完一个变体。d12 配置：n_layer=12, n_embd=768, n_head=8, n_kv_head=2, 参数量约 50M；数据用 climbmix-400b-shuffle 的前 8 个分片（~800MB）；训练 2000 步约 5 分钟即可收敛到 BPB $\approx$ 2.65。这一规模适合“实验快速迭代”，比 d24 的 1.5 小时快 20 倍。

```
# 启动 d12 实验
# 在单 A100 80GB 上跑
torchrun --nproc_per_node=1 -m scripts.base_train \
    -- --depth=12 --target-flops=5e15 \
    --device-batch-size=32 \
    --n_data_shards=8 # 只下载 8 个分片

# 或更小规模（如 M1 Max 32GB）
bash runs/runcpu.sh # 用 CPU/MPS 演示
```

10.2 架构与优化器实验

10.2.1 实验 1：词表大小消融

目标：对比 8K / 16K / 32K / 64K 词表对 BPB、训练速度、最终 CORE 的影响。实现：修改 scripts/tok_train.py 的 --vocab_size 参数，分别训 4 个 tokenizer，然后用对应 tokenizer 训 4 个 d12 模型，对比最终 BPB。

```
# 训 4 个不同词表 tokenizer
for V in 8192 16384 32768 65536; do
    python -m scripts.tok_train -- --vocab_size=$V
    python -m scripts.tok_eval -- --vocab=$V
done

# 用每个 tokenizer 训 d12 模型
for V in 8192 16384 32768 65536; do
    torchrun ... -m scripts.base_train \
        -- --depth=12 --vocab_size=$V --output_dir=base_d12_v$V
    python -m scripts.base_eval -- --ckpt=base_d12_v$V
done
```

预期结果：32K 在 BPB 上最优（小词表压缩率不够，大词表 embedding 参数浪费）；64K 训练稍慢（embedding lookup 慢）；8K 在代码与数学上 BPB 显著差于 32K。这个实验让读者直观理解“32K 是 nanochat 的工程最优选择”。

10.2.2 实验 2：LR schedule 对比

目标：在 d12 上对比 cosine / linear warmdown / WSD / constant 4 种 schedule 的 val_bpb 曲线。实现：修改 base_train.py 中的 get_lr_multiplier 函数，分别实现 4 种 schedule。每 50 步记录一次 val_bpb，画出 4 条曲线对比。

```
# 4 种 schedule 实现
def lr_cosine(step, total, warmup, final_frac=0.05):
    if step < warmup: return step / warmup
```

```

    p = (step - warmup) / (total - warmup)
    return final_frac + (1 - final_frac) * 0.5 * (1 + cos(pi * p))

def lr_linear_warmdown(step, total, warmup, warmdown_ratio=0.65):
    if step < warmup: return step / warmup
    stable_end = int(total * (1 - warmdown_ratio))
    if step < stable_end: return 1.0
    p = (step - stable_end) / (total - stable_end)
    return 1.0 - 0.95 * p

def lr_wsd(step, total, warmup, warmdown_ratio=0.65, final_frac=0.05):
    # nanochat 默认
    if step < warmup: return step / warmup
    stable_end = int(total * (1 - warmdown_ratio))
    if step < stable_end: return 1.0
    p = (step - stable_end) / (total - stable_end)
    return 1.0 - (1.0 - final_frac) * p

def lr_constant(step, total, warmup):
    return min(1.0, step / warmup)

```

预期结果：WSD 与 cosine 最终 BPB 接近，但 WSD 在 stable 阶段可以“中途延长”——如果在 step 1500 发现 loss 还在下降，把 stable 阶段延长到 step 2000 即可，而 cosine 必须重新设计 schedule。constant schedule 训练不稳定（后期 loss 震荡），linear warmdown 收敛稍慢但最终 BPB 略差。这个实验让读者理解 WSD 的工程价值。

10.2.3 实验 3：Muon vs AdamW 对比

目标：把 setup_optimizer 中矩阵组改为 AdamW，其他不变，对比收敛速度与最终 bpb。实现：修改 GPT.setup_optimizer，把所有参数都用 kind="adamw"（覆盖默认的 muon），其他超参不变，训 2000 步。

```

# 修改 gpt.py 的 setup_optimizer
def setup_optimizer(self, ...):
    # 默认：矩阵参数走 muon，向量参数走 adamw
    muon_params = []
    adamw_params = []
    for name, p in self.named_parameters():
        if p.dim() == 2 and 'embed' not in name and 'lm_head' not in name:
            muon_params.append(p)
        else:
            adamw_params.append(p)

    # 实验：把 muon_params 也用 adamw
    if USE_ADAMW_ONLY:
        adamw_params.extend(muon_params)
        muon_params = []

    return MuonAdamW([
        {'params': muon_params, 'kind': 'muon', 'lr': matrix_lr},
        {'params': adamw_params, 'kind': 'adamw', 'lr': embedding_lr},
    ], ...)

```

预期结果：AdamW 在前 500 步收敛与 Muon 接近，但 1000 步后 Muon 的 BPB 明显更低（约 0.05–0.10）。最终 2000 步 Muon 的 BPB 约 2.65，AdamW 约 2.72。这个差距在大模型上会更

明显——Muon 在 d24 上比 AdamW 优约 0.15 BPB。这一实验是"理解 Muon 价值"的最直接证据。

10.3 训练策略实验

10.3.1 实验 4：DPO vs 简化 GRPO 对比

目标：把 chat_rl.py 改写为 DPO（需要 reference model + 偏好对数据），在 GSM8K 上对比 pass@k 与样本效率。DPO 的核心思想：用偏好对（chosen, rejected）直接训练 policy，删除 RL 训练循环，把 RL 变成"分类问题"。

```
# DPO 损失函数
def dpo_loss(policy_logits, ref_logits, chosen_ids, rejected_ids, beta=0.1):
    # policy 与 ref 对 chosen/rejected 的对数概率
    policy_chosen_lp = compute_logp(policy_logits, chosen_ids)
    policy_rejected_lp = compute_logp(policy_logits, rejected_ids)
    ref_chosen_lp = compute_logp(ref_logits, chosen_ids).detach()
    ref_rejected_lp = compute_logp(ref_logits, rejected_ids).detach()

    # DPO 目标: log( $\sigma(\beta \cdot (\log\_ratio\_chosen - \log\_ratio\_rejected))$ )
    chosen_ratio = policy_chosen_lp - ref_chosen_lp
    rejected_ratio = policy_rejected_lp - ref_rejected_lp
    return -F.logsigmoid(beta * (chosen_ratio - rejected_ratio)).mean()
```

实现：从 GSM8K RL 的 rollout 数据构造偏好对——同一题的 16 个 rollout 中，答对的为 chosen，答错的为 rejected。然后训 DPO 1000 步，对比 GRPO 1000 步的 pass@1 与 pass@16。预期：DPO 在样本效率上更优（每步只用 2 个 rollout 而非 16 个），但 pass@16 提升幅度略小。

10.3.2 实验 5：SFT 数据混合比例扫描

目标：改 chat_sft.py 的 --mmlu-epochs / --gsm8k-epochs，观察各 ChatCORE 子项此消彼长。实验设置：默认 SmolTalk + MMLU×3 + GSM8K×4，对比 4 个变体：（1）MMLU×1 + GSM8K×1（轻量化）；（2）MMLU×5 + GSM8K×1（重学科轻数学）；（3）MMLU×1 + GSM8K×8（轻学科重数学）；（4）MMLU×3 + GSM8K×4（默认）。

预期结果：变体 2 的 MMLU 子项高但 GSM8K 低；变体 3 的 GSM8K 高但 MMLU 低；变体 1 整体 ChatCORE 最低；变体 4（默认）整体最优——但如果你只关心 GSM8K，变体 3 更好。这个实验揭示"数据混合是 task-specific 的工程权衡"，没有"全局最优配方"。

10.3.3 实验 6：添加新能力 (strawberry 数 r)

目标：参考 Karpathy 的 Guide "counting r in strawberry"，构造一个小型 SFT 数据集教模型数数字母，跑 SFT + RL，观察 ChatCORE 子项变化。数据生成：用 Python 脚本生成 5000 条样本，格式为"数 strawberry 中的 r: 5 个 → 答: 3 个"。

```
# 生成 strawberry r 计数数据
import random, string
data = []
for _ in range(5000):
    word = random.choice(['strawberry', 'raspberry', 'mississippi',
                          'banana', 'apple', 'orange', 'cherry'])
    target_char = random.choice('rseau')
    count = word.count(target_char)
    data.append({
```

```

        'role': 'user',
        'content': f"How many '{target_char}' in '{word}'?"
    })
    data.append({
        'role': 'assistant',
        'content': f"There are {count} '{target_char}'s in '{word}'."
    })

# SFT 训练
torchrun ... -m scripts.chat_sft -- --extra_data=letter_count.json

```

预期：训练后 strawberry 数 r 任务的准确率从接近 0 提升到 95%+。但 ChatCORE 的其他子项基本不变（或略降，因新数据挤占了部分训练步数）。这个实验示范"如何给小模型加新能力"的完整闭环：数据生成 → SFT → 评估。

10.4 推理优化实验

10.4.1 实验 7：int8/int4 量化推理

目标：训练完成后把权重转 int8/int4（用 torchao 或自写），用 infer_bench.py 测 MBU/tok/s/VRAM。量化是把 fp16 权重压缩到 int8/int4，显存降为 1/2 或 1/4，但精度损失需要评估。

```

# 使用 torchao 做 int8 量化
from torchao.quantization import quantize_

model = load_model('sft', dtype=torch.bfloat16)

# int8 weight-only 量化
quantize_(model, Int8WeightOnlyConfig())

# 或 int4
quantize_(model, Int4WeightOnlyConfig(group_size=128))

# 量化后跑推理 benchmark
python -m scripts.infer_bench -- --ckpt=sft_int8

```

预期：int8 显存降 50%，tok/s 几乎不变（H100 int8 GEMM 与 bf16 速度相当）；int4 显存降 75%，tok/s 略升（更小 KV cache 读取）；但 int4 的 ChatCORE 可能损失 1-3%，int8 几乎无损。这一实验让读者理解"量化是显存-精度权衡"。

10.4.2 实验 8：batch 扫描与 KV cache 显存

目标：用 infer_bench.py 扫 batch=1/8/32/128/256，画 latency↔throughput 曲线，观察"哪个 batch 是最优"以及"超过多少 batch 显存爆"。这一实验直接验证第七章讲的推理 roofline 理论。

```

# 扫不同 batch
for bs in 1 8 32 128 256 512; do
    python -m scripts.infer_bench \
        -- --ckpt=sft --batch_size=$bs \
        --max_seq_len=2048 --n_warmup=5 --n_iter=50
done

# 输出 CSV: batch, latency_ms, throughput_tok_s, MBU, vram_gb

```


10.4.3 实验 9：KV cache 压缩

目标：实验"只保留最近 N 个 KV"的压缩策略（sliding window KV），对比 N=512/1024/2048 的 perplexity 与显存。实现：在 engine.py 的 KVCache 类加一个 max_cache_len 参数，超过时丢弃最早的 KV。

```
# 修改 KVCache 类
class KVCache:
    def __init__(self, ..., max_cache_len=None):
        self.max_cache_len = max_cache_len or seq_len

    def append(self, k_new, v_new):
        # 加新 KV
        self.k[..., cache_seqLens:cache_seqLens+T] = k_new
        self.v[..., cache_seqLens:cache_seqLens+T] = v_new
        self.cache_seqLens += T

        # 滑动窗：超过 max 时丢弃最早的
        if self.cache_seqLens[0] > self.max_cache_len:
            overflow = self.cache_seqLens[0] - self.max_cache_len
            self.k = torch.roll(self.k, -overflow, dims=1)
            self.v = torch.roll(self.v, -overflow, dims=1)
            self.cache_seqLens -= overflow
```

预期：max_cache_len=2048 时与无限制一致；max_cache_len=512 时显存降 75%，但长文本任务的 perplexity 略升——这一实验让读者理解"KV cache 的用多少留多少"是关键工程决策。

10.5 工程优化实验

10.5.1 实验 10：tokenizer split pattern 消融

目标：修改 SPLIT_PATTERN 中 `\p{N}{1,2}` 为 `{1,3}`（GPT-4 默认），测数字 token 占比与数学任务表现。实现：修改 tokenizer.py 的 SPLIT_PATTERN 常量，重新训 tokenizer，然后用新 tokenizer 训 d12 模型，对比 GSM8K pass@1。

```
# 修改 tokenizer.py
SPLIT_PATTERN = r"""
    's|'t|'re|'ve|'m|'ll|'d
    | ?\p{L}+
    | ?\p{N}{1,3} # ← 改这里：{1,2} → {1,3}
    | ?[^\s\p{L}\p{N}]+
    | \s+(?!\s)
    | \s+
    """
```

预期：`{1,3}` 在数字密集文本（如数学公式、代码）上压缩率略好（约 5%），但 GSM8K pass@1 几乎不变（数学任务的关键 token 是符号而非数字）。这个实验让读者理解 nanochat 选 `{1,2}` 的实证依据。

10.5.2 实验 11：滑动模式消融

目标：把 window_pattern 从 "SSSL" 改为 "L"（全上下文）、"SL"（交替）、"SSSL"（nanochat 默认），对比 prefill FLOPs、MFU、CORE 与推理 KV 读取字节（kv_read_bytes）。


```
# 修改 gpt.py 的 _compute_window_sizes
WINDOW_PATTERNS = {
    'full': 'L',
    'alternate': 'SL',
    'nanochat': 'SSSL',
}

for pattern_name, pattern in WINDOW_PATTERNS.items():
    cfg.window_pattern = pattern
    train_d12(cfg)
    eval_d12(cfg) # 记录 prefill_flops, mfu, core, kv_read_bytes
```

10.5.3 实验 12：FP8 vs BF16 训练对比

目标：在 d12 模型上对比 FP8 与 BF16 训练的速度 (tok/s) 与最终 BPB。实现：在 base_train.py 加 --fp8 参数（已有），分别训两个 d12 模型，对比训练日志。

```
# FP8 vs BF16 对比
torchrun ... -m scripts.base_train -- --depth=12 --fp8=false --output_dir=d12_bf16
torchrun ... -m scripts.base_train -- --depth=12 --fp8=true --output_dir=d12_fp8

# 对比训练日志中的 tok/s 与 final BPB
grep 'tok/s' d12_bf16/log.txt | tail
grep 'tok/s' d12_fp8/log.txt | tail
```

预期：FP8 在 H100 上比 BF16 快 1.5–1.8×，最终 BPB 略差（约 0.01–0.03），因为 FP8 的动态范围与精度都低于 BF16。在 A100 上 FP8 不可用（A100 不支持 e4m3/e5m2 格式），需要 BF16 训练。

10.6 实验记录与对比

做完所有实验后，建议把结果汇总到一张表格，便于横向对比：

实验编号	变量	默认值	实验值	BPB 变化	ChatCORE 变化	其他
1	vocab_size	32768	8K/16K/64K	-0.05 ~ +0.05	—	64K 训练慢
2	LR schedule	WSD	cosine/linear/constant	< 0.01	—	const 不稳定
3	optimizer	Muon+AdamW	纯 AdamW	+0.07	-1.5%	Muon 明显优
4	RL algo	GRPO	DPO	—	pass@1 相近	DPO 样本效率高
5	SFT mix	M3+G4	M1+G1 / M5+G1 / M1+G8	—	±3%	无全局最优

6	新能力	—	+letter_count	—	strawberry 0→95%	其他子项略降
7	量化	bf16	int8 / int4	—	int8≈0%, int4-2%	int4 显存 1/4
9	KV cache	full	滑窗 N=512	+0.05	—	显存降 75%
10	split pattern	{1,2}	{1,3}	< 0.01	—	GSM8K 几乎不变
11	window pattern	SSSL	L / SL	< 0.01	—	L 全上下文慢
12	FP8 vs BF16	BF16	FP8	+0.02	—	FP8 快 1.5×

表 10-1 12 个扩展实验的预期结果汇总

10.7 实验心得

做完这 12 个实验后，你应该对 nanochat 的设计有"手到擒来"的理解。几个关键心得：（1）每个"现代 LLM 技巧"都有 ablation 支持——不是凭直觉堆叠；（2）小模型上的 ablation 结果可以外推到大模型（scaling laws 让结果有预测性）；（3）工程权衡无处不在——没有"全局最优"，只有"task-specific 最优"；（4）nanochat 的价值不在"发明新方法"，而在"用最小代码组合已验证方法"，让读者能在一个可读的代码库内完成所有 ablation。

一个延伸方向：把你做的 ablation 结果提交到 nanochat 的 GitHub Discussions，社区会很快反馈你的实验是否符合主流观察，甚至会启发新的 ablation 方向。这种"开源实验社区"模式是 modded-nanoGPT speedrun 排行榜背后的驱动力，也是 nanochat 项目能持续演化的关键。

第十一章 论文阅读路径与进一步学习

11.1 一份推荐的 8 周论文阅读计划

在讲论文分组之前，先给出一份具体的 8 周阅读计划。假设你刚读完本课程前 10 章，想用 8 周时间把 11 篇核心论文读一遍，下面是每周读什么、读完应该理解什么、配套实验做什么。

```
# 第 1 周: Scaling Laws 三部曲 (5 小时)
# Kaplan 2020 (1h) → Chinchilla 2022 (2h) → Power Lines 2025 (2h)
# 读完应该理解: 为什么 nanochat 用 target_param_data_ratio=12
# 配套实验: 在 d12 上跑 scaling_laws.sh, 扫 5 个 depth 看参数-数据比

# 第 2 周: 优化器基础 (4 小时)
# AdamW 2017 (1h) → Muon 博客 (1.5h) → Polar Express 2025 (1.5h)
# 读完应该理解: 为什么 Muon 比 AdamW 收敛快
# 配套实验: ch9 实验 3, d12 上 Muon vs AdamW, 画 val_bpb 曲线

# 第 3 周: 架构演化 (5 小时)
# Transformer 2017 (1h) → RoPE 2021 (1.5h) → QK-Norm 2024 (1h) → Zoology 2024 (1.5h)
# 读完应该理解: 为什么 nanochat 用 ReLU2 而非 SwiGLU
```

配套实验：ch9 实验 11，d12 上 SSSL vs L vs SL 滑窗模式

第 4 周：RLHF 谱系 (5 小时)

PPO 2017 (2h) → GRPO 2024 (1h) → DPO 2023 (2h)

读完应该理解：为什么 nanochat 简化为"GRPO in quotes"

配套实验：ch9 实验 4，d12 上 DPO vs 简化 GRPO

第 5 周：评估方法学 (3 小时)

DCLM CORE 2024 (3h)

读完应该理解：为什么 BPB 跨词表可比，CORE 反映实际任务能力

配套实验：跑 base_eval.py，对比 d12 base 与 sft 的 ChatCORE

第 6-8 周：综合项目

选一个方向做综合实验：

- # - 中文 LLM：用 ch2.7 + ch3.9 + ch9.5 的方法训一个 d12 中文模型
- # - 量化推理：用 ch9.7 + ch9.9 做完整的 int8/int4 推理 pipeline
- # - 新能力：参考 ch9.6 教模型数 r in strawberry
- # - 论文复现：选一篇 NorMuon / MuonEq 论文，在 d12 上做 ablation

上面这份 8 周计划对应下面的 4 个主题组。按周读、按周配套实验，能让你在两个月内完成从"读懂 nanochat 代码"到"做出自己的 ablation 实验"的跃迁。下面逐组列出每篇论文的核心贡献、难度、推荐阅读顺序。

11.2 主题组 1: Scaling Laws 基础

Scaling Laws 是理解 LLM 训练"参数化决策"的钥匙。从 Kaplan 的原始论文开始，看 Chinchilla 如何推翻 Kaplan 的结论，再看 Power Lines 如何把 batch size 纳入 scaling，形成完整的演进脉络。

序号	论文	年份	核心贡献	难度	阅读顺序
1	Kaplan et al. Scaling Laws for Neural LMs	2020	建立 $\text{loss} \propto C^{-\alpha}$ 幂律， $D/N \approx 1.5$	★★★	先读
2	Hoffmann et al. Chinchilla	2022	推翻 Kaplan， $D/N \approx 20$ 才是 compute-optimal	★★★	接着
3	Bjorck et al. Power Lines	2025	$B \propto D^{0.383}$ ，batch size 也应随数据量扩展	★★★ ★	最后

表 11-1 Scaling Laws 基础论文

Kaplan 论文的精华在前 5 节——参数量 N、数据量 D、算力 C 三个维度的 scaling 关系。Chinchilla 论文的关键在 Section 4——通过 400+ 次实验拟合出"compute-optimal frontier"。Power Lines 论文需要一定的优化理论基础，但 Section 3 的实验数据非常直观。读完这 3 篇，你会理解 nanochat 的 base_train.py 中"从 --depth 推导所有超参"的设计来源。

11.3 主题组 2: 优化器

从 Adam 到 Muon，优化器的演进反映了"梯度方向选择"这一基础问题的深入研究。Adam 是基础，Muon 是 2024 年的新突破，Polar Express、NorMuon、MuonEq 等是 Muon 的工程改进。

序号	论文	年份	核心贡献	难度
4	Loshchilov & Hutter. Decoupled Weight Decay Regularization (AdamW)	2017	把 weight decay 从梯度更新解耦，让 wd 在自适应优化器中正确工作	★★
5	Jordan et al. Muon Optimizer Blog	2024	提出"梯度正交化"更新方向，比 Adam 收敛更快	★★★★ (博客)
6	Amsel et al. Polar Express	2025	用 5 次迭代高精度逼近正交化，替代 Newton-Schulz	★★★★★
7	NorMuon	2025	factored second moment 方差归约，让 Muon 数值稳定	★★★★★
8	MuonEq	2025	行均衡：按行范数 rescale 谱条件	★★★★★

表 11-2 优化器演进论文

推荐顺序：先读 AdamW（经典易懂）→ 看 Muon 博客（直观理解"为什么要正交化"）→ 然后读 Polar Express（数学细节）→ NorMuon / MuonEq 是改进论文，可以略读。读完这组，你会理解 nanochat 的 optim.py 中"Muon 融合核"每个步骤的设计动机。

11.4 主题组 3：架构

从原始 Transformer 到现代 LLM 架构，关键变化是位置编码、归一化、激活函数、注意力机制。下面是 4 篇必读的架构演进论文。

序号	论文	年份	核心贡献	难度
9	Vaswani et al. Attention Is All You Need	2017	原始 Transformer，绝对位置编码 + Multi-Head Attention	★★
10	Su et al. RoFormer (RoPE)	2021	旋转位置编码，相对位置 + 长度外推	★★★★
11	He et al. Zoology (ResFormer)	2024	Value Embedding 残差注入，跨越多层的"输入相关"信号	★★★★
12	Henry et al. QK-Norm	2024	Query-Key 归一化，稳定 attention 训练	★★★★

表 11-3 架构演进论文

推荐顺序：原始 Transformer（经典）→ RoPE（理解现代位置编码）→ QK-Norm（理解训练稳定技巧）→ Zoology（理解 ResFormer VE 设计）。读完这组，你会理解 nanochat 的 gpt.py 中apply_rotary_emb、CausalSelfAttention、VE 注入等所有架构细节的来源。

11.5 主题组 4：RL 与评估

RLHF/GRPO/DPO 是对齐方法的核心，CORE/BPB 是评估方法的核心。下面是 4 篇必读论文。

序号	论文	年份	核心贡献	难度
13	Schulman et al. PPO	2017	Proximal Policy Optimization, clip ratio 防止策略更新过大	★★★ ★
14	DeepSeek. GRPO	2024	Group Relative Policy Optimization, 删除 critic, 组内相对优势	★★★★
15	Rafailov et al. DPO	2023	Direct Preference Optimization, 把 RL 变成分类问题	★★★★ ★
16	Li et al. DCLM CORE	2024	ICL few-shot 评估方法, centered accuracy	★★★★

表 11-4 RL 与评估论文

推荐顺序：PPO（理解 RLHF 基础）→ GRPO（理解简化路径）→ DPO（理解更激进的简化）→ DCLM CORE（理解评估方法学）。读完这组，你会理解 nanochat 的 `chat_rl.py` 与 `core_eval.py` 的设计来源。

11.6 进一步学习路径

掌握 nanochat 后，下一步可以探索几个方向：

11.6.1 从 nanochat 到 torchtitan

torchtitan（PyTorch 官方）是更大规模的分布式训练框架，支持 70B+ 模型的 FSDP/Tensor Parallel/Pipeline Parallel。nanochat 的“单节点 8 GPU”上限在 torchtitan 中可以扩展到“多节点 100+ GPU”。推荐学习路径：先跑通 nanochat 的 d24 → 在 torchtitan 中复现 Llama 3 8B → 尝试 Llama 3 70B。

11.6.2 从 speedrun 到 leaderboard

nanochat 维护的“GPT-2 speedrun”排行榜（dev/LEADERBOARD.md）是社区驱动的工程优化竞赛。当前记录 1.65 小时（CORE 0.2626），起点 3.04 小时。推荐参与方式：在 d12 小模型上做你的优化（如更激进的 FP8、新优化器、新架构变体），在 speedrun 模式下测时间，提交到排行榜。

11.6.3 从 nanochat 到 DeepSeek/Llama

nanochat 的代码约 7000 行，DeepSeek-V3 与 Llama 3 的工程代码量约 50K-100K 行——10 倍的复杂度主要来自：分布式通信（多节点 NCCL）、混合精度（FP8 + pipeline）、架构变体（MLA、SwiGLU、RoPE scaling）、数据预处理（dedup、filter、tokenize）。推荐路径：读懂 nanochat → 读 DeepSeek-V3 论文（架构细节）→ 读 Llama 3 论文（数据细节）→ 尝试在小规模上复现 DeepSeek 的 MLA 或 Llama 3 的数据 pipeline。

11.6.4 进阶话题

几个值得深入探索的前沿话题：（1）长上下文：YaRN、Pi、LongRoPE 等长度外推方法，让模型支持 100K+ 上下文；（2）混合专家（MoE）：DeepSeek-V3、Mixtral 的稀疏激活，让模型

在不增加 FLOPs 的情况下扩大参数量；（3）推理时计算：o1/R1 的"thinking tokens"让推理时多算几步提升复杂推理能力；（4）多模态：把 LLM 扩展到 vision/audio，关键在 tokenizer 与 cross-attention 设计。

11.7 课程总结

本课程以 nanochat 为底座，覆盖了现代 LLM 工程的全栈实现：分词器、模型架构、预训练、SFT、RL、推理、评估。一个核心洞察是：现代 LLM 工程不是"魔法"，而是大量"经过 ablation 验证"的工程细节的组合——RoPE、QK-Norm、GQA、ReLU²、Muon、Scaling Laws、ZeRO-2、FP8、KV Cache、CORE 等每一个设计都有论文或社区实验支持。理解这些设计的来源，比记住"代码应该这样写"更重要。

一个常被忽视的事实是：现代 LLM 工程能力"在社区"而非"在公司"。Karpathy 的 nanoGPT、Keller Jordan 的 modded-nanoGPT、DeepSeek 的开源论文——这些社区工作让"训练小模型"的技术门槛持续降低。完成本课程后，建议你主动参与社区：在 GitHub 提 issue、在 Discussions 分享你的 ablation、在 Discord 与其他研究者交流。开源社区是这一轮 AI 浪潮的最大民主化力量。

最后，希望本课程能成为你认识现代 LLM 工程、动手实现、参与开源社区的一份扎实起点。nanochat 是 2025 年的"最小全栈"，但 LLM 工程还在快速演化——明年的 nanochat 可能完全不同，但其中蕴藏的工程思想与 ablation 精神会持续影响这一领域。